R for Health Data Science

Heather Waddell Alexandra Richards Clarisse de Vries Jocelyn Friday Frederick Ho

2026-03-18

Contents

About This Ebook

This is a companion ebook for the MED5686 - R for Health Data Science course in School of Health and Wellbeing, University of Glasgow.

How to Use This Ebook

A box in this colour will often come in two sections: the top will be example code that you should be able to copy into your own R programme and run, and under the dashed line, the output of that code.

A box in this colour will include suggestions for things to try to help improve your understanding of the previous section.

The following text formations have been used throughout the document to help with understanding:

- Bolded words, such as **this**, either refer to a specific R package, which will have an associated glossary entry, or the word(s) will refer to a specific concept.
- Monospace font words, such as `this`, indicate code. These words may or may not be black.
- Italicised words, such as *this*, indicate datasets that will be used throughout this course.

Data Sets

This document was built with the intention of acting as an introduction to R. To do that, it draws upon data provided and modified from the following sources:

- The '*02-hrqol_data.csv*' is a provided, simulated dataset used in Chapters 2 to practice importing into R.
- The '*who*'[1] dataset is an inbuilt dataset for the list of countries and is used in Chapter 3.
- The '*03-diabetic_demog.csv*' dataset is a provided dataset used in Chapter 3 and 5 for merging.
- The '*03-diabetic_labs_final_with_missing.csv*' dataset is a provided dataset used in Chapters 3 and 5 for missing data and merging.
- The '*iris*' dataset is an inbuilt dataset for simple classifications and used in Chapter 4 for data visualisation.

[1] The capitalisation here is to match the name of the dataset within the survival package.

Table 1: A description of the variables available within the diabeticDemog dataset.

Variable	Description
id	Patient id
age	Age at diagnosis
Gender	Patient's gender
BMI	Patient BMI at baseline
SIMD	Patient SIMD quintile at baseline

Table 2: A description of the variables available within the diabeticLabs dataset.

Variable	Description
id	Patient id
SampleDate	The date the blood sample was taken
SampleTime	The time on a 24-hour scale the blood sample was taken
Chol	Cholesterol (mmol/L)
TG	Triglycerides (mmol/L)
HDL	HDL (mmol/L)
LDL	LDL (mmol/L)
Cr	Serum creatinine (ţmol/L)
BUN	Blood urea nitrogen (mmol/L)

- The '*06-alert_rct.dta*' dataset is a provided dataset on a real-world randomised controlled trial data (Holdaas et al. 2003) and used in 6 for survival analysis
- The '*06-understanding_soc.csv*' dataset is a provided dataset on a observational, prospective cohort study (Buck & McFall 2012) used in Chapters 6 and 7 for various advanced topic in regression analysis, and causal inference.
- The '*07-iv.csv*' dataset is a provided dataset on a simulated intervention to practice instrumental variable analyss in Chapter 7.

The *03-diabetic_demog.csv* and *03-diabetic_labs_final_with_missing.csv* were created by combining information from the diabetic dataset in the survival package and the 'The Health Test by Blood Dataset' (Anjali n.d.) for predicting the development of diabetes mellitus (DM). The variables for *diabeticDemog* are described in Table 1 and the variables for *diabeticLabs* are described in Table 2. The provided datasets are available on the course Moodle page.

It is important to note that sex and gender are not the same. In electronic patient records, biological sex is usually recorded and used for analysis. In this case, gender was used, since this is what Anjali (n.d.) recorded in the 'The Health Test by Blood' dataset. Scottish Index of Multiple Deprivation (SIMD) was added to the diabeticDemog dataset for some local flair. SIMD is an area-based measurement of socioeconomic deprivation assigned to residents of Scotland according to where they live. Scottish residents' SIMD status is calculated by the Scottish Government using thirty-one indicators from seven different aspects of deprivation: income, employment, health, education, housing, geographic access, and crime. The indicators are combined using a weighted sum to create a single index, providing a relative ranking for each small geographic area in Scotland. The areas average approximately 800 individuals (The Scottish Government 2012). It is important to note that SIMD can only measure the level of deprivation in an area, not an individual's level. The absence of deprivation should not necessarily be correlated with affluence (The Scottish Government 2012).

Chapter 1

Welcome to R

1.1 Coding to Spot and Prevent Errors

When writing code, the computer will only give you an answer if the input is correctly defined, and will only give the right answer if the input is correct. For example, asking a computer to do the sum `2 + elephant` would be non-sensical and would result in an error. This is an example of the input not being correctly defined. If you wanted to find the sum of 2 and 3, but had a typo and asked for `2 + 4`, you would not get an error, as the inputs are correctly defined, but you would not get the correct answer, as the inputs are not correct. Ensuring that the inputs are correct can sometimes be the harder error to spot, as they do not result in an error (at least at the point of the error). However, errors in the definition of inputs can be just as challenging to solve after the initial error message. As you work through this course, you'll learn about R and ways to write code, including writing scripts, loops (coming in Sections 3.7.5.1 and 3.7.5.2), functions, and more. Here, we will outline a few tactics to ensure the code you write, whether a single line or a multi-document script, remains error-free and produces the expected outcomes. We will also discuss strategies to identify where errors occur in larger scripts.

1.1.1 Unit Testing

The first rule of programming is that it should be built up in small steps, and each step should be tested before the next is built. Ideally, the larger code is composed of smaller functions that can be individually tested, known as unit testing. Unit testing checks that your code takes the correct inputs and returns the correct outputs, and, where necessary, tests "edge cases". If we take the example from before of a function that adds exactly two numbers, we would write some unit tests. These could include:

- Is the output for `3` and `4 = 7`? (testing baseline functionality)
- If I give the function three numbers, does it return either an error or a warning that the third number is being ignored? (checking that no more than two inputs are being added)
- If I give the function one number, does it return an error? (checking that it doesn't try to add fewer than two numbers)
- Is the output for `2.5` and `3.2 = 5.7`? (checking that it can handle more than integers)
- Is the output for `2` and `elephant` an error? (checking that it cannot accept character strings and errors appropriately)
- Is the output for `-1` and `-4 = -5`? (checking that it can correctly handle negative numbers)
- Is the output for `-1` and `3 = 2`? (checking that it can handle negative and positive numbers)
- Is the output for `0` and `0 = 0`? (checking that it can handle zero and identical inputs)

Unit tests are specific to each function and must be written to identify potential problems. This is why writing small functions are preferable to large functions, as the testing can be more specific. Tests should have a known output to ensure that the functions are

returning the expected output, as there is no point in testing something where you don't know what the output is supposed to be. If the unit test fails, then you need to work out why and fix it before moving on. The skills to work out why are also relevant to picking up errors that come from scripts and functions, regardless of whether you are unit testing.

1.1.2 Understanding Error Messages

Error messages should be written in a way that points to where and why the error occurred. In theory, this should make the error really easy to spot, but in practice, it does not always work that way. Some typical error messages might include the following.

- `object 'X' not found` - this means that you haven't previously defined `X` and if you look in the Environment (this will be further expanded on Chapter 2) you will not find anything under `X`. This could be because you previously defined it and then cleared it, or you are looking for a variable in a data.frame without telling R where the variable is located in, or that you defined it in a loop/function that you then exited or that it hasn't been introduced to the function
- `unexpected '=' in ...` where any other symbol could also be included (e.g. `(`, `}`, `*`, `$`, and many others) and `...` is the line of code where the unexpected symbol is. This normally means you have either left in a symbol where you should have deleted it (like an extra closing bracket), or you've not properly defined the inputs for the function you are using
- `could not find function 'pivot_longer()'` probably means you have not loaded the package with the function you are trying to use, or that you have mistyped the function name

1.1.3 Looking Inside Functions

When writing your own code and functions, it is sometimes easiest to evaluate any issues in the middle of execution. The simplest approach is to add print statements. These are useful to check whether a loop is iterating correctly or if an if / otherwise statement is working as expected. You can print the counter (`print(i)`) or some of the data (`print(data)`) or even just a message (`print("success")`). For more complex error identification, browser or break points within the file (click to the left of the line number in RStudio) pause the evaluation of a function when it is run, allowing you to see the state of the function and work through step by step while the function is running. It is also possible to set a general option that whenever an error occurs, rather than returning an error message, you enter browser mode as if you had used the browser function, by using `**options(error = browser)**`.

1.2 Help I'm Stuck

This course and guide are intended to provide an introduction to R, which means that you can start to use R and RStudio more widely. Inevitably, as you use R more, you will come across questions on how to accomplish things, whether that is trying to determine whether you are using a function correctly, whether the method you are using is the most appropriate, or whether someone has already done what you are trying to do and can save you time by sharing a potential solution. This guide can be a good starting point to try to answer these questions, but it will not provide all the answers. Therefore, we have compiled a few other places to look for solutions.

1.2.1 Cheat Sheets

Within RStudio, there are some cheat sheets that are easily accessible. For the purposes of this course, the following are the most useful:

- RStudio **integrated development environment (IDE)** cheat sheet
- Data transformation with dplyr (see Chapter 3 for more information on dplyr)
- Data Visualization in ggplot2 (see Chapter 4 for more information on ggplot2)

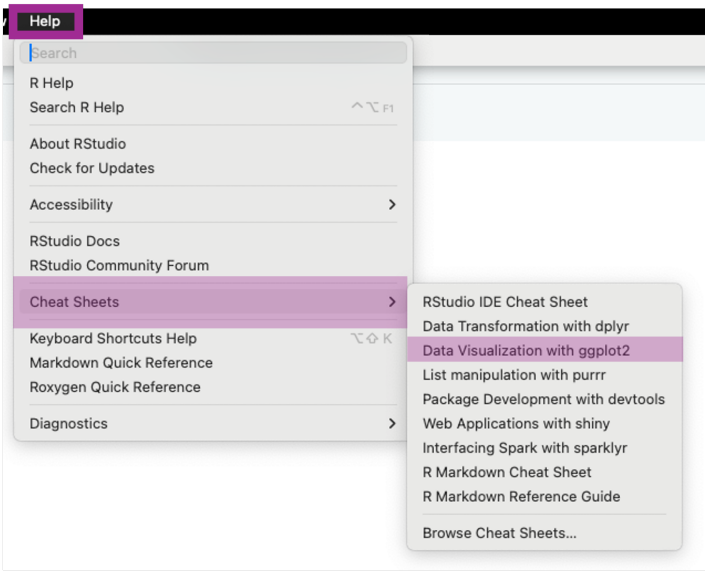


Figure 1.1: How to access data visualisation cheat sheets in R

Using the ggplot2 cheat sheet as an example, they can be found under Help -> Cheat Sheets -> Data visualisation in ggplot2.

As you can see, there are more cheat sheets available within that menu, and even more cheat sheets are available under "Browse Cheat Sheets..."

1.2.2 `help` Function in R

To understand how a specific function works, there are two functions you can use: `help` or `?`. This will open some documentation under the **Help** tab within **RStudio**. This documentation provides a brief description of the function. It also includes an example of its use that covers various input arguments, default values for optional arguments, written descriptions of both required and optional arguments, and other relevant information, such as references, similar functions, and worked examples. Using an example for the function `sum`, from the base package, which is a function used to add a list of numbers together, the help documentation looks like Figure 1.2. To access the help documentation, we have to type `help(sum)` or `?sum` into the console and then press enter.

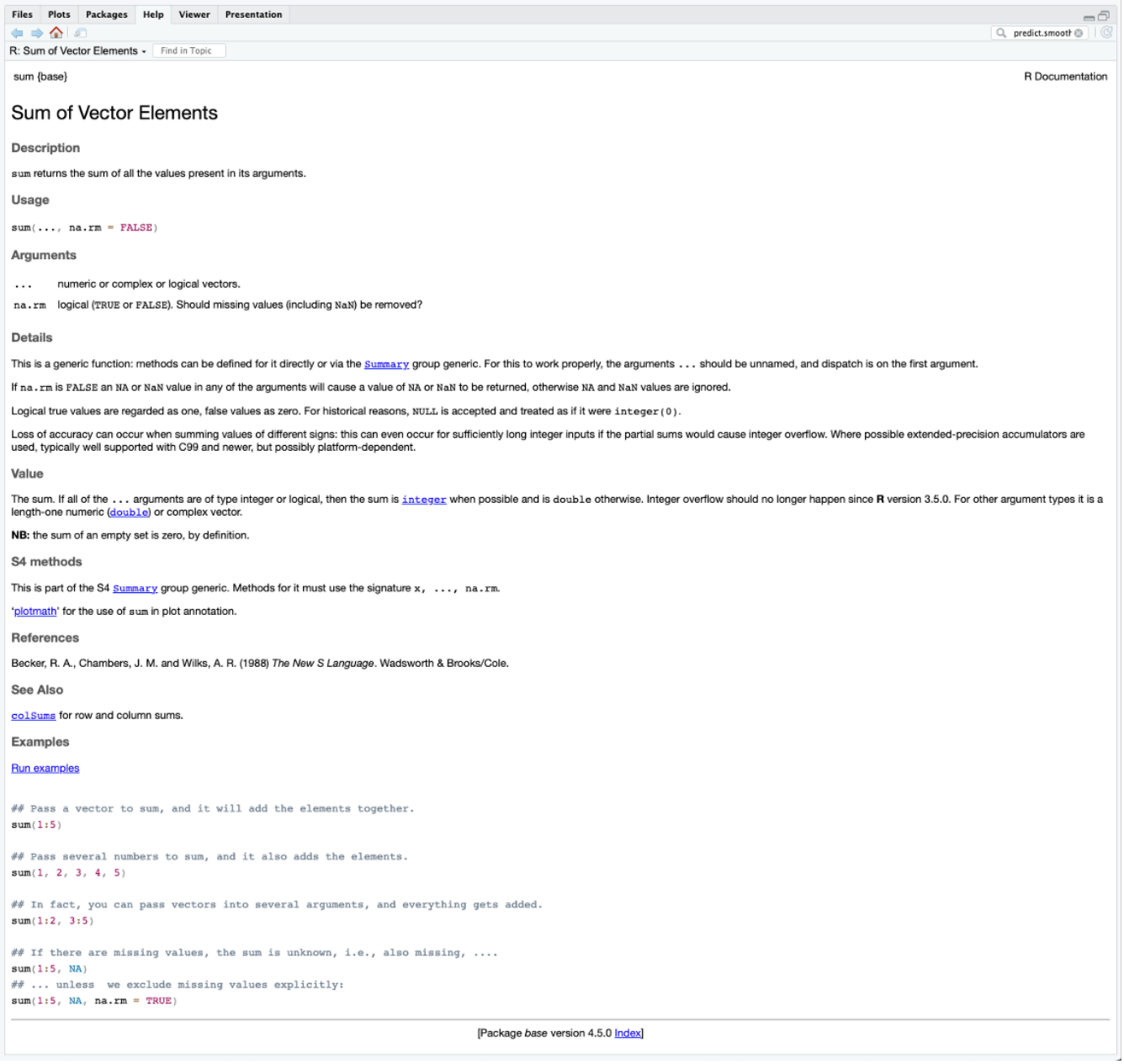
1.2.3 Reference Manuals and Vignettes

Many packages have reference manuals which describe each function within the package. These can be very technical and inaccessible for someone starting out, or trying to understand how to use the functions. These reference manuals are a compilation of the outputs for `help` for every function within a package.

Most packages also come with user guides known as vignettes. Within RStudio, these can be found by typing `**browseVignettes(package = "dplyr")**` (using dplyr here as an example package) and then pressing Enter. This will open a new page on your web browser with links to the vignettes available for the package as seen in Figure 1.3. Vignettes can also be found on the CRAN page for the package, which is also where the full reference manuals can be found, as seen in Figure 1.4.

1.2.4 Stack Overflow

You are very unlikely to come across a problem that is not at least similar to something someone else has encountered, and with the power of the internet, we can be connected to all the people who have tried before us. Stack Overflow is a website where people go to ask for help with programming questions. Often, if you Google a question, a Stack Overflow page will come up as a suggestion, and more often than not, there will be a solution in the answers. Answers are provided by other users, and up- and down-voted according



Files Plots Packages Help Viewer Presentation

predict.smooth

R: Sum of Vector Elements — Find in Topic

sum {base} R Documentation

Sum of Vector Elements

Description

`sum` returns the sum of all the values present in its arguments.

Usage

```
sum(..., na.rm = FALSE)
```

Arguments

`...`	numeric or complex or logical vectors.
`na.rm`	logical (`TRUE` or `FALSE`). Should missing values (including `NaN`) be removed?

Details

This is a generic function: methods can be defined for it directly or via the `Summary` group generic. For this to work properly, the arguments `...` should be unnamed, and dispatch is on the first argument.

If `na.rm` is `FALSE` an `NA` or `NaN` value in any of the arguments will cause a value of `NA` or `NaN` to be returned, otherwise `NA` and `NaN` values are ignored.

Logical true values are regarded as one, false values as zero. For historical reasons, `NULL` is accepted and treated as if it were `integer(0)`.

Loss of accuracy can occur when summing values of different signs: this can even occur for sufficiently long integer inputs if the partial sums would cause integer overflow. Where possible extended-precision accumulators are used, typically well supported with C99 and newer, but possibly platform-dependent.

Value

The sum. If all of the `...` arguments are of type integer or logical, then the sum is `integer` when possible and is `double` otherwise. Integer overflow should no longer happen since **R** version 3.5.0. For other argument types it is a length-one numeric (`double`) or complex vector.

NB: the sum of an empty set is zero, by definition.

S4 methods

This is part of the S4 `Summary` group generic. Methods for it must use the signature `x, ..., na.rm`.

'plotmath' for the use of `sum` in plot annotation.

References

Becker, R. A., Chambers, J. M. and Wilks, A. R. (1988) *The New S Language*. Wadsworth & Brooks/Cole.

See Also

`colSums` for row and column sums.

Examples

Run examples

```
## Pass a vector to sum, and it will add the elements together.
sum(1:5)

## Pass several numbers to sum, and it also adds the elements.
sum(1, 2, 3, 4, 5)

## In fact, you can pass vectors into several arguments, and everything gets added.
sum(1:2, 3:5)

## If there are missing values, the sum is unknown, i.e., also missing, ....
sum(1:5, NA)
## ... unless  we exclude missing values explicitly:
sum(1:5, NA, na.rm = TRUE)
```

[Package *base* version 4.5.0 Index]

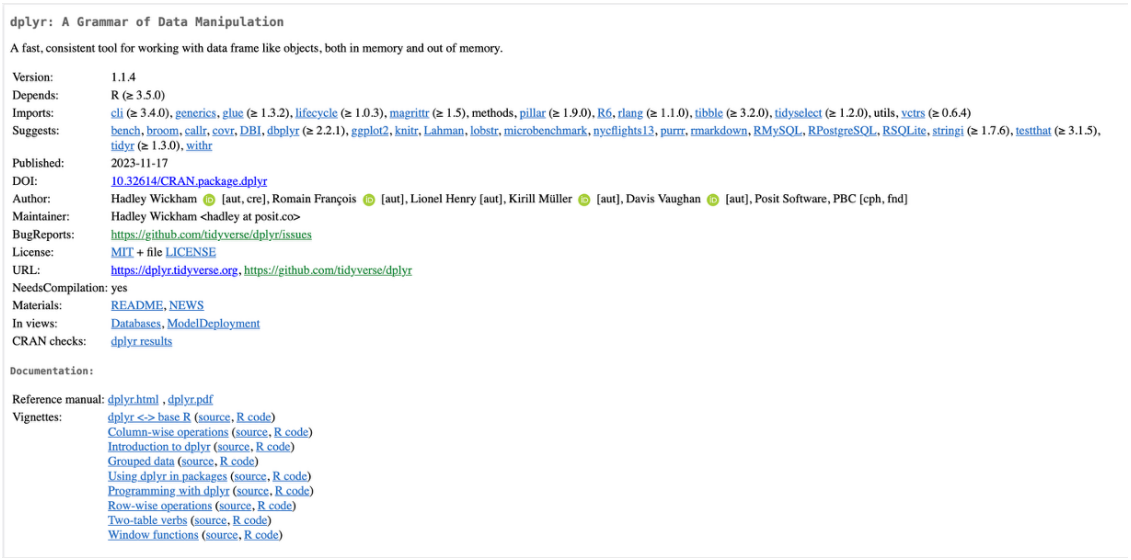
Figure 1.2: The documentation for 'help(sum)' or '?sum'

Vignettes found by "browseVignettes(package = "dplyr")"

Vignettes in package dplyr

- Column-wise operations - HTML source R code
- dplyr <-> base R - HTML source R code
- Grouped data - HTML source R code
- Introduction to dplyr - HTML source R code
- Programming with dplyr - HTML source R code
- Row-wise operations - HTML source R code
- Two-table verbs - HTML source R code
- Using dplyr in packages - HTML source R code
- Window functions - HTML source R code

Figure 1.3: The vignettes available for dplyr as shown when using the browseVignettes function



dplyr: A Grammar of Data Manipulation

A fast, consistent tool for working with data frame like objects, both in memory and out of memory.

Version:	1.1.4
Depends:	R (≥ 3.5.0)
Imports:	cli (≥ 3.4.0), generics, glue (≥ 1.3.2), lifecycle (≥ 1.0.3), magrittr (≥ 1.5), methods, pillar (≥ 1.9.0), R6, rlang (≥ 1.1.0), tibble (≥ 3.2.0), tidyselect (≥ 1.2.0), utils, vctrs (≥ 0.6.4)
Suggests:	bench, broom, callr, covr, DBI, dbplyr (≥ 2.2.1), ggplot2, knitr, Lahman, lobstr, microbenchmark, nycflights13, purrr, rmarkdown, RMySQL, RPostgreSQL, RSQLite, stringi (≥ 1.7.6), testthat (≥ 3.1.5), tidyr (≥ 1.3.0), withr
Published:	2023-11-17
DOI:	10.32614/CRAN.package.dplyr
Author:	Hadley Wickham [aut, cre], Romain François [aut], Lionel Henry [aut], Kirill Müller [aut], Davis Vaughan [aut], Posit Software, PBC [cph, fnd]
Maintainer:	Hadley Wickham <hadley at posit.co>
BugReports:	https://github.com/tidyverse/dplyr/issues
License:	MIT + file LICENSE
URL:	https://dplyr.tidyverse.org, https://github.com/tidyverse/dplyr
NeedsCompilation:	yes
Materials:	README, NEWS
In views:	Databases, ModelDeployment
CRAN checks:	dplyr results

Documentation:

Reference manual:	dplyr.html , dplyr.pdf
Vignettes:	dplyr <-> base R (source, R code) Column-wise operations (source, R code) Introduction to dplyr (source, R code) Grouped data (source, R code) Using dplyr in packages (source, R code) Programming with dplyr (source, R code) Row-wise operations (source, R code) Two-table verbs (source, R code) Window functions (source, R code)

Figure 1.4: The CRAN page for the dplyr package, including urls to the reference manuals and the vignettes

to how helpful people find them. An example of a resolved question in R (that you should be able to answer yourself by the end of this course) is one about removing axis labels from a plot in ggplot2. The person asking the question provided some simple code that showed the issue without any irrelevant code (called a minimal reproducible example [**minimal reproducible example (MRE)**] or minimal working example [**minimal working example (MWE)**]). Then the person answering gave a very short response, again with a code example, that provides the solution. Stack Overflow also tends to be very useful when you come across an error you've not encountered before. For example, this person found an error when trying to sort some data, and someone has provided both a solution and an explanation for the error. Stack Overflow is part of a larger group of websites called the Stack Exchange, which all function in the same way. Often stats questions can be answered from the stats version of Stack Overflow.

1.2.5 Other

There are other sources of information when struggling with a problem in R

- R-bloggers has blogs dedicated to different topics
- A selection of useful data science resources compiled by Jocelyn Friday
- Tutorials for learning R

Chapter 2

Introduction and basic usage

2.1 What are R and RStudio?

R is a programming language. Like any language, it gives you the ability to communicate, in this case with your computer. By writing R code, you can give your computer instructions to carry out.

RStudio is an **IDE**. You can use R without RStudio, but RStudio provides a user-friendly method of using R to communicate with your computer. If you want to use R and RStudio on your computer or laptop, you first need to download and install them. You can download R from here: https://cloud.r-project.org/, and RStudio from here: https://posit.co/download/rstudio-desktop/. Make sure to select the version that matches your operating system (e.g., Windows, macOS, or Linux) when downloading.

2.2 Why use R?

Many programmes and languages can be used for statistical analysis and data management, but there are several advantages to using and learning R.

While R is not as feature-rich as Python and SAS, it is an all-rounder that can accomplish a wide variety of tasks, e.g., from managing data easily with dplyr and making beautiful graphs with ggplot2, to fitting *P*-splines in generalised additive models using mgcv and causal mediation analysis using CMAverse. If you are analysing big data with a lot of repetition, you can also speed things up with the (relatively) efficient parallel computing with furrr and doParallel (outside the scope of this course). R also has some capabilities as a general programming language, such as managing file systems (e.g., creating, moving, and deleting files). You can also use R to write replicable reports using **R Markdown** or **Quarto**, and interactive websites and dashboards using `shiny`. Lastly, R is free and open source, and it is one of the most widely used languages for statistics and data science. This makes it a valuable skill for careers such as data analyst or data scientist across many fields.

2.3 Using R and RStudio

The first time you open RStudio after installing it, it will look like this:

The large window on the left is called the **console** pane. In the console, you can type a command, which will appear next to the blue arrow (>). If you press Enter, R will execute your command and display the result. Try running the following code and see what happens:

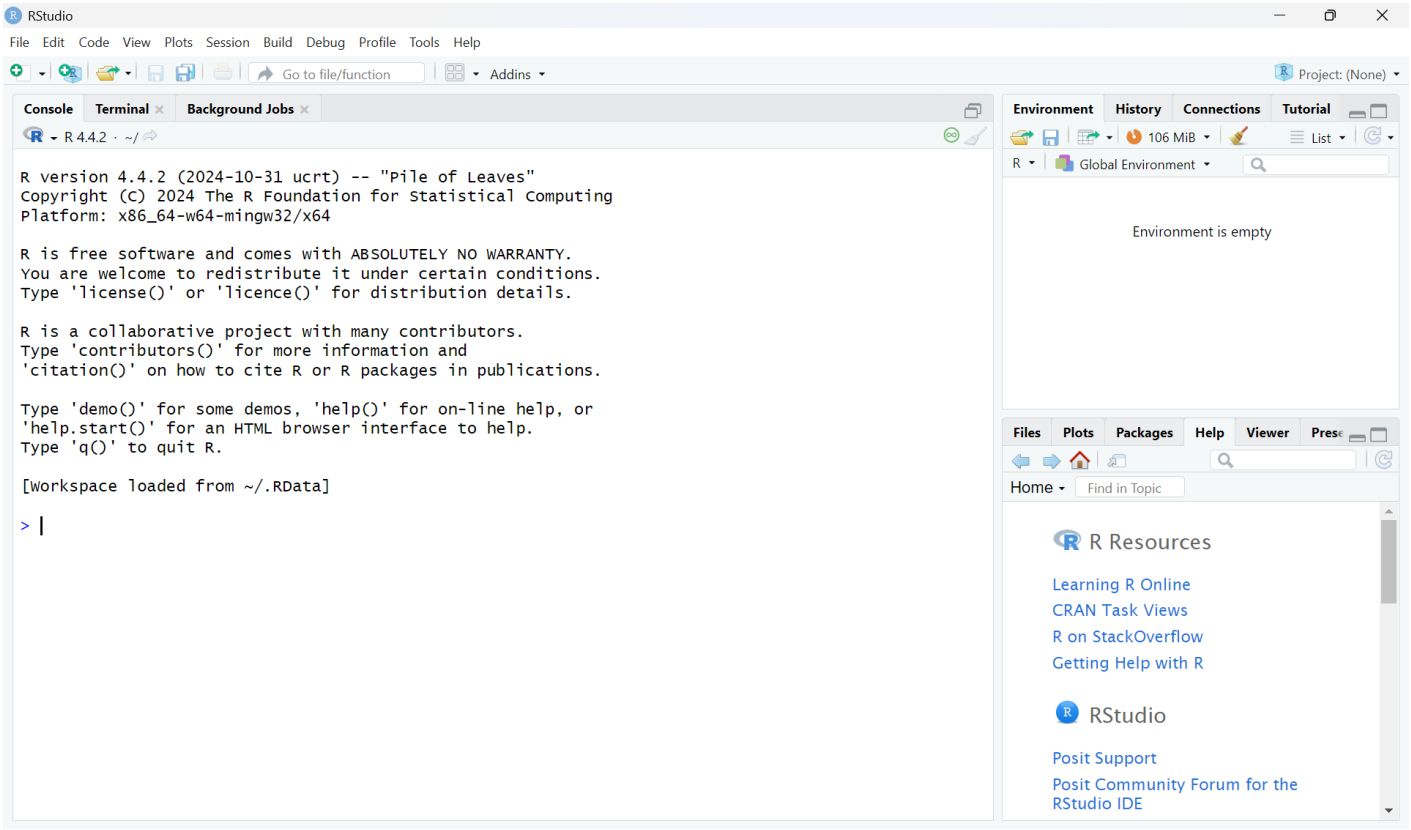


Figure 2.1: Home RStudio Screen highlighting the console

```
2+2
```

```
## [1] 4
```

You can also assign a value to a variable, which allows you to reuse the value that is now stored in a. Try typing in your console:

```
a <- 3
```

Next, look at the Environment pane in the top right. This pane shows you the variables you have created. You should be able to see that the value "3" is now associated with your variable "a". Try typing in your console:

```
a
```

```
## [1] 3
```

The console should return "3" to you. You could also create a vector of values, and assign this to a variable:

```
v <- c(1, 5, 8)
v
```

```
## [1] 1 5 8
```

2.4 R scripts

For simple commands, you can use the console as shown above. However, for more complicated tasks, it will be easier to create an R script. You can create an R script by clicking on File > New File > R Script. This will open an R file in which you can write your code. RStudio should then look like this:

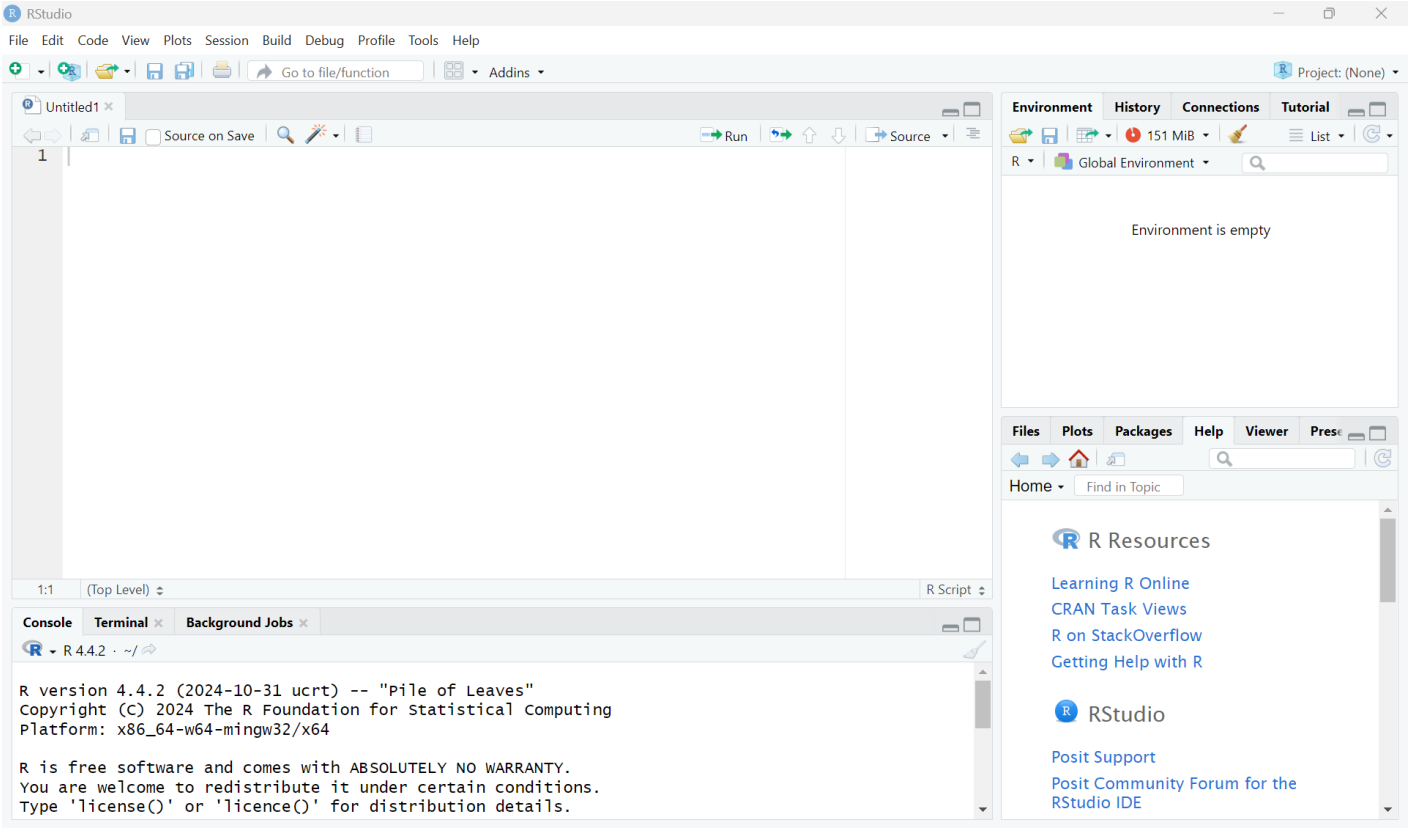


Figure 2.2: Home RStudio Screen highlighting a blank R script

The top left window is the empty script. Add the following code to your R script:

```
a <- 3
b <- 10
c <- 5
(a+b)*c
```

```
## [1] 65
```

The calculation adds two numbers and then multiplies this sum by a third number. You can run the script line by line by pressing the Run button repeatedly. You can also run the entire script by pressing the Source button. You can use the drop-down menu next to the Source button and click on "Source with Echo" to ensure your code and its output are printed to the console. Once you have run the script, the Environment pane displays all the variables you created (a, b, c) along with their values.

Once you have created a script, you can save it, so that you can reuse it again later. You can do this by clicking File > Save or File > Save As... and specifying a name for your script.

2.5 R functions

If you wanted to do the same calculation multiple times, but for different numbers, you could create something called a function. A function takes an input and uses it to produce an output by using the code defined within the function. Functions can be called multiple times, helping to avoid unnecessary code duplication.

The following code defines the function `sum_and_multiply()`, and calls it with multiple different number combinations as inputs. **Try it out!**

```
sum_and_multiply <- function(a, b, c) {
  (a+b)*c
}

sum_and_multiply(2, 4, 10)
```

```
## [1] 60
```

```
sum_and_multiply(6, 8, 2)
```

```
## [1] 28
```

```
sum_and_multiply(1, 5, 2)
```

```
## [1] 12
```

Now try writing your own function!

What if you now wanted to calculate the mean and the **standard deviation (SD)** of a series of numbers? You could again write your own function. Alternatively, you can use existing functions. R has a large number of built-in functions that are ready to use. The functions `mean()` and `sd()` can be used to calculate the mean and standard deviation, respectively. You can use the `help()` function or place a "?" in front of your function. This will provide access to the documentation pages of that function and provide guidance on how to use it.

Try this:

```
utils::help(mean)
```

and this:

```
?sd
```

The documentation should have appeared in the bottom-right corner of RStudio. Did the documentation give enough information to use these two functions?

Here is one way in which you can calculate the mean and standard deviation of a series of numbers:

```
x <- c(1, 4, 7, 3)
base::mean(x)
```

```
## [1] 3.75
```

```
stats::sd(x)
```

```
## [1] 2.5
```

2.6 Environment and scope

The above examples have shown how to create and save data in R. By default, these data are stored in the **global environment**, which is accessible to all functions in R. However, variables created inside a function are generally only available within that function's scope and are not accessible to other functions. Below is an example of calculating the area of a right-angled triangle:

```
area <- function(base, hypotenuse){
  x <- base * hypotenuse
  return(x / 2)
}

area(3, 2)
```

```
## [1] 3
```

```
x
```

```
## [1] 1 4 7 3
```

Note: even though we have run the `area()` function, we can't recall the value of x (which is defined and calculated inside that `area()` function. This is because the value of x was not saved in the global environment and is gone as long as the function call is finished. This concept is not necessary to understand to use R, however it is useful for anyone who may use functions in the future.

2.7 Packages

Base R encompasses all the functions and features included with R upon installation. The double-colon operator '::' tells R to select definitions from the named package on the right. For example, mean and sd are examples of base R functions. Other R users have created their own functions and shared them online as installable packages, enabling others to use them for free. These are not part of base R and, therefore, need to be installed separately. Most R packages can be found on CRAN (https://cran.r-project.org/). To install a package from CRAN, you can use the install.packages function. Let's say you wish to now calculate the **standard error (SE)** of a series of numbers. You can calculate this by using a formula, but you can also use an already existing function. The package plotrix contains the function std.error, which can calculate this. plotrix is not part of base R, so needs to be installed.

Try:

```
utils::install.packages("plotrix")
```

To use a package you installed, you first need to load it with the `library()` function. You should then be able to use the std.error function:

```
base::library(plotrix)

plotrix::std.error(x)
```

```
## [1] 1.25
```

Can you find a different function that is not part of base R, install the relevant package, and use it?

2.8 Reserved words

R has reserved words that have a special meaning within the language. You cannot use them as a function or a variable name. To find out what the reserved words are, you can type:

```
?reserved
```

From the list, you can see that "if" and "else" are reserved words. Try writing:

```
if <- 3
```

```
## Error in parse(text = input): <text>:1:4: unexpected assignment
## 1: if <-
##        ^
```

As expected, the R console will print an error, and will not have assigned the value "3" to "if". R is case sensitive. This means that it distinguishes between if, If, iF, and IF. This means that you would be able to assign a value to If, iF, or IF. Try it:

```
If <- 3
```

Now, the value of "3" should be assigned to If.

2.9 Error vs warning messages

In the previous section, the R console printed an error message and did not run the code. Running R code can also generate warning messages. The code will still run, but may not behave as expected. Here is an example of that code that runs, but will generate a warning message:

```
v <- c("1", "2", "3")
base::mean(v)
```

```
## Warning in mean.default(v): argument is not numeric or logical: returning NA
```

```
## [1] NA
```

You may expect the output of the above code to be 2. However, the function mean expects its input to be a vector of numbers, not strings. Therefore, the code produces a warning message, and **not available (NA)** as the output.

2.10 Working directory

R allows you to read files from your computer and save files to it. How does R know where to read from or save to? You can provide a full file path. If no full path is provided, R assumes the file is located in the working directory. The working directory is the default location on your computer where R reads and saves files. To find out what your working directory is, type `getwd()`:

```
getwd()
```

```
## [1] "C:/Users/cresc/OneDrive - University of Glasgow/Teaching/MED5686 - R for Health Data Science/R4HDS_bo
```

Are you happy with that working directory? If not, you can change it using `setwd()`:

```
setwd('path')
```

Table 2.1: Functions to import data into R

File format (file name extension)	base R	Using additional packages
CSV (.csv)	read.csv	read_csv
TSV (.tsv)	read.table	read_tsv
TXT (.txt)	read.delim	read_delim
Excel data file (.xlsx / .xls)		read_excel
STATA data file (.dta)		read_dta
SPSS data file (.sav)		read_sav
SAS data file (.sas7bdat)		read_sas

To demonstrate the working directory, you can create a simple text file called "R output.txt" by using the command `writeLines()`. Please note that the command will overwrite any existing text file with the same name.

```
base::writeLines("Hello World", "R output.txt")
```

Open your file explorer and navigate to your working directory. You should be able to find the file `R output.txt`, with the line `Hello World` within it.

Please note: you can also set your working directory by clicking Session > Set Working Directory > Choose Directory (or source file location).

2.11 Import data

The examples above used data that were input directly into R's console. However, real-world data analysis typically utilises data stored in files on either local or network drives. In those scenarios, you will need to import the data from the files into R. The exact functions used to import data depend on the data format. One of the most common formats of data is a **comma separated value (CSV)** file, and we will use that as an example. Please download **02-hrqol_data.csv** from Moodle, and place it in a convenient folder. For example, if you have put the data file in `C:/Users/ABC/R_course/Data`, then you can import the data using `read.csv()`:

```
dat <- utils::read.csv("./data/02-hrqol_data.csv")
```

Note that the code not only imports the data, but also stores the imported data in a data frame object called `dat` (see Section 3.3.3.1). It is common practice to store the imported data in an object, because without doing so, you cannot interact with the data after loading it.

Below is a table showing common data formats and some functions to import them:

If you haven't installed those additional packages (readr, readxl, haven), the code on the right column won't work. You must install the package as described in the above section before running the code. Also note that the readr in the read_csv command means that the read_csv function is in the readr package specifically.

2.12 Export data

If you have made changes to the data in R and want to export a copy to a local or network drive. You can export data in a similar way with `write.csv()`:

```
utils::write.csv(dat * 2, 'C:/Users/ABC/R_course/Data/test_data_new.csv')
```

Table 2.2: Functions to export data from R

File format (file name extension)	Base R	Using additional packages
CSV (.csv)	write.csv	write_csv
TSV (.tsv)	write.table	write_tsv
TXT (.txt)	writeLines	write_delim
Excel data file (.xlsx)		write_xlsx
STATA data file (.dta)		write_dta
SPSS data file (.sav)		write_sav
SAS data file (.sas7bdat)		write_sas

As with data import, R can export data in various formats using different packages.The corresponding functions are listed in the table below:

Be aware that some data formats may not correspond to the standard extension. For example, a data formatted as **tab separated value (TSV)** could have an extension of '.txt'. If the imported data doesn't make sense (e.g., only one column), please review the imported data and determine if a different function should be used instead. Also note that the Excel data requires a different package for export than it did for import (writexl).

2.13 Project

The **Project** functionality in R is a good way to manage different projects. For example, you can use one project for all scripts for this course and another project for your **Master's of Public Health (MPH)** project (if you decide to use R for it).

The main advantage of using the **project** functionality is that the **working directory** is set automatically to your project folder, and it could be easier to import data or scripts into R without using absolute paths. For example, if you have a project folder `"C:/Users/ABC/R_course/"` for this course, and the data '02-hrqol_data.csv' located in the `"C:/Users/ABC/R_course/Data/02-hrqol_data.csv"`, you can either read in the data using an absolute path:

```
dat <- read.csv("C:/Users/ABC/R_course/data/02-hrqol_data.csv")
```

Or using a relative path, if your working directory is set correctly (or if you use the Project function):

```
dat <- read.csv("data/02-hrqol_data.csv")
```

Similarly, you can use a relative path when you export files.

To create an R Project, you can click on the project button in the top-right corner of your RStudio.

You then can create a project by clicking the **New Project...** button, which then gives you an option to create a new project by creating a new folder (which would be empty) or to create a new project based on an existing folder (where there may already be data and other relevant files in it). In Figure 2.3, you can also see that all recent projects that you have used are listed there, which could be handy to switch between ongoing projects.

2.14 R Markdown

R Markdown enables you to combine code, code output, images, and plain text into one document (Word, **portable document format (PDF)** or **hypertext markup language (HTML)**). R Markdown lets you directly produce a nicely formatted report, without needing to copy and paste the results of your analysis into a separate document.

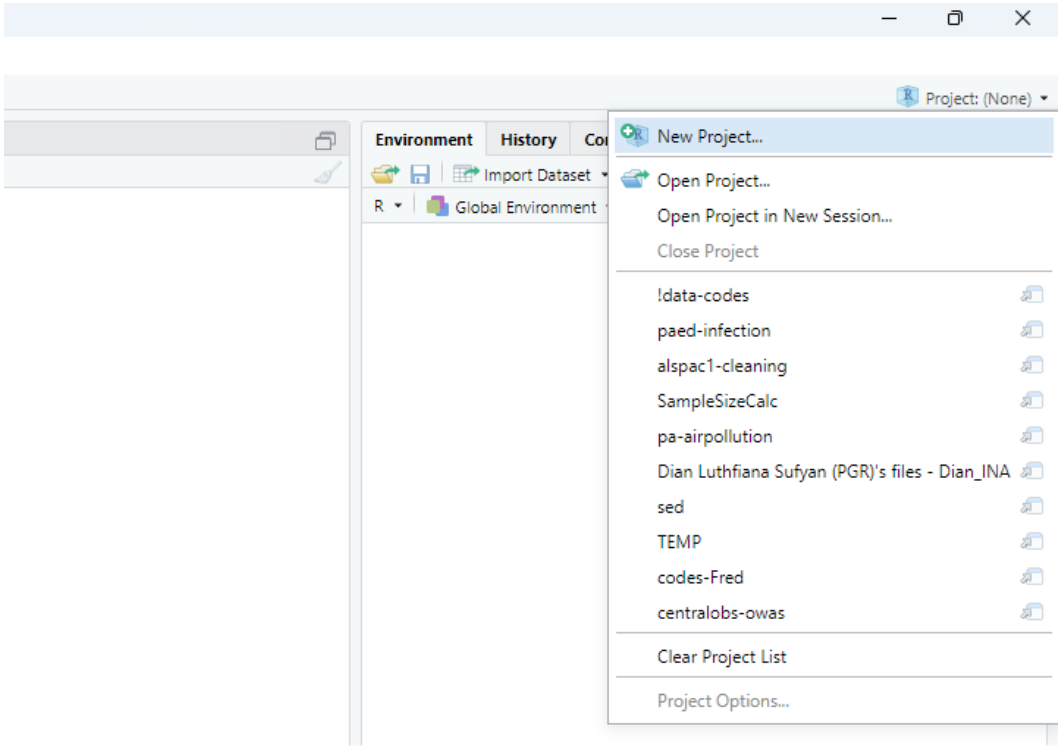


Figure 2.3: Project Menu in RStudio

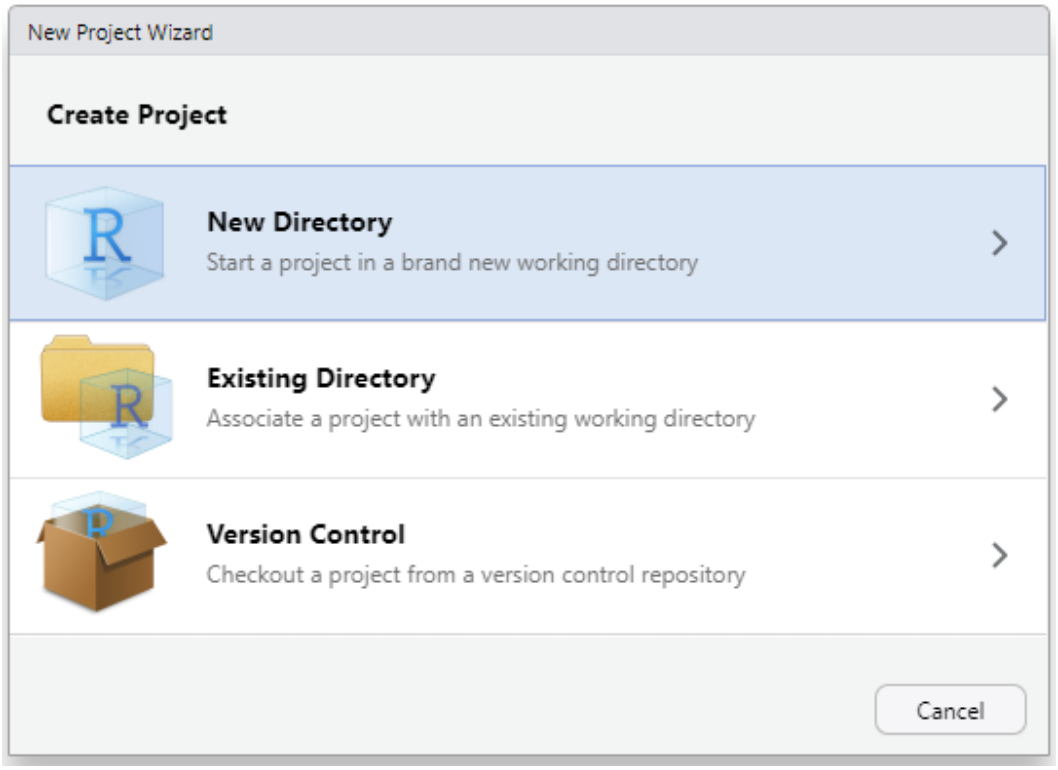


Figure 2.4: Options to create a new project

Using R Markdown can be more efficient, transparent, and supports the reproducibility of your work. To use R Markdown you first need to install the rmarkdown package.

```
install.packages('rmarkdown')
```

If you would like to create a **PDF** or Word document using R Markdown, you will also need to have a version of LaTeX installed on your computer. If you do not yet have a version of LaTeX installed on your computer, tinytex (https://yihui.org/tinytex/) is a good choice. You can install rmarkdown and tinytex with the following commands:

```
utils::install.packages('tinytex')
tinytex::install_tinytex()
```

You can create an R Markdown document by clicking New File > R Markdown... Give the document a title and add your name in the 'Author box', and click on 'OK'. This will create an R Markdown file. Save this file in your working directory. You can now knit it by clicking on the 'Knit' button. Wait for it to finish and have a look at the output.

You can also run the code sections ("chunks") without 'knitting' the document by pressing the green arrow in each chunk. Have a look at the Run drop-down menu for other Run options.

Try adding some code and text of your own to your R Markdown file, run your code chunks, and knit the file to get a feel for how it works.

2.15 R coding tips

The above examples are relatively simple. However, if you start writing more complicated and larger pieces of code, consistently formatting your code will help its readability, maintainability, and help you spot errors. Below are some tips for writing your R code.

2.15.1 Indentation

Compare these two code snippets:

```
x <- 3


if(x %% 2 == 0) {
base::print("Even number")
} else if (x %% 2 == 1) {
base::print("Odd number")
} else
base::print("Number is neither even nor odd")
```

```
## [1] "Odd number"
```

```
if(x %% 2 == 0) {
  base::print("Even number")
} else if (x %% 2 == 1) {
  base::print("Odd number")
} else
  base::print("Number is neither even nor odd")
```

```
## [1] "Odd number"
```

The first code snippet does not use indentation (spaces at the beginning of a line of code), while the second does. Consistent use of indentation can aid readability.

2.15.2 Commenting

For the code above, you may have inferred that it tells you whether a number is even or odd (or neither). However, you may not understand how it does that. Adding comments to your code can help explain to others how it works. It will also help you to understand your own code if you need to use or edit it in the future. You can add comments by using the "#" symbol. See below how the added comments help to clarify what the code does and how it accomplishes this.

```
# the following code determines whether a number is even, odd, or neither
# it does this by using the modulo (%%) operator
# the modulo operator returns the remainder of a division
# the remainder of an even number divided by 2 is 0
# the remainder of an odd number divided by 2 is 1
# the remainder of a decimal number divided by 2 is always another decimal number
# therefore, for a decimal number, the else condition would hold

if(x %% 2 == 0) {
  base::print("Even number")
} else if (x %% 2 == 1) {
  base::print("Odd number")
} else
  base::print("Number is neither even nor odd")
```

```
## [1] "Odd number"
```

Try adding relevant comments to the function you wrote previously.

2.15.3 Naming conventions

Being consistent with how you name your variables and functions can further improve your code's readability. Try to choose names that are short and meaningful.

```
# short, but not informative
x <- "December"

# informative, but long
twelfth_month_of_the_year <- "December"

# both short and informative
month_12 <- "December"
```

Did the function you wrote previously have a short and meaningful name? If not, change it.

It is also best not to reuse existing function names. Reusing a function name will overwrite the original function. Here is an example:

```
c <- function(a, b, c) {
  (a+b)*c
}
```

The above code will overwrite the base R function c, which combines values into a vector (this will be covered further in Section 3.3.1). If you were to use c again, it would refer to the function in the above yellow box.

2.15.4 Style guides

The above sections provide a few tips on how to improve the readability of your code. For further ideas/guidance on how to write and format your code, you can refer to a style guide. Here is an example of a style guide: Tidyverse Style Guide. Have a look!

2.16 References/further reading

1. https://rstudio-education.github.io/hopr/
2. https://intro2r.com/
3. https://docs.posit.co/ide/user/

Chapter 3

Data Types & Manipulation

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.6
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.2     v tibble    3.3.1
## v lubridate 1.9.4     v tidyr     1.3.2
## v purrr     1.2.1
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
##
## Attaching package: 'hms'
##
##
## The following object is masked from 'package:lubridate':
##
##     hms
```

```
##            used  (Mb) gc trigger  (Mb) max used  (Mb)
## Ncells 2279404 121.8    4157530 222.1  4157530 222.1
## Vcells 4002148  30.6    8388608  64.0  6671981  51.0
```

At the heart of almost all of R, nearly everything in R is an **object** of some form or fashion. These can be almost anything, from a single number or word to more complex things such as functions (as seen in Section 2.5), data sets, plots (coming in Chapter 4), and analysis results (coming in Chapter 5).

Figure 3.1 provides a rough outline of the chapter to come, starting with the basic and complex data types in Section 3.1, followed by built-in constants in Section 3.2, then data structures in Section 3.3, and concluding with data manipulation in Sections 3.5-3.7 and merging data sets in Section 3.8.

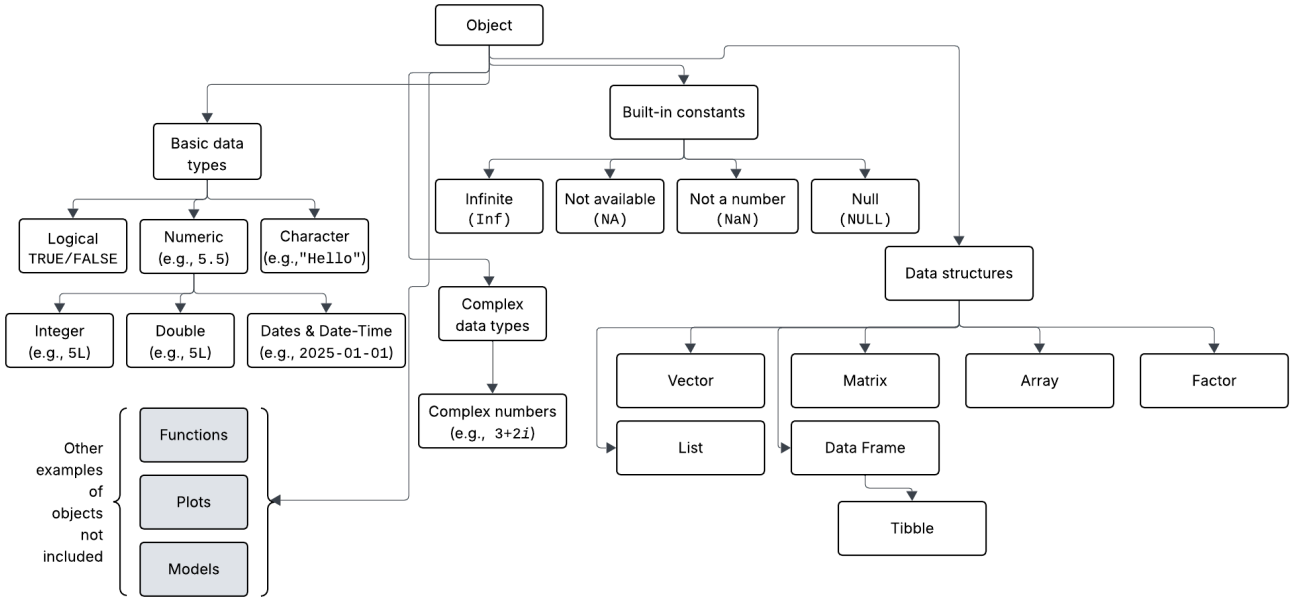


Figure 3.1: A rough diagram of the object hierarchy of what will be covered in this chapter. Most things in R descend and inherit functionality from the main Object class, and as such are related. Examples of each object type have been presented in parentheses where possible. The information in the grey boxes either has been or will be covered in other chapters, and this does not include topics such as user-defined classes and objects.

3.1 Data Types

R has object-oriented programming capabilities, which means that almost everything in R is an object which has a **class**. The class defines the object's type (e.g., what information it can hold) and how R behaves when functions (see Section 2.5) are applied to it. This means that regardless of object type, the `class` function can be used to check the data type.

Define a function and check its class:

```
# Define the function
a <- function(){
    base::print("hello world")
}

# Check the class
base::class(a)
```

```
## [1] "function"
```

These objects can be used to store different types of data, and each data type has inherently distinct properties, allowing R to use them for various tasks. In R, variables do not need to be declared as a specific type of data, as shown in Chapter 2; an object can change type after having been set. This is distinctly different from languages such as Java and C++.

3.1.1 Logical (Booleans)

One of the most basic and simple data types in R are **logical** or **boolean** values, which can either be `TRUE` (`T` also works) or `FALSE` (or `F`).

Table 3.1: A table of the most common operators used to compare two values.

Operator	Description	Example
>	Greater than	'2 > 1' returns 'TRUE'
<	Less than	'2 < 1' returns 'FALSE'
==	Equals to	'2 == 2' returns 'TRUE'
!=	Not equal to	'2 != 2' returns 'FALSE'
>=	Greater than or equal to	'2 >= 1' returns 'TRUE'
<=	Less than or equal to	'2 <= 1' returns 'FALSE'

Check the class:

```
# Of the straight boolean
base::class(TRUE)
```

```
## [1] "logical"
```

```
# Or of an assigned variable
a <- FALSE
base::class(a)
```

```
## [1] "logical"
```

Rather self-explanatory, **logical** data are most often used when one is checking to see if an expression is either `TRUE` or `FALSE`. That is, when comparing values or objects, such as `x > 15` or `x < y`.

Examples of expressions and their boolean results:

```
# Expressions comparing two numbers
1 < 2
```

```
## [1] TRUE
```

```
2 == 2
```

```
## [1] TRUE
```

```
2 > 1
```

```
## [1] TRUE
```

```
# Or comparing two variables
a <- 1
b <- 2
a < b
```

```
## [1] TRUE
```

3.1.1.1 Boolean with Comparison Operators

Comparison operators are used to compare two values and return a **boolean** value based on the result. The operators and examples are described in Table 3.1.

3.1.1.2 Boolean with Logical Operators

Following on, **logical operators** are used to compare the output of two comparisons. The three options in R are:

1. `AND` operator (`&`),
2. `OR` operator (`|`),
3. and `NOT` operator (`!`)

The `AND` operator takes a logical input on either side of the `&` and returns `TRUE` only if **both** sides of the `&` evaluate to `TRUE`.

For example:

```
# The following will evaluate to true:
TRUE & TRUE
```

```
## [1] TRUE
```

```
# while the following will evaluate to false
FALSE & TRUE
```

```
## [1] FALSE
```

```
TRUE & FALSE
```

```
## [1] FALSE
```

What do you think `FALSE & FALSE` will evaluate to? Note that both sides of `&` are the same.

Using the same format, the `OR` takes a logical input on either side of the `|` and returns `TRUE` if **either** side of the `|` evaluate to `TRUE`. As only one of the arguments needs to evaluate to `TRUE` for `OR` to be true, if the first argument evaluates as `TRUE`, the second will not be checked.

What do you expect the following to evaluate? 1. `TRUE | TRUE` 2. `FALSE | FALSE` 3. `TRUE | FALSE` 4. `FALSE | TRUE`

Conversely, the `NOT` operator `!` takes one logical input and negates (flips) it. So, if a value is `TRUE`, using `!` would negate it and turn it to `FALSE`, and vice versa.

For example:

```
# The following will evaluate to true:
!FALSE
```

```
## [1] TRUE
```

```
!FALSE & TRUE
```

```
## [1] TRUE
```

```
# while the following will evaluate to false
!TRUE
```

```
## [1] FALSE
```

```
!TRUE & TRUE
```

```
## [1] FALSE
```

Table 3.2: A subset of the most common operators grouped and listed by order of precedence, i.e., order of execution.

Order	Operator	Meaning
1.	::	Access functions and variables in a namespace
2.	$	Component extraction
3.	[, [[	Indexing
4.	^	Exponentiation
5.	+, -	Characters representing the sign of a number (i.e., unary minus or plus)
6.	:	Sequence operator (e.g., 1:5)
7.	%%, |>, %*%, etc.	Special operators (e.g., %% for modulus, the remainder from division)
8.	<, >, <=, >=, ==, !=	Ordering and comparison
9.	!	Negation
10.	&	Logical AND
11.	|	Logical OR
12.	~	As found in formulae
13.	->	Rightward assignment
14.	<-	Assignment (right to left)
15.	=	Assignment (right to left)
16.	?	Help

Finally, the `NOT` operator can be combined with comparisons using parentheses to build more complicated comparisons. For example, `!(x > 10)` is equivalent to saying that `x` is not greater than 10, which means that it is less than or equal to 10, i.e., `x <= 10`. As an aside, the `NOT` operator (`!`) can also be used on functions that evaluate to booleans.

3.1.1.3 Operators and Precedence

In a similar vein to the order of operations in mathematics, R executes code based on a precedence, analogous to the mathematical order of operations (remember the acronym PEMDAS or BODMAS from maths at school?). In R and computational languages more broadly, this expanded to include things such as variable assignment logic.

In particular, assignment operators have the (almost) lowest precedence; this means that variable assignment is the last thing that should be executed when processing a row of code. As seen in Table 3.2, part of the reason for preferring a `<-` assignment operator for variables over a more traditional-looking `=`, is that R gives the former a higher precedence. Parentheses blocks, denoted with `()`, can be used to group comparisons or mathematical statements together to ensure that the code block inside is evaluated before continuing to the rest of the code.

The logical `AND` (`&`) has higher precedence than `OR` (`|`):

```
TRUE | TRUE & FALSE # this is equivalent to the following
```

```
## [1] TRUE
```

```
TRUE | (TRUE & FALSE)
```

```
## [1] TRUE
```

```
# Whereas, the "()" force the OR ("|") to evaluate first:
(TRUE | TRUE) & FALSE
```

```
## [1] FALSE
```

What happens if you flip the exercise, replacing the `OR`s with `AND`s? Is this what you expect?

3.1.2 Numeric

A **Numeric** object or value contains only numbers (e.g., 1, 2, 5.5, pi, etc.).

Check the class:

```
# Of the straight number
base::class(5.5)
```

```
## [1] "numeric"
```

```
# Or of an assigned variable
a <- 5
base::class(a)
```

```
## [1] "numeric"
```

Alternatively, `is.numeric` can be used to check to see if an object is **numeric**.

What do you think the following code will evaluate to?

```
x <- 5 base::print(base::is.numeric(x) & (x < 5 | x == 5))
```

See Section 3.1.1.2 for help with logical operators.

The numeric data type is R's default treatment for numbers and includes integers and doubles.

3.1.2.1 Integer

Integers are the whole numbers that come to mind when thinking about a numeric value, e.g., 1, 2, 3, or 4. Integers are specified by putting a capital `L` after an integer value.

Check the class:

```
# Of a whole number
base::is.integer(1000)
```

```
## [1] FALSE
```

```
# Of a specified integer
base::class(1000L)
```

```
## [1] "integer"
```

```
# Or of an assigned variable
a <- 5333L
base::class(a)
```

```
## [1] "integer"
```

That being said, R handles the differences between the usual, unspecified numerics (see Section 3.1.2) and integers in the background, so they are often not noticed. The value of 1,000 has been deliberately chosen to show the differences and similarities between numeric and integer values.

Background differences:

```
base::is.numeric(1000)
```

```
## [1] TRUE
```

```
base::is.integer(1000)
```

```
## [1] FALSE
```

```
base::is.numeric(1000L)
```

```
## [1] TRUE
```

```
base::is.integer(1000L)
```

```
## [1] TRUE
```

Think of **integers** as a subset of **numeric**, and follow a pattern of all **integers** are **numeric**, but not all **numeric** are **integers**.

3.1.2.2 Double

Doubles are any numbers that have a decimal point. Take 5.0 or 5.5 for example.

Check the class:

```
# Of a double number
base::is.double(5.5)
```

```
## [1] TRUE
```

```
base::typeof(5.5)
```

```
## [1] "double"
```

As seen in Section 3.1.2, the number 5.5 is also a **numeric**. Following the phrasing of integers above, all **doubles** are **numeric**, but not all **numeric** values are **doubles**.

3.1.2.3 Integers Versus Doubles

Programmers and analysts will typically use **integers** when they know that decimals will never be needed, for example, when dealing with IDs. The advantage is that **integers** take up less memory, but the trade-off is that they can only go a little more than two billion.

Size and memory differences:

```
# Maximum size of an integer
.Machine$integer.max
```

```
## [1] 2147483647
```

```
# Maximum size of a double (numeric)
.Machine$double.xmax
```

```
## [1] 1.797693e+308
```

```
# Implications of using integers versus numeric/doubles at scale (1 million)
ints <- base::rep(1L, 1e6)
numeric <- base::rep(1, 1e6)

## Check memory usage
utils::object.size(ints)
```

```
## 4000048 bytes
```

```
utils::object.size(numeric)
```

```
## 8000048 bytes
```

This clearly shows the two-time size difference between **integers** and **doubles**, which is why it is helpful to think about data types when working with data. The absolute difference (8 MB vs. 4 MB) may look small in this toy example. However, in real world data where you have multi-million rows, it would become important, especially because R needs to store all active data in the computer memory (RAM).

One place where it pays to be aware of **integers** versus **doubles** is when dividing. The result of the division (/) will always be a **double** because it is calculated using floating-point division.

Differences in dividing:

```
# Dividing numeric values
x <- 5/2
base::print(x)
```

```
## [1] 2.5
```

```
base::typeof(x)
```

```
## [1] "double"
```

```
# Dividing integers
y <- 4L/2L
base::print(y)
```

```
## [1] 2
```

```
base::typeof(y)
```

```
## [1] "double"
```

Can you think of three examples of when you would want to use an integer instead of a numeric value and vice versa?

3.1.2.4 Dates and Date-times

Dates (e.g., `"2025-01-01"`) and **date-times** (e.g., `"2025-01-01 10:30:00"`) are grouped under numeric data types because R actually stores dates in numerical format, technically based on the difference/distance from R's origin date of 1st January 1970.

Working with time seems like an easy thing. For instance, it is easy to say that there are 365 days in a year, except for a leap year, which has 366 days. The convention, then, when converting days into years, is to split the leap day into 0.25 days per year, resulting in 365.25 days per year. Of course, time has an analogous issue surrounding daylight saving time, so one day a year will have 23 hours while another will have 25 hours. Furthermore, the application of daylight saving time is subject to various geopolitical factors. For this reason, the `lubridate` package has become the default when working with dates and times in R, and is fortunately part of the core `tidyverse` universe.

Using the diabeticLabs dataset, explore the SampleDate column

```
lab_data <- utils::read.csv("data/03-diabetic_labs_final_with_missing.csv", head = TRUE)

# Get a summary of the SampleDate
head(lab_data, 3L)
```

```
##   id SampleDate SampleTime Chol   TG      HDL      LDL   Cr  BUN
## 1  5 26/01/2009   16:26:00 4.90 2.10 1.100000 2.500000 56.0 5.10
## 2  5 19/10/2012   10:22:00 3.60 5.10 0.900000 2.500000 63.0 5.00
## 3  5 19/10/2012   09:31:00 4.85 1.63 4.860753 4.860753 96.7 5.24
```

```
# Get a summary of the data
summary(lab_data$SampleDate)
```

```
##    Length     Class      Mode
##       478 character character
```

```
# check the format of the data
base::class(lab_data$SampleDate)
```

```
## [1] "character"
```

From the `utils::head(lab_data, 3L)` call, the `SampleDate` column looks like valid dates, all be it specified in a `day-month-year` format, yet the `class` indicates that the column contains characters (see Section 3.1.3). If the data was used without converting it, R would not behave as expected.

Try a character string comparison

```
# Pick out the first row of the file for simplicity
lab_data <- utils::read.csv("data/03-diabetic_labs_final_with_missing.csv")
base::print(row <- lab_data[1,])
```

```
##   id SampleDate SampleTime Chol  TG HDL LDL Cr BUN
## 1  5 26/01/2009   16:26:00  4.9 2.1 1.1 2.5 56 5.1
```

```
base::print(row$SampleDate < "26/01/2009a")
```

```
## [1] TRUE
```

This evaluates to `TRUE` as R is lexicographically comparing two character strings rather than two dates, and the `a` at the end of `26/01/2009a` makes it longer and therefore a larger string than `26/01/2009`.

Convert to Dates:

```
lab_data <- utils::read.csv("data/03-diabetic_labs_final_with_missing.csv", head = TRUE)
lab_data$SampleDate <- base::as.Date(lab_data$SampleDate)

utils::head(lab_data, n = 3L)
```

```
##   id SampleDate SampleTime Chol   TG      HDL      LDL   Cr  BUN
## 1  5 0026-01-20   16:26:00 4.90 2.10 1.100000 2.500000 56.0 5.10
## 2  5 0019-10-20   10:22:00 3.60 5.10 0.900000 2.500000 63.0 5.00
## 3  5 0019-10-20   09:31:00 4.85 1.63 4.860753 4.860753 96.7 5.24
```

So, this kind of worked, but the date format isn't quite as expected; the year is slightly off. To prevent this from happening, specify the format of the date, where `%d`, specifies the location of the days, `%m` specifies the location of the month, and `%Y` specifies the location of the year, all stitched together with the linking character `/` in this case.

Convert to Dates using base and lubridate's as_date:

```
lab_data <- utils::read.csv("data/03-diabetic_labs_final_with_missing.csv")

# Using base R
lab_data$SampleDate <- base::as.Date(lab_data$SampleDate, format = "%d/%m/%Y")
utils::head(lab_data, n = 3L)
```

```
##   id SampleDate SampleTime Chol   TG      HDL      LDL   Cr  BUN
## 1  5 2009-01-26   16:26:00 4.90 2.10 1.100000 2.500000 56.0 5.10
## 2  5 2012-10-19   10:22:00 3.60 5.10 0.900000 2.500000 63.0 5.00
## 3  5 2012-10-19   09:31:00 4.85 1.63 4.860753 4.860753 96.7 5.24
```

```
# Using lubridate
lab_data <- utils::read.csv("data/03-diabetic_labs_final_with_missing.csv")
lab_data$SampleDate <- lubridate::as_date(
  lab_data$SampleDate, format = "%d/%m/%Y")

utils::head(lab_data, n = 3L)
```

```
##   id SampleDate SampleTime Chol   TG      HDL      LDL   Cr  BUN
## 1  5 2009-01-26   16:26:00 4.90 2.10 1.100000 2.500000 56.0 5.10
## 2  5 2012-10-19   10:22:00 3.60 5.10 0.900000 2.500000 63.0 5.00
## 3  5 2012-10-19   09:31:00 4.85 1.63 4.860753 4.860753 96.7 5.24
```

Now that formatting is correct, let's convert to Dates and try again:

```
lab_data <- utils::read.csv("data/03-diabetic_labs_final_with_missing.csv", head = TRUE)
lab_data$SampleDate <- lubridate::as_date(lab_data$SampleDate, format = "%d/%m/%Y")
```

```
# Pick out the 1st row of the file for simplicity
base::print(row <- lab_data[1,])
```

```
##   id SampleDate SampleTime Chol  TG HDL LDL Cr BUN
## 1  5 2009-01-26   16:26:00  4.9 2.1 1.1 2.5 56 5.1
```

```
# check the comparison of Date and String
base::print(row$SampleDate < "2009-01-26a")
```

```
## [1] FALSE
```

```
# check the comparison of Date and character string that can be coerced to date
base::print(row$SampleDate <= "2009-01-26")
```

```
## [1] TRUE
```

```
# check the comparison of Date and Date
base::print(row$SampleDate <= lubridate::as_date("2009-01-27"))
```

```
## [1] TRUE
```

In the first comparison of dates and characters, R is trying to coerce the character string to a date. This fails and therefore returns **FALSE**. In the second such example, without the `a`, R can coerce the character string to a date and perform the comparison, where the dates are evaluated to be equal. The third example is preferable because it explicitly says that `2009-01-27` is a date.

Try summary and class one more time:

```
lab_data <- utils::read.csv("data/03-diabetic_labs_final_with_missing.csv", head = TRUE)
lab_data$SampleDate <- as.Date(lab_data$SampleDate, format = "%d/%m/%Y")

base::summary(lab_data$SampleDate)
```

```
##         Min.      1st Qu.       Median         Mean      3rd Qu.         Max.
## "2009-01-01" "2010-01-01" "2011-02-27" "2011-01-21" "2012-01-22" "2012-12-31"
```

```
base::class(lab_data$SampleDate)
```

```
## [1] "Date"
```

From this it follows that `SampleDate` is from the date class and the minimum (`Min.`) date is the 1st of January 2009. For continuous data, the `summary` function also provides the first quartile (`1st Qu.`), median (`Median`), mean (`Mean`), third quartile (`3rd Qu.`), and maximum (`Max.`) values. As has been alluded to throughout the section, dates can and are stored in different formats, and R might not recognise the date as a `Date` object during import. The alternative to specifying the `format =` in `as_date`, there are a couple of functions from the `lubridate` package that respecify the format:

Creating `Date` objects from character strings using lubridate and checking the class:

```
library(lubridate)
lubridate::is.Date(lubridate::dmy('18-09-2025'))
```

```
## [1] TRUE
```

```
lubridate::is.Date(lubridate::mdy('09-18-2025'))
```

```
## [1] TRUE
```

```
lubridate::is.Date(lubridate::ymd('2025-09-18'))
```

```
## [1] TRUE
```

Finally, **date-time** objects add a time component to the date (e.g., `lubridate::as_date('2025-09-19')` is the year-month-day Date format, while `lubridate::as_datetime('2025-09-19 14:30:00')` adds the 24-hour time down to the second).

Use as_datetime to convert SampleTime to date-time

```
lab_data <- utils::read.csv("data/03-diabetic_labs_final_with_missing.csv")

lab_data$SampleTime <- lubridate::as_datetime(
  lab_data$SampleTime, format = "%H:%M:%S")

utils::head(lab_data, n = 3L)
```

```
##   id SampleDate          SampleTime Chol   TG      HDL      LDL   Cr  BUN
## 1  5 26/01/2009 0000-01-01 16:26:00 4.90 2.10 1.100000 2.500000 56.0 5.10
## 2  5 19/10/2012 0000-01-01 10:22:00 3.60 5.10 0.900000 2.500000 63.0 5.00
## 3  5 19/10/2012 0000-01-01 09:31:00 4.85 1.63 4.860753 4.860753 96.7 5.24
```

Note that the date part of the date-time is a nonsensical `0000-01-01`. The `as_hms` function within the `hms` package allows for the date component to be removed.

Use as_datetime and as_hms to store just the time component of date-time

```
lab_data <- utils::read.csv("data/03-diabetic_labs_final_with_missing.csv")

lab_data$SampleTime <- hms::as_hms(lubridate::as_datetime(
  lab_data$SampleTime, format = "%H:%M:%S"))

utils::head(lab_data, n = 3L)
```

```
##   id SampleDate SampleTime Chol   TG      HDL      LDL   Cr  BUN
## 1  5 26/01/2009   16:26:00 4.90 2.10 1.100000 2.500000 56.0 5.10
## 2  5 19/10/2012   10:22:00 3.60 5.10 0.900000 2.500000 63.0 5.00
## 3  5 19/10/2012   09:31:00 4.85 1.63 4.860753 4.860753 96.7 5.24
```

This is the jumping-off point for data cleaning.

3.1.3 Character

Roughly speaking, a **character** is anything that can be typed on a keyboard, as long as it is encapsulated in a pair of quotes (""). A **character string** is a series of characters within a pair of quotes.

Check the class:

```
# Of the straight character
base::class("BMI")
```

```
## [1] "character"
```

```
# Or of an assigned variable (note the quotes)
a <- "body mass index"
class(a)
```

```
## [1] "character"
```

For example, "**body mass index (BMI)**" and "body mass index" are each character objects with a single character value (the string). Notably, R allows single and double quotes to be used to encapsulate and denote character strings; however, they must be used in pairs (i.e., whatever symbol starts the character string must **also** end the character string). Analogously to checking the class of numeric objects, `is.character` can be used to check whether an object is a character string.

What do you expect the class of `"TRUE"` to be? Try and see. Is this what you expected?

3.1.4 Complex

Complex values are included here for completeness, but something probably went wrong if they appear in the day-to-day health data science work. Briefly, **complex** values are expressed in the form a + bi, where a and b are the "real" numbers (i.e., numeric), and i is the imaginary unit, derived from the square root of -1.

Check the class:

```
# Of the straight complex number
class(2i)
```

```
## [1] "complex"
```

```
is.complex(2i)
```

```
## [1] TRUE
```

```
# Or of an assigned variable
a <- 3+ 2i
class(a)
```

```
## [1] "complex"
```

Unless explicitly working with **complex** numbers, start debugging and looking for where negative numbers could be produced when `class` returns "complex".

3.2 Important Built-in Constants

This section builds on the concept of reserved words presented in Section 2.8. The built-in **constants** is a type of reserved word and represents common fixed values or special cases of data. They are built into R, and as such, are always there and do not need to be created.

3.2.1 Infinite

Infinite values in R are stored as either `-Inf` for negative infinity, and `Inf` for positive infinity values. These are useful in mathematical computations that result in values that are beyond the largest representable number.

An example of `Inf`, where a value is divided by zero:

```
i <- 5/0
print(i)
```

```
## [1] Inf
```

3.2.2 Missing Data

The next couple of sections will use the `diabetic` dataset that's part of the `survival` package, combined with blood test data modified from the Health Test by Blood Dataset.

3.2.2.1 Not available (NA)

\acr{NA}, meaning not available, is generally used and interpreted as a missing value. It is important to note that R often, but not always, defaults to allowing `**\acr{NA}**` values to cause problems so that missing values are not missed by default. The Health Test by Blood Dataset has been deliberately modified to introduce **\acr{NA}** values to help illustrate how to handle them.

An example of how data are handled differently depending on `rm.na` is provided:

```
lab_data <- utils::read.csv("data/03-diabetic_labs_final_with_missing.csv", head = TRUE)

# Check out the top of the file
utils::head(lab_data)
```

```
##   id SampleDate SampleTime Chol   TG      HDL      LDL   Cr  BUN
## 1  5 26/01/2009   16:26:00 4.90 2.10 1.100000 2.500000 56.0 5.10
## 2  5 19/10/2012   10:22:00 3.60 5.10 0.900000 2.500000 63.0 5.00
## 3  5 19/10/2012   09:31:00 4.85 1.63 4.860753 4.860753 96.7 5.24
## 4 14 13/07/2009   10:23:00 6.49 1.48 1.370000 4.170000 41.1 3.46
## 5 14 07/10/2010   11:05:00   NA 0.92 4.860753 4.860753 60.0 4.72
## 6 14 22/02/2011   13:11:00 4.80 1.00 1.720000 2.570000 73.0 4.67
```

```
# Find the mean cholesterol (Chol) value
base::mean(lab_data$Chol, na.rm = FALSE)
```

```
## [1] NA
```

```
# Find the mean cholesterol (Chol) value, remvoing NA values
base::mean(lab_data$Chol, na.rm = TRUE)
```

```
## [1] 4.969544
```

The `head` function returns the first rows of the data set. Note the `**\acr{NA}**` value in the fifth row. Then, the `mean` function returns `**\acr{NA}**` rather than throwing an error or warning message. The `**\acr{NA}**` values need to be removed using the `na.rm = TRUE`, with `rm` standing for remove. Use `is.na` to check if an object is `**\acr{NA}**`.

3.2.2.2 Not a Number (NaN)

not a number (NaN) (pronounced 'nan') is a special kind of missing value used to represent undefined or unrepresentable values (i.e., impossible values). The most common reason for this is that a function or a fragment of code involves dividing by zero.

`**\acr{NaN}**` is typically generated from mathematical operations which are undefined:

```
# Diving by zero
0/0
```

```
## [1] NaN
```

```
# In multiplication
0 * Inf
```

```
## [1] NaN
```

```
# Indeterminate result
Inf - Inf
```

```
## [1] NaN
```

```
# When taking the square root of a negative number
sqrt(-1)
```

```
## Warning in sqrt(-1): NaNs produced
```

```
## [1] NaN
```

See Section 3.1.4 for a refresher on sqrt(-1).

`**\acr{NaN}**` generally behaves like `**\acr{NA}**`:

```
# To allow for comparison
a <- c(NA, NaN)

# In multiplication
10 * a
```

```
## [1]  NA NaN
```

```
# Assert equal to a number
a == 5
```

```
## [1] NA NA
```

```
# Check if NA
is.na(a)
```

```
## [1] TRUE TRUE
```

```
# Check if NaN
is.nan(a)
```

```
## [1] FALSE  TRUE
```

Use `is.nan` to check if an object is `NaN`.

3.2.2.3 Null

A **null** object (remember, everything in R is an object) is an object that occurs when an expression or function returns an undefined value. The reserved word in R is `NULL`, in all capitals. The most common use of null is to represent lists (see Section 3.3.2) with zero length, usually before objects have been added.

Exploring `NULL`:

```
n <- NULL
base::length(n)
```

```
## [1] 0
```

```
utils::str(n)
```

```
##  NULL
```

```
a <- c()
base::is.null(a)
```

```
## [1] TRUE
```

Null can also be used to drop columns from a data frame by assigning the variable to be `NULL`. This is called being 'null-able', and only some objects are amenable to this behaviour.

A fun side benefit of NULL is that you can use it to remove columns in a data frame:

```
data("iris")

#Get all column names
base::colnames(iris)
```

```
## [1] "Sepal.Length" "Sepal.Width"  "Petal.Length" "Petal.Width"  "Species"
```

```
#drop the Sepal.Length column
iris[, c("Sepal.Length")] <- NULL

#confirm that Sepal.Length has been dropped
base::colnames(iris)
```

```
## [1] "Sepal.Width"  "Petal.Length" "Petal.Width"  "Species"
```

Table 3.3: A subset of the base data structures summarised by the type or types of data it can hold.

Dimensionality	Homogeneous	Heterogeneous
1D	Vector	List
2D	Matrix	Data frame & Tibble
nD	Array	

3.3 Data Structures

R's base data structures can be summarised based on how many data types they can contain (homogeneous if only one, otherwise heterogeneous), and how many dimensions (1D, 2D, or nD) they can hold, which are presented in Table **??**.

One thing to note is that data structures should only be used to store a collection of **related** objects. For instance, a **list** of participants in a study, or a **data frame** of information about hospital stays. Don't use a data structure to store objects that **are not** related. An example would be the minimum, maximum, average, and count of a variable. This type of issue should be addressed in alternative ways, such as storing the values separately or developing a custom Object (see Section 3.3.6).

3.3.1 Vector

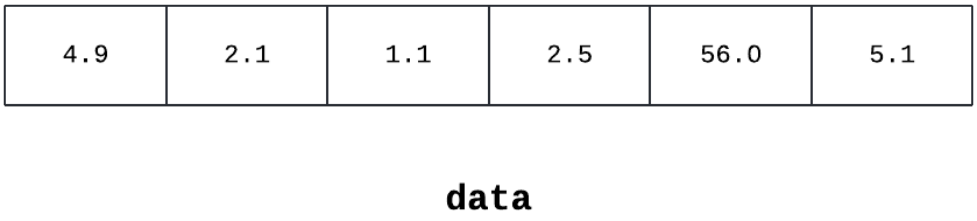


Figure 3.2: An illustration of a vector containing numeric values.

Within R, **vectors** are the simplest of the data structures and are one of the most commonly used to aid in the preparation and presentation of data. A vector is an ordered collection of values. They will store only objects of the same type. For instance, vectors are often used to store different colours to be used in a plot or other values that a variable can represent (this will be demonstrated in Chapter 4).

Vectors are generally created using the `c` function:

```
# a vector containing a collection of numbers
numbers <- c(1, 2, 3, 4)

# display the vector
numbers
```

```
## [1] 1 2 3 4
```

```
# Find out the data class being stored within the vector
base::typeof(numbers)
```

```
## [1] "double"
```

As mentioned above, a vector can contain only one type of data. If more than one type is supplied, R will try to coerce, i.e., convert, the lower-level, more specific, data types (e.g., logical) to a higher-order, more general type (e.g., characters) so that the function will work. Use the `typeof` function to check the data class of an object.

What happens when numbers and characters are forced into the same vector?

```
# A factor containing different data types, where 1 and 3 are integers
# and "two" and "four" are characters.
numbers <- c(1, "two", 3, "four")

# Display the vector
numbers
```

```
## [1] "1"    "two"  "3"    "four"
```

```
# Find the data class that is stored within the vector
class(numbers)
```

```
## [1] "character"
```

R coerced the numbers into characters, as the characters can represent both types of data.

What would R coerce the following vectors into? 1. `x <- c(TRUE, 1, FALSE, 0)` 2. `x <- c(0, FALSE, "NO")`

3.3.2 List

Lists are one step up in complexity from vectors in that they are heterogeneous and, as such, can contain a variety of data types within a single list. However, like vectors, lists can store only one dimension of data, even though each element in a list can be completely different. It is also possible to create a list of lists.

Lists are created using the `list` function:

```
# A list containing different data types, where 1 and 3 are numbers
# and "two" and "four" are characters.
numbers <- list(1, "two", 3, "four")

# Find out the data class being stored within the vector
utils::str(numbers)
```

```
## List of 4
##  $ : num 1
##  $ : chr "two"
##  $ : num 3
##  $ : chr "four"
```

3.3.3 Data Frames and Tibble

Tibbles and **data frames** appear as tables within R. Like a list, data frames is a collection of a variety of data types. However, there are more restrictions on what can be put into a data frame because the data frame has to be 'tabular' in nature (or rectangular in shape). Typically, a data frame is a collection of numeric (integer, double, etc.), string/factor, Boolean, or date/time vectors of the same length. Data are considered to be **tidy** when each **variable** has its own column and each **observation** (or case) has its own row.

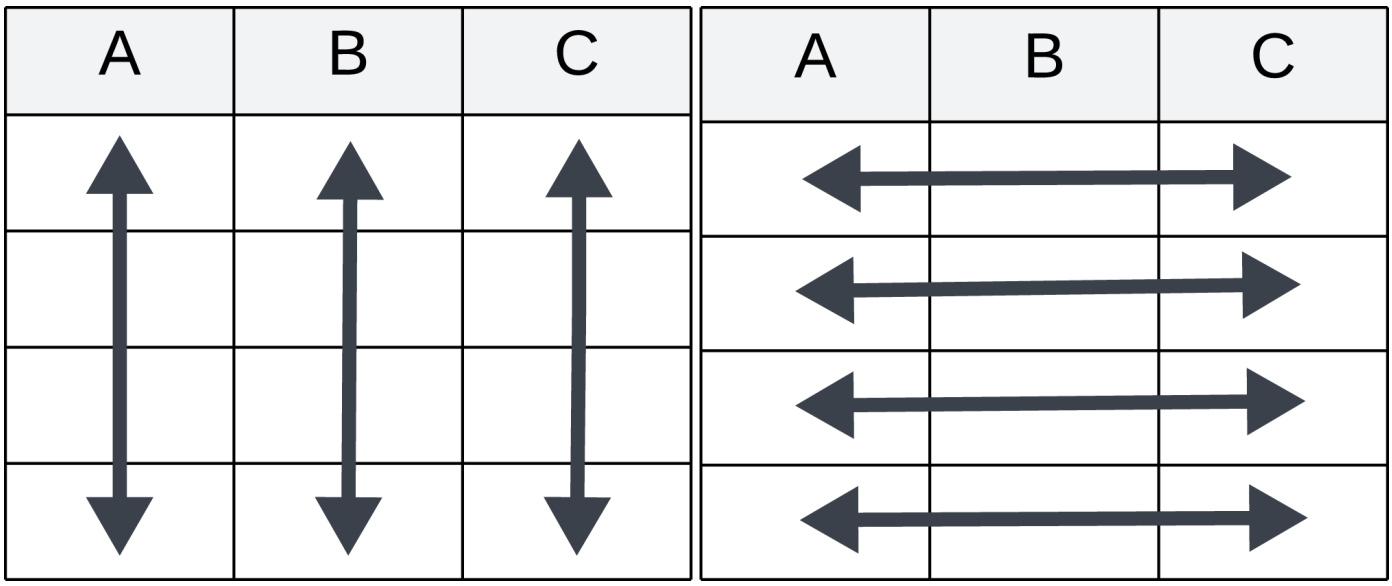


Figure 3.3: A visual representation of tabular tidy data, where columns contain one variable and rows contain one observation or case.

3.3.3.1 Data Frame

Data frames are constructed using `data.frame` by providing a sequence of vectors, one for each variable.

Example of how to build a data frame:

```
# Specify the data frame
df <- base::data.frame(
   id = c(5, 14, 16),
   SampleDate = c(as.Date("2009-01-26"), as.Date("2009-07-13"),
           as.Date("2010-05-30")),
   Cr = c(56, 41.1, 71)
)

# examine the structure of the data frame.
utils::str(df)
```

```
## 'data.frame':    3 obs. of  3 variables:
##  $ id        : num  5 14 16
##  $ SampleDate: Date, format: "2009-01-26" "2009-07-13" ...
##  $ Cr        : num  56 41.1 71
```

The function `nrow` can be used to get the number of rows (observations) in a data frame, while `ncol` can be used to get the number of columns (variables).

Try your hand at creating a data frame.

One important note: make sure to use = instead of <- when defining the variables of a data frame.

3.3.3.2 Tibble

Tibbles are tidyverse's modern take on R's `data.frame`, in that they have kept the most used features and have modified or dropped others. Of note, most of the tidyverse functions return tibble, while most other R packages take and return data frames. In an analogous grammar, a tibble is constructed using `tibble`, providing a sequence of vectors, one for each variable.

Example of how to build a tibble:

```
# Specify the data frame
tib <- tibble::tibble(
   id = c(5, 14, 16),
   SampleDate = c(as.Date("2009-01-26"), as.Date("2009-07-13"),
           as.Date("2010-05-30")),
   Cr = c(56, 41.1, 71)
)

# examine the structure of the data frame.
utils::str(tib)
```

```
## tibble [3 x 3] (S3: tbl_df/tbl/data.frame)
##  $ id        : num [1:3] 5 14 16
##  $ SampleDate: Date[1:3], format: "2009-01-26" "2009-07-13" ...
##  $ Cr        : num [1:3] 56 41.1 71
```

From the first line of the `str` output, `tib` object is a `tibble`, which extends the functionality of a `data.frame`.

Try creating a tibble using an analogous structure to the one created above.

3.3.3.3 Tibble Versus Data frame

In day-to-day use, it is not that important to know or focus on the differences between data frames and tibbles. Data frames can generally be converted to tibbles using `as_tibble`.

Converting a data frame to a tibble using the tibble presented in Section 3.3.3.2

```
library(tibble)
df <- base::data.frame(
    id = c(5, 14, 16),
    SampleDate = c(as.Date("2009-01-26"), as.Date("2009-07-13"),
            as.Date("2010-05-30")),
    Cr = c(56, 41.1, 71)
)
# Data frame to tibble
utils::str(df)
```

```
## 'data.frame':    3 obs. of  3 variables:
##  $ id        : num  5 14 16
##  $ SampleDate: Date, format: "2009-01-26" "2009-07-13" ...
##  $ Cr        : num  56 41.1 71
```

```
t <- tibble::as_tibble(df)
utils::str(t)
```

```
## tibble [3 x 3] (S3: tbl_df/tbl/data.frame)
##  $ id        : num [1:3] 5 14 16
##  $ SampleDate: Date[1:3], format: "2009-01-26" "2009-07-13" ...
##  $ Cr        : num [1:3] 56 41.1 71
```

Tibbles can **always** be cast into data frames using `as.data.frame`.

Converting a tibble to a data frame using the data frame presented in Section 3.3.3.2:

```
library(tibble)
tib <- tibble::tibble(
    id = c(5, 14, 16),
    SampleDate = c(as.Date("2009-01-26"), as.Date("2009-07-13"),
            as.Date("2010-05-30")),
    Cr = c(56, 41.1, 71)
)
# Tibble to data frame
utils::str(tib)
```

```
## tibble [3 x 3] (S3: tbl_df/tbl/data.frame)
##  $ id        : num [1:3] 5 14 16
##  $ SampleDate: Date[1:3], format: "2009-01-26" "2009-07-13" ...
##  $ Cr        : num [1:3] 56 41.1 71
```

```
df1 <- base::as.data.frame(tib)
utils::str(df1)
```

```
## 'data.frame':    3 obs. of  3 variables:
##  $ id        : num  5 14 16
##  $ SampleDate: Date, format: "2009-01-26" "2009-07-13" ...
##  $ Cr        : num  56 41.1 71
```

Visible in the two examples above, a benefit of tibbles is that their print function provides summary statistics, and it does not try to print the whole thing. That being said, almost every function that takes a tibble will also take a data frame, although the analogous is not true. If working within the tidyverse, consider the two interchangeable and focus on using tibbles as they are generally more user-friendly. However, when working outside of tidyverse, consider using data frames, as some packages and functions will only work with data frames.

3.3.4 Factor

Factors are a data structure that should be used when dealing with categorical data (i.e., data that contain a finite number of distinct values, for example, biological sex or countries). Variables can be converted to factors using `as.factor` when the alphanumeric order of levels is desired (i.e., alphabetical order).

Example of converting a variable to a factor using `as.factor`:

```
# Load in the demographic data
demog <- utils::read.csv("data/03-diabetic_demog.csv")

# Check the basic structure
utils::str(demog)
```

```
## 'data.frame':    197 obs. of  5 variables:
##  $ id    : int  5 14 16 25 29 46 49 56 61 71 ...
##  $ age   : int  28 12 9 9 13 12 8 12 16 21 ...
##  $ Gender: chr  "M" "M" "F" "M" ...
##  $ BMI   : int  26 23 30 28 36 20 28 35 33 38 ...
##  $ SIMD  : int  4 4 2 3 4 2 4 3 5 2 ...
```

```
# Convert gender to a factor
demog$Gender <- base::as.factor(demog$Gender)

# Check the structure again
utils::str(demog)
```

```
## 'data.frame':    197 obs. of  5 variables:
##  $ id    : int  5 14 16 25 29 46 49 56 61 71 ...
##  $ age   : int  28 12 9 9 13 12 8 12 16 21 ...
##  $ Gender: Factor w/ 2 levels "F","M": 2 2 1 2 2 1 1 2 1 2 ...
##  $ BMI   : int  26 23 30 28 36 20 28 35 33 38 ...
##  $ SIMD  : int  4 4 2 3 4 2 4 3 5 2 ...
```

It is typically a vector or variable (column) in a data frame. In particular, it is important to ensure that ordinal data (data that have a natural rank order, e.g., quintiles of deprivation within **SIMD**) are converted into factors to prevent R from extrapolating values between individual levels, as R would do with continuous data,(e.g., keeping the order of 1, 2, 3, 4, and then 5 for the quintiles of **SIMD**, or reporting countries in alphabetical order).

Example of converting a variable to a factor using `as.factor`:

```
# Load in the demographic data
demog <- utils::read.csv("data/03-diabetic_demog.csv")

# Convert SIMD to a factor
demog$SIMD <- base::as.factor(demog$SIMD)

# Use summary() to check the order of the factor levels
base::summary(demog$SIMD)
```

```
##  1  2  3  4  5
## 34 42 36 50 35
```

From the `summary` function call, **SIMD** quintile 1 (the most deprived) has 34 individuals and is the first level of the **SIMD** factor. To flip the order, `factor` can be used to order and/or rename levels within a factor, where desired.

Example of converting a variable to a factor using `factor` to add some additional information and specify the order of the levels:

```
# Load in the demographic data
demog <- utils::read.csv("data/03-diabetic_demog.csv")

# Convert SIMD to a factor
demog$SIMD <- base::factor(demog$SIMD,
                           levels = c(5, 4, 3, 2, 1),
                           labels = c("5 (least deprived)", "4", "3", "2",
                           "1 (most deprived)")
                         )

# Use base::summary() to check the order of the factor levels
base::summary(demog$SIMD)
```

```
## 5 (least deprived)                  4                  3                  2
##                 35                 50                 36                 42
##  1 (most deprived)
##                 34
```

In addition to being useful for statistics, factors are also more memory-efficient than storing information as a character string.

The impact of converting fields from character strings to factors:

```
data("who")

# Get a summary of the country variable (the first column)
# in the 'who' dataset
summary(who$country)
```

```
##    Length     Class      Mode
##      7240 character character
```

```
# Get the initial size of the country field within the dataset
format(object.size(who$country), unit = 'auto')
```

```
## [1] "70.3 Kb"
```

From the results of the `summary` call, it is possible to see that there are 7,240 records within the `country` field and that the field contains character strings, as denoted by `Class character`. That, at the moment, provides information on the number of records (`Length` as 7240), but much more beyond that.

Now convert the country field to a factor and observe how the amount of memory needed to store it changes:

```
# Load data
utils::data("who")

# Convert the country field to a factor
who$country <- as.factor(who$country)
```

```
# Rerun the first five levels of the 'country' field
base::summary(who$country, maxsum = 5)
```

```
##    Afghanistan        Albania        Algeria American Samoa        (Other)
##             34             34             34             34           7104
```

```
# Check the new size of the 'who' dataset
format(object.size(who$country), unit = 'auto')
```

```
## [1] "44.1 Kb"
```

Converting `country` reduced the amount of memory needed to store that field by just over half. Additionally, the `summary` function now provides a more meaningful result.

How memory-efficient can you make the `who` dataset in three minutes? Which columns did you change, to what, and why?

3.3.5 Array

Compared to the other data structures mentioned in Sections 3.3.1 through 3.3.3.1, **arrays** can have varying dimensions, often more than two, but must contain the same type of data. Arrays are created using `array`, which takes the data first, and then the dimensions through the `dim =`, as parameters.

Examples of creating arrays:

```
# An example of a 2D array
my_2d_array <- base::array(1:12, dim = c(3, 4))
base::print(my_2d_array)
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    4    7   10
## [2,]    2    5    8   11
## [3,]    3    6    9   12
```

```
# An example of a 3D array
my_3d_array <- base::array(1:24, dim = c(3, 4, 2))
base::print(my_3d_array)
```

```
## , , 1
##
##      [,1] [,2] [,3] [,4]
## [1,]    1    4    7   10
## [2,]    2    5    8   11
## [3,]    3    6    9   12
##
## , , 2
##
##      [,1] [,2] [,3] [,4]
## [1,]   13   16   19   22
## [2,]   14   17   20   23
## [3,]   15   18   21   24
```

The first example above is similar to a matrix, data frame, or tibble. The second example illustrates the n-dimensionality that arrays allow. In addition to `str`, `nrow` and `ncol` can be used to retrieve individual dimensions.

3.3.6 Others

A few other types of data structures that are less commonly used include matrices, `data.table`, and custom objects. **Matrices** are effectively the matrices of linear algebra and, as such, are often used to represent the data used in mathematical models. Their functionality is provided in base R. Matrices are two-dimensional data structures, similar to data frames. However, unlike data frames, all elements of a matrix need to be of the same data type.

The **data.table** data structure is provided through the `data.table` package and provides an enhanced version of data frames for working with larger data sets or where speeding up computational time is desired. Check out https://cran.r-project.org/web/packages/data.table/vignettes/datatable-intro.html for further information.

Ultimately, sometimes the standard data structures just don't quite fulfil all of the requirements. If this is the case, consider whether the code and data requirements can be restructured to use one of the previously mentioned data structures. If restructuring does not work, this is the point of jumping off into the deeper end of defining **custom object** classes using **object-oriented programming** becomes a consideration. This is beyond the scope of the course, but additional information can be found here:

- https://adv-r.hadley.nz/oo.html
- https://stat.ethz.ch/R-manual/R-devel/library/methods/html/setClass.html

3.4 Statistical Data Types

Now that the computational aspects of defining data types and their containers have been covered, this section will cover the statistical aspects of the same topic. From the statistical side, the main types of statistical data that will be used or modelled are nominal, ordinal, interval, and ratio. Each of the four statistical data types provides different kinds of information and requires different interpretations.

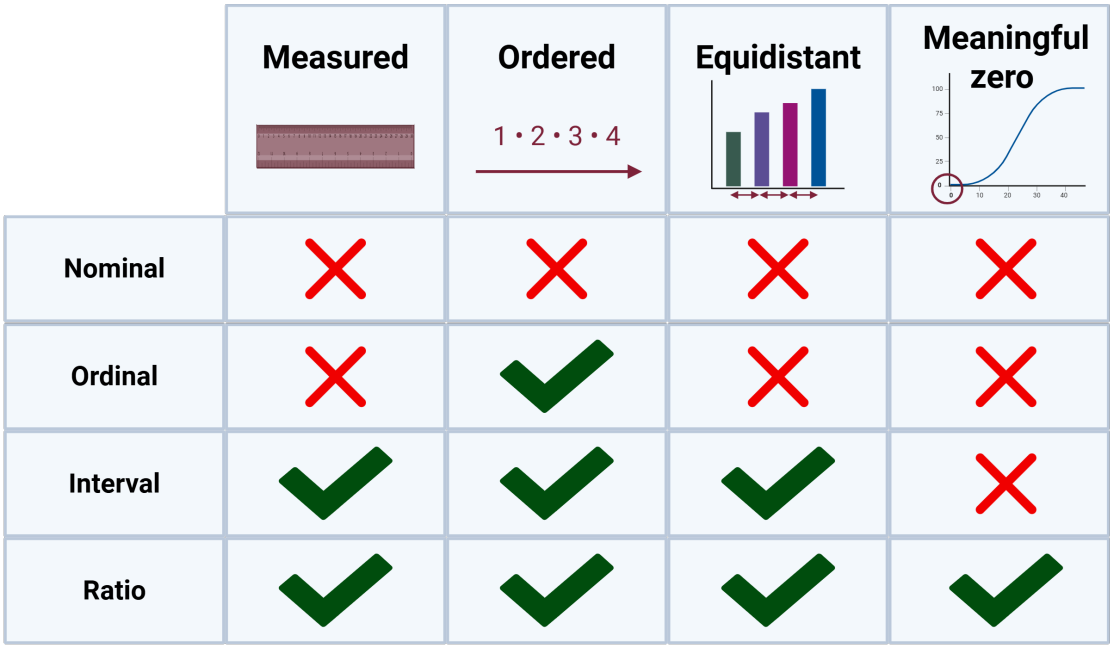


Figure 3.4: An illustration of differences and commonalities between statistical data types.

3.4.1 Nominal Data

Starting from the simplest form, **nominal data** are defined as the data used to name (hence nominal borrowing from the Latin version for concerned with names). As illustrated in Figure 3.4, nominal data are data such as patient ID, nationality, ethnicity, or name that are used to identify objects or events. The numbers do not have an order, implying that one value is not superior to another; nor is a zero meaningful; nominal data classifies objects. Use factors (see Section 3.3.4) when dealing with nominal data. **Dichotomous data** are a subtype of nominal data that have only two categories. Examples for this include death status (alive versus dead) or sex (men versus women).

3.4.2 Ordinal Data

In contrast to nominal data, **ordinal data** have an inherent order but do not provide an amount or a degree of quality. Often, one means that something is better or higher in rank than that in rank number 2, rank 2 is better or higher than rank 3, and so forth. Notably, the numerical difference between the levels can be arbitrary, and the levels do not need to differ by a fixed amount, unlike interval data. Both ordinal and nominal fall under the colloquial term of **categorical** data.

3.4.3 Interval Data

Interval data are numerical data that have equal distances between adjacent values and no meaningful zero. There can be a finite number of intervals, where the data are organised into categories (**discrete**), or there can be an infinite number of intervals (**continuous**). Either way, the interval data must have an equal space between the different values, and hence the 'interval' in the interval data. Examples of interval data include dates (e.g., 2022, 2023) or times (e.g., 5 am, 5 pm).

3.4.4 Ratio Data

Ratio data are data that are numeric, have equal distance between adjacent values (a commonality with interval data above), and have a meaningful zero, unlike the other numerical data types. Ratio data can either be **discrete** or **continuous**. The key point is that the ratio data must have a non-arbitrary zero value.

Can you come up with two examples for each statistical data type: 1. Nominal 2. Ordinal 3. Interval 4. Ratio

And how would you represent these in code?

3.5 Tidyverse

The `tidyverse` has become a powerhouse set of R packages designed to work seamlessly together to streamline the data science process and development of pipelines for data science. In short, within `tidyverse`, all packages share the same underlying design, philosophy, grammar, and data structures (e.g., tibbles from Section 3.3.3.2). Some `tidyverse` packages have already been introduced, such as `lubridate` in Section 3.1.2.4.

Further reading: * The main textbook: https://r4ds.hadley.nz/ * The history of Tidyverse: https://hadley.github.io/25-tidyverse-history/

3.5.1 Piping

One of the big innovations that came with `magrittr`, an integral part of tidyverse, is **piping**. In plain English, pipes work by 'taking *this* object, passing it to a function to do something, and then potentially passing it to another function, and so on.' Instead of nesting functions, the pipe allows the result of each step to be passed to the next function using `%>%` (or the newer `|>` in base R).

Example of piping data:

```
# Define a vector of data
data <- c(1, 2, 5, 7)

# finding the mean without using pipes
base::mean(data)
```

```
## [1] 3.75
```

```
# finding the mean while using pipes
data |> base::mean()
```

```
## [1] 3.75
```

Pipes mean that data are carried forward and don't need to be re-included as the first argument to a function; however, if data needs to be passed in as somewhere other than the first argument, use `.` as a placeholder.

Example of piping data:

```
# Where the data are the first argument
data %>% some_function(argument1, agument1, ...)

# Where the data need to be the second argument
data %>% some_function(argument1, data = .)
```

Piping can be a useful way to write easy-to-understand code as it avoids nesting or multiple, repeated simple operations. It also shows the operations step by step. More examples will be shown below of how piping can make code more readable. Generally speaking, the goal is to write R code so that it is as readable as possible for two reasons: (1) it allows others to understand it more easily, and (2) it minimises human errors in coding.

3.5.2 Tidy Data

The philosophy for tidy data was developed around avoiding the Anna Karenina principle, which comes from the first line of Tolstoy's *Anna Karenina*, "All happy families are alike; each unhappy family is unhappy in its own way". This is evidenced by the quote from Hadley Wickham, the leader of the tidyverse team, "Tidy datasets are all alike, but every messy dataset is messy in its own way". This drives the tidyverse definition of **tidy data**. Within the tidyverse, and general good practice in R, tidy data frames and tibbles follow the following three rules:

1. Each variable is a column, and each column is a variable;
2. Each observation is a row, and each row is an observation;
3. And each value is a cell, and each cell is a single value.

The value of using tidy data is that picking one consistent way to store data means that it's easier to learn new tools and take advantage of R's vectorised nature. It's also handy that all packages in the tidyverse are designed to work with tidy data.

3.5.2.1 Wide Versus Long Data

To date, most of the data that has been presented are in what is called **wide format**, where each participant/entity has one row per table and multiple columns of values. In the case of the lab data, each sample at a given time has multiple test results; although some patients have multiple samples, this is still considered wide data. The inverse of this is called **long data**, where the participant/entity has multiple rows, and with the type of record recorded as a variable.

id	SampleDate	SampleTime	Chol	TG	HDL	LDL	Cr	BUN
5	26/01/2009	16:26:00	4.9	2.1	1.1	2.5	56	5.1
14	13/07/2009	10:23:00	6.49	1.48	1.37	4.17	41.1	3.46
16	30/05/2010	16:31:00	3.3	1.7	1.27	1.93	71	4.95

id	SampleDate	SampleTime	Test	Value
5	26/01/2009	16:26:00	Chol	4.9
5	26/01/2009	16:26:00	TG	2.1
5	26/01/2009	16:26:00	HDL	1.1
5	26/01/2009	16:26:00	LDL	2.5
5	26/01/2009	16:26:00	Cr	56
5	26/01/2009	16:26:00	BUN	5.1
14	13/07/2009	10:23:00	Chol	6.49
⋮	⋮	⋮	⋮	⋮
16	30/05/2010	16:31:00	BUN	4.95

Figure 3.5: A visual representation of wide versus long data, in wide data, each row contains multiple measurements, versus in long data, where each row contains one measurement.

The wide format data are easier for reading, while the long format data are typically easier for analysis and data visualisation.

3.5.2.2 Pivoting Data

Data frames and tibbles can easily be converted from wide to long and vice versa through **pivoting**. The term **pivot** comes from the idea of a pivot table implemented in spreadsheets, where a pivot table rotates data so that they can be viewed from different angles. The phrase **pivot longer** means taking columns/variables and rotating them *downwards* into rows/observations. This can be done using `pivot_longer` from the `tidyr` package.

Convert from wide to long format:

```
library(tidyr)

wide_lab <- utils::read.csv("data/03-diabetic_labs_final_with_missing.csv", head = TRUE)

#pivot around id, SampleDate and SampleTime
```

```
long_data <- wide_lab %>% tidyr::pivot_longer(
    # columns to pivot
    cols = c("Chol", "TG", "HDL", "LDL", "Cr", "BUN"),
    # new column name for old headers
    names_to = "Test",
    # new column name for values
    values_to = "Value"
)

head(long_data, n=5)
```

```
## # A tibble: 5 x 5
##      id SampleDate SampleTime Test  Value
##   <int> <chr>      <chr>      <chr> <dbl>
## 1     5 26/01/2009 16:26:00   Chol    4.9
## 2     5 26/01/2009 16:26:00   TG      2.1
## 3     5 26/01/2009 16:26:00   HDL     1.1
## 4     5 26/01/2009 16:26:00   LDL     2.5
## 5     5 26/01/2009 16:26:00   Cr     56
```

The phrase **pivot wider** means taking rows/observations and rotating them *outwards* into columns/variables. This can be done using `pivot_wider` from the `tidyr` package.

Convert from long to wide format:

```
#pivot around id, SampleDate and SampleTime
wide_data <- long_data %>% tidyr::pivot_wider(
    # new column name for old headers
    names_from = "Test",
    # values in this column fill the cells
    values_from = "Value"
)

head(wide_data, n=5)
```

```
## # A tibble: 5 x 9
##      id SampleDate SampleTime  Chol    TG   HDL   LDL    Cr   BUN
##   <int> <chr>      <chr>      <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1     5 26/01/2009 16:26:00    4.9   2.1   1.1   2.5   56    5.1
## 2     5 19/10/2012 10:22:00    3.6   5.1   0.9   2.5   63    5
## 3     5 19/10/2012 09:31:00    4.85  1.63  4.86  4.86  96.7  5.24
## 4    14 13/07/2009 10:23:00    6.49  1.48  1.37  4.17  41.1  3.46
## 5    14 07/10/2010 11:05:00   NA     0.92  4.86  4.86  60    4.72
```

The use of a long versus a wide data format depends on the specific analysis or operation required. Typically, most analyses require wide-format data; however, in cases where data have repeated measures, long-format data may be more suitable, as many mixed models require this format. In addition, `ggplot2` often requires that data are summarised in a long format. More about that will be covered in Chapter 4.

3.6 Data Transformation

Very rarely will a data set be presented as analysis-ready. Rather, the typical situation involves importing flat files that store records in a single table, followed by formatting (e.g., converting character strings of dates into Dates, nominal data into factors, and making sure that ordinal data are ordered correctly).

3.6.1 Summaries

Summaries are a way to condense many values into fewer, essentially creating a concise overview of the data provided, hence the name. In general, this is done using the `summary` function, which has been used throughout previous sections. One of the benefits of working within an object-oriented programming environment is that `summary` can be applied to any object. However, each will have a different summary.

Try the following codes and see how R shows different output depending on the data type. Note that the iris dataset will be extensively used in Chapter 4 and the code used in Example 4 will be taught in Chapter 5. 1. Whole dataset: `base::summary(demog)` 2. A series: `base::summary(1:5)` 3. A variable within a dataset: `base::summary(demog$BMI)` 4. A function call: `base::summary(stats::lm(BMI ~ SIMD, data=demog))`

Ultimately, the base R version of creating summaries is more ad hoc. In contrast, `dplyr` provides a `summarise` function, which is often integrated into pipelines by providing summary statistics that can be used in figures or tables downstream.

Examples of dplyr's summarise function using the diabeticDemog dataset:

```
library(dplyr)
demog <- utils::read.csv("data/03-diabetic_demog.csv")

demog %>%
    # define the minimum, mean, and maximum age
    dplyr::summarise(
                mean_age = mean(age, na.rm=TRUE),
                mim_age = min(age, na.rm=TRUE),
                max_age = max(age, na.rm=TRUE)
            )
```

```
##   mean_age mim_age max_age
## 1 20.78173       1      58
```

3.6.2 Mutating

Within R, the term **mutating** is used when new variables are added or existing variables are modified within data frames or tibbles. In short, it is used when an object is changed after its creation.

Using dplyr to add a variable:

```
#define the data frame
df <- data.frame(id = c(5, 14, 16),
        SampleDate = c("26-01-2009", "13-07-2009", "30-05-2010"),
        SampleTime = c("16:26:00", "10:23:00", "16:31:00"),
        Chol = c(4.9, 6.49, 3.3),
        TG = c(2.1, 1.48, 1.7),
```

```
        HDL = c(1.1, 1.37, 1.27),
        LDL = c(2.5, 4.14, 1.93),
        Cr = c(56.0, 41.1, 71),
        BUN = c(5.1, 3.46, 4.95))

library(dplyr)
# Creating a new column
df <- df %>% dplyr::mutate(HDL_mgdL = HDL/38.67)
head(df, 2)
```

```
##   id SampleDate SampleTime Chol   TG  HDL  LDL   Cr  BUN   HDL_mgdL
## 1  5 26-01-2009   16:26:00 4.90 2.10 1.10 2.50 56.0 5.10 0.02844582
## 2 14 13-07-2009   10:23:00 6.49 1.48 1.37 4.14 41.1 3.46 0.03542798
```

```
# Modifying a column
df<- df %>% dplyr::mutate(HDL_mgdL = HDL_mgdL*10)
head(df, 2)
```

```
##   id SampleDate SampleTime Chol   TG  HDL  LDL   Cr  BUN  HDL_mgdL
## 1  5 26-01-2009   16:26:00 4.90 2.10 1.10 2.50 56.0 5.10 0.2844582
## 2 14 13-07-2009   10:23:00 6.49 1.48 1.37 4.14 41.1 3.46 0.3542798
```

In the above code, both the original (**\acr{HDL}**) and new (HDL_mgdL) variables remain in the data frame. The `mutate` function provides the ability to keep or drop variables by specifying the desired functionality using the `.keep` parameter. The default option is `.keep = "all"` where all variables, both used and unused, are kept. The `none` option only keeps the new variable; the `used` option only keeps the new variable and the variables used to create it; and the `unused` option keeps the result and drops the variables used to create the new variable.

Using dplyr to add a variable with `.keep`:

```
# Creating a new column
df <- df %>% dplyr::mutate(HDL_mgdL_new = HDL/38.67, .keep = "unused")
head(df, 2)
```

```
##   id SampleDate SampleTime Chol   TG  LDL   Cr  BUN  HDL_mgdL HDL_mgdL_new
## 1  5 26-01-2009   16:26:00 4.90 2.10 2.50 56.0 5.10 0.2844582   0.02844582
## 2 14 13-07-2009   10:23:00 6.49 1.48 4.14 41.1 3.46 0.3542798   0.03542798
```

Of note, when first developing code, it is best to keep all variables so that the logic and execution of the new variable can be sense checked.

Using the DiabeticLabs_final_withMissing.csv and DiabeticDemog.csv, experiment with creating different variables and changing the `.keep` parameter.

3.6.3 Grouping and Ungrouping

Intuitively, **grouping** is a way of splitting data into smaller chunks based on values of one or more variables for the purpose of applying operations or functions (e.g., finding the mean, minimum, maximum, etc.) to each group of data. In base R, grouping is temporary and tied to the function call by being passed as an argument each time, so the groups only exist inside the function call.

Grouping examples using dplyr:

```
library(dplyr)
lab_data <- utils::read.csv("data/03-diabetic_labs_final_with_missing.csv", head = TRUE)

# Example using dplyr::group_by()
lab_data %>%
    dplyr::group_by(id) %>%
    dplyr::summarise(mean_chol = mean(Chol)) %>%
    head(3)
```

```
## # A tibble: 3 x 2
##      id mean_chol
##   <int>     <dbl>
## 1     5      4.45
## 2    14     NA
## 3    16      4.34
```

In the above example, `dplyr::group_by(id)` attaches the grouping to the data and `summarise` works inside these groups, `id` in this case, and provides the mean cholesterol value per `id`. The functionality provided through `dplyr` shines when stacking different data transformations.

Showing off dplyr R:

```
lab_data %>%
    # group by ID
    dplyr::group_by(id) %>%
    # remove the NA values during summarising
    dplyr::summarise(mean_chol = mean(Chol, na.rm = TRUE)) %>%
    # ungroup
    dplyr::ungroup() %>%
    # find grand mean
    mutate(grand_mean = mean(mean_chol)) %>%
    head(3)
```

```
## # A tibble: 3 x 3
##      id mean_chol grand_mean
##   <int>     <dbl>      <dbl>
## 1     5      4.45       4.96
## 2    14      5.64       4.96
## 3    16      4.34       4.96
```

Using the DiabeticLabs_final_withMissing.csv and DiabeticDemog.csv, experiment with building small pipelines by combining grouping and mutating functions.

3.7 Logic Flow

This section introduces the concepts and functions needed to build an analysis pipeline from a single flat file using the following steps: * **Select** - Decide the variables that matter * **Filter** - Narrow down the observations * **Transform** - Transform/add new variables based on conditions * **Repeat** (if needed) - Develop re-usable code when an action needs to be done over and over.

3.7.1 Selecting

Think of **selecting** as choosing which variables should be carried forward or removing variables that have served their purpose and now take up unnecessary space or may cause confusion.

3.7.1.1 1D Data Structures

When working with a one-dimensional data structure, such as a vector, there are a few ways to access data. The basic way to access data is to use square brackets `[]` with a single index, an index range, or a vector of indexes as the parameter.

Using base R to access data from a vector:

```
# Define the vector
data <- c(4.9, 2.1, 1.1, 2.5, 56.0, 5.1)

# Access a single value
data[2]
```

```
## [1] 2.1
```

```
# Access a range, data indexes 2 through 4, inclusive
data[2:4]
```

```
## [1] 2.1 1.1 2.5
```

```
# Access a vector of indexes
data[c(1, 3, 4)]
```

```
## [1] 4.9 1.1 2.5
```

3.7.1.2 2D Data Structures

id	SampleDate	SampleTime	Chol	TG	HDL	LDL	Cr	BUN
5	26/01/2009	16:26:00	4.9	2.1	1.1	2.5	56	5.1
14	13/07/2009	10:23:00	6.49	1.48	1.37	4.17	41.1	3.46
16	30/05/2010	16:31:00	3.3	1.7	1.27	1.93	71	4.95

lab_data

Figure 3.6: An illustration of a data frame containing patient haematology and biochemistry values.

Matrices, **data frames**, and **tibbles** are stored in row, column notation, indexed from 1. So, `df[i,j]` will return the `ith` row and the `jth` column. For data structures with named columns, `$` can be used to access the named variable.

Exploring dplyr's `select` function:

```
library(dplyr)
lab_data <- data.frame(id = c(5, 14, 16),
        SampleDate = c("26-01-2009", "13-07-2009", "30-05-2010"),
        SampleTime = c("16:26:00", "10:23:00", "16:31:00"),
        Chol = c(4.9, 6.49, 3.3),
        TG = c(2.1, 1.48, 1.7),
        HDL = c(1.1, 1.37, 1.27),
        LDL = c(2.5, 4.14, 1.93),
        Cr = c(56.0, 41.1, 71),
        BUN = c(5.1, 3.46, 4.95))

# Select a range of columns
lab_data %>% dplyr::select(HDL:Cr)
```

```
##    HDL  LDL   Cr
## 1 1.10 2.50 56.0
## 2 1.37 4.14 41.1
## 3 1.27 1.93 71.0
```

```
# Removing variables with "-"
lab_data %>% dplyr::select(-SampleTime)
```

```
##   id SampleDate Chol   TG  HDL  LDL   Cr  BUN
## 1  5 26-01-2009 4.90 2.10 1.10 2.50 56.0 5.10
## 2 14 13-07-2009 6.49 1.48 1.37 4.14 41.1 3.46
## 3 16 30-05-2010 3.30 1.70 1.27 1.93 71.0 4.95
```

3.7.2 Filtering

In contrast to selecting, **filtering** deals with observations. Within `dplyr`, this is done with the `filter` function, where the filtering conditions are passed in as parameters.

Filtering examples using dplyr's `filter` function:

```
library(dplyr)

# Select all observations where TG > 1.5
lab_data %>% dplyr::filter(TG > 1.5)
```

```
##   id SampleDate SampleTime Chol  TG  HDL  LDL Cr  BUN
## 1  5 26-01-2009   16:26:00  4.9 2.1 1.10 2.50 56 5.10
## 2 16 30-05-2010   16:31:00  3.3 1.7 1.27 1.93 71 4.95
```

```
# Or combine to filter on multiple conditions
lab_data %>% dplyr::filter((TG > 1.5) & (HDL < 1.2))
```

```
##   id SampleDate SampleTime Chol  TG HDL LDL Cr BUN
## 1  5 26-01-2009   16:26:00  4.9 2.1 1.1 2.5 56 5.1
```

3.7.3 If/Else

Within R, and many other coding languages, an **if statement** is a block of code that will only execute if the condition inside the parentheses is `TRUE`. Often, there will be an associated **else** code block that will execute only if the if statement was false.

3.7.3.1 Element-Wise Comparisons

Element-wise comparisons work across vectors, touching each element within the vector. The base R version of `ifelse` takes the format of `base::ifelse(test, yes, no)`.

Examples of using `ifelse`:

```
a <- c(1, 2, 3)

# Example 1: Replace even numbers with TRUE and odds with FALSE
base::ifelse(a %% 2 == 0, TRUE, FALSE)
```

```
## [1] FALSE  TRUE FALSE
```

```
#Example 3: NA values
a <- c(1, 2, NA)
base::ifelse(a %% 2 == 0, TRUE, FALSE)
```

```
## [1] FALSE  TRUE    NA
```

In the tidyverse world, this type of functionality has been implemented using the `if_else` function within the `dplyr` package. It has a slightly more explicit formulation about how missing values are handled.

Example using dplyr's `if_else`:

```
a <- c(1, 2, NA)
(dplyr::if_else(a %% 2 == 0, TRUE, FALSE, missing = NULL))
```

```
## [1] FALSE  TRUE    NA
```

```
# Instead, change all missing values to FALSE
(dplyr::if_else(a %% 2 == 0, TRUE, FALSE, missing = FALSE))
```

```
## [1] FALSE  TRUE FALSE
```

3.7.3.2 Logic Flow

Logic flow if/else statements are often used to set an internal parameter within a function based on a passed parameter. The `if` function tests to see if the provided conditional evaluates to `TRUE`. If it does, the code within the body of the if statement defined by `{}` will be executed in sequence; otherwise, the code within the `else` block, if it exists, will be executed.

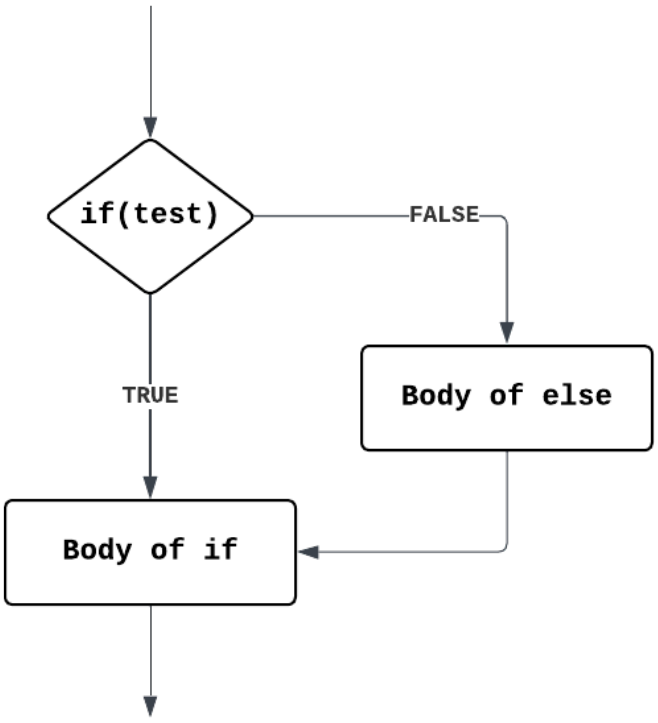


Figure 3.7: A flowchart of how an if/else statement is executed

3.7.4 Multiple Comparisons

If/else is a great option when there is one comparison and two possible options. When there are multiple exclusive comparisons to be made, the `case_when` function within the `dplyr` package exists for when `if_else` is too simple.

Multiple comparisons within one `case_when` with an informative default:

```
demog <- utils::read.csv("data/03-diabetic_demog.csv", head = TRUE)

# Now using case_when()
demog <- demog %>% dplyr::mutate(
    bmi_obesity_level = dplyr::case_when(
                            is.na(BMI) ~ NA, #added check for NA values
                            BMI < 18.5 ~ "underweight",
                            BMI < 25 ~ "healthy weight",
                            BMI < 30 ~ "overweight",
                            BMI < 40 ~ "obese",
                            BMI >= 40 ~ "morbidly obese", # added case
                            .default = "logic error" #informative logic check
                        ) #close case_when()
                    ) #close mutate()

# Turn into a factor
demog$bmi_obesity_level <- base::as.factor(demog$bmi_obesity_level)

# Get a summary and look for any error values
```

```
base::summary(demog$bmi_obesity_level)
```

```
## healthy weight morbidly obese          obese     overweight    underweight
##              19             23             96             53              6
```

3.7.5 Loops

R uses loops to repeat tasks, which means that the code can be written and tested once but repeated numerous times. There are two main classifications of loops within R, the `for loop` and the `while loop`.

3.7.5.1 For loops

The `for loop` is best used when a chunk of code should be run for a known number of times.

A simple implementation:

```
for (i in 1:4){
    base::print(i)
}
```

```
## [1] 1
## [1] 2
## [1] 3
## [1] 4
```

3.7.5.2 While loops

`For loops` execute a chunk of code for a set and pre-defined number of times, while `while loops` execute as long as the test condition iterates to `TRUE`, i.e., the exact number of iterations is unknown.

A simple while loop controlled by a counting functionality:

```
# define the incremental value
n = 1
while(n < 3){
    base::print(n)
    # Increment n
    n = n + 1
}
```

```
## [1] 1
## [1] 2
```

3.8 Merging datasets

Very rarely will datasets be presented as 'analysis-ready'. Data are often provided in separate tables and need to be combined into one, final, 'analysis-ready' table. This is done by **merging** tables using a variety of `joins`.

3.8.1 The Mechanics

Joins work by connecting tables through a pair of **keys**, or a specified variable within each table that identifies an observation within a table. There are two types of keys: `primary` and `foreign keys`.

3.8.1.1 Checking Primary Keys

There are two things to worry about regarding primary keys: (1) they are unique, and (2) they are not missing.

Check to see if the primary keys are unique:

```
# Demographics
demog <- utils::read.csv("data/03-diabetic_demog.csv")

## count the number of times the primary key is seen
demog %>% dplyr::count(id) %>%
## filter for counts more than 1
    dplyr::filter(n > 1)
```

```
## [1] id n
## <0 rows> (or 0-length row.names)
```

```
# Lab data
lab_data <- utils::read.csv("data/03-diabetic_labs_final_with_missing.csv")

## count the number of times the primary key is seen
utils::head(lab_data %>% dplyr::count(id) %>%
## filter for counts more than 1
    dplyr::filter(n > 1), n = 4L)
```

```
##   id n
## 1  5 3
## 2 14 3
## 3 16 3
## 4 25 2
```

The zero returned rows indicate that diabeticDemog has no repeated keys. In contrast, the numerous rows from diabeticLabs indicate that `id` is not enough to identify a row uniquely, so joining these two tables using only the `id` column produces a **many-to-one** join.

3.8.1.2 Introduction to Joining

There are two ways to join two tables. The first is to pass the table using a pipe to pass one table (the **left** table) to one of the join functions, where the second table is passed in as the first argument (this table is referred to as the **right** table).

Example of an inner join:

```
demog <- utils::read.csv("data/03-diabetic_demog.csv")
lab_data <- utils::read.csv("data/03-diabetic_labs_final_with_missing.csv")

# method 1
```

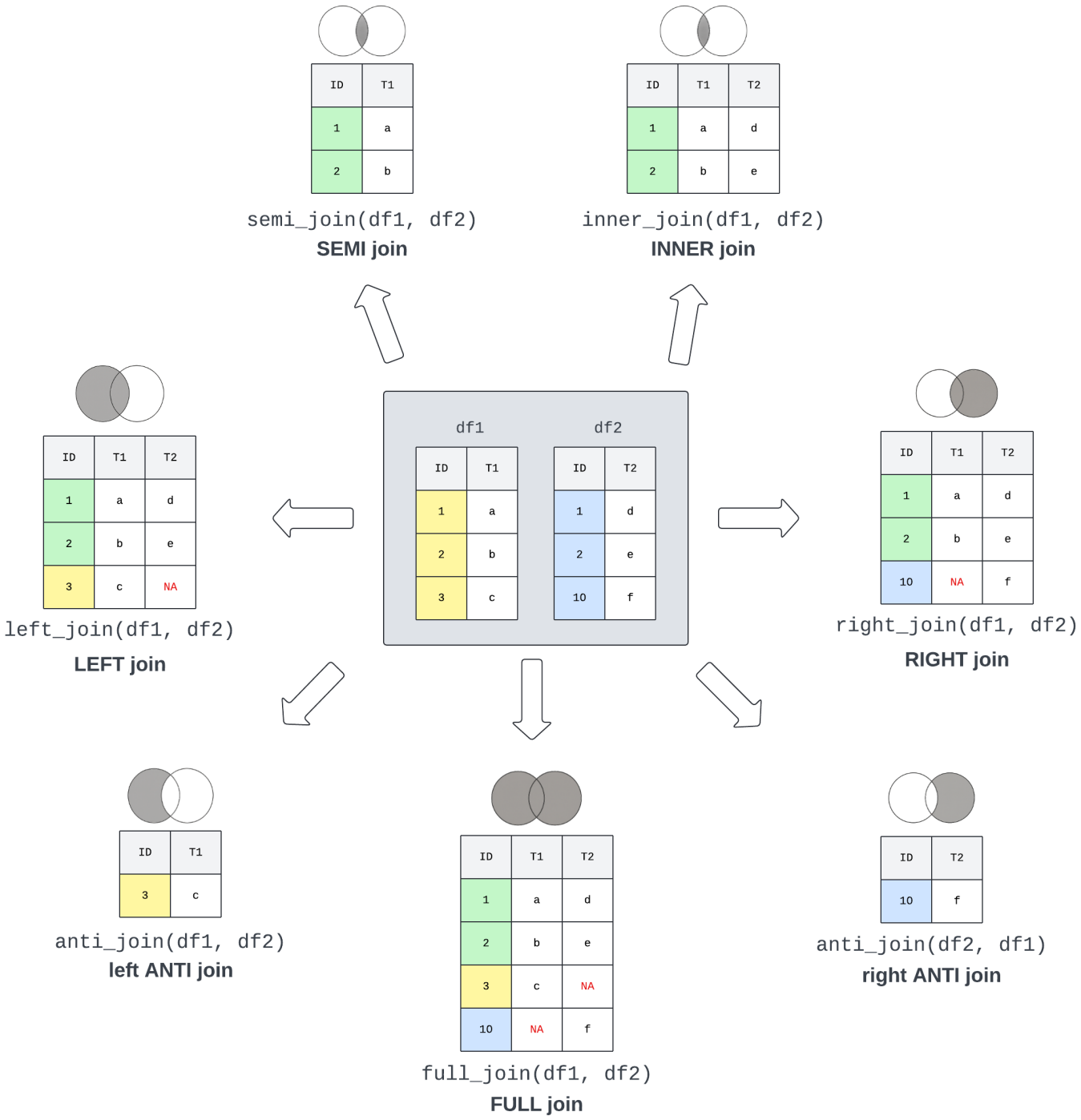


Figure 3.8: An illustration of the most common joins used to combine information from two data frames used within dplyr, including code fragments. The Venn diagrams illustrate how the two data frames are integrated, while the tables illustrate the resulting data frames.

```
joined1 <- demog %>% dplyr::inner_join(lab_data, by="id")
utils::head(joined1, n = 3L)
```

```
##   id age Gender BMI SIMD SampleDate SampleTime Chol   TG      HDL      LDL   Cr
## 1  5  28      M  26    4 26/01/2009   16:26:00 4.90 2.10 1.100000 2.500000 56.0
## 2  5  28      M  26    4 19/10/2012   10:22:00 3.60 5.10 0.900000 2.500000 63.0
## 3  5  28      M  26    4 19/10/2012   09:31:00 4.85 1.63 4.860753 4.860753 96.7
##    BUN
## 1 5.10
## 2 5.00
## 3 5.24
```

```
# method 2
joined2 <- dplyr::inner_join(demog, lab_data, by = "id")
utils::head(joined2, n = 3L)
```

```
##   id age Gender BMI SIMD SampleDate SampleTime Chol   TG      HDL      LDL   Cr
## 1  5  28      M  26    4 26/01/2009   16:26:00 4.90 2.10 1.100000 2.500000 56.0
## 2  5  28      M  26    4 19/10/2012   10:22:00 3.60 5.10 0.900000 2.500000 63.0
## 3  5  28      M  26    4 19/10/2012   09:31:00 4.85 1.63 4.860753 4.860753 96.7
##    BUN
## 1 5.10
## 2 5.00
## 3 5.24
```

3.9 Cautionary Tale

3.9.1 Check for Uniqueness

Often, in the course of health data science, statistical analysis plans may request the first or lowest value. In the case of the first or last test, what happens if there is more than one test on the same day? For this reason, add additional columns to sort by so that ties are broken and unique values are therefore selected.

Now add an additional value to break the ties

```
library(tidyverse)
library(hms)
lab_data <- utils::read.csv("data/03-diabetic_labs_final_with_missing.csv")

# Convert dates and times
lab_data$SampleDate <- lubridate::as_date(
        lab_data$SampleDate, format = "%d/%m/%Y")
lab_data$SampleTime <- hms::as_hms(lubridate::as_datetime(
  lab_data$SampleTime, format = "%H:%M:%S"))

# collect the last test value and print the first 10 rows
utils::head(lab_data %>%
    # sort by id, date, and time in descending order
```

```
    dplyr::arrange(id, dplyr::desc(SampleDate), dplyr::desc(SampleTime)) %>%
    # group by id and SampleDate to break ties
    dplyr::group_by(id, SampleDate) %>%
    # add a rank to see ties
    dplyr::mutate(rank = dplyr::min_rank(dplyr::desc(SampleTime))),
    n = 10L)
```

```
## # A tibble: 10 x 10
## # Groups:   id, SampleDate [9]
##       id SampleDate SampleTime  Chol    TG   HDL   LDL    Cr   BUN  rank
##    <int> <date>     <time>     <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <int>
##  1     5 2012-10-19 10:22       3.6   5.1   0.9   2.5   63    5         1
##  2     5 2012-10-19 09:31       4.85  1.63  4.86  4.86  96.7  5.24      2
##  3     5 2009-01-26 16:26       4.9   2.1   1.1   2.5   56    5.1       1
##  4    14 2011-02-22 13:11       4.8   1     1.72  2.57  73    4.67      1
##  5    14 2010-10-07 11:05      NA     0.92  4.86  4.86  60    4.72      1
##  6    14 2009-07-13 10:23       6.49  1.48  1.37  4.17  41.1  3.46      1
##  7    16 2012-06-24 12:34       5.4   1.6   1.6   3.1   NA    4.1       1
##  8    16 2012-02-17 10:43       4.31  0.84  4.86  4.86  66    7.23      1
##  9    16 2010-05-30 16:31       3.3   1.7   1.27  1.93  71    4.95      1
## 10    25 2011-10-16 10:41       4.6   1.2   1.3   2.8   49    3.2       1
```

3.9.2 Cartesian product joins

Below is a toy example to illustrate how quickly **many-to-many** joins can produce undesired results due to the Cartesian product.

Cartesian product due to many-to-many matches:

```
df1 <- dplyr::tibble(
        id = c(1, 1, 2),
        value1 = c("a", "b", "c")
        )
df2 <- dplyr::tibble(
        id = c(1, 2, 2),
        value2 = c("x", "y", "z")
        )

# Depending on dplyr version, this may throw a warning
df1 %>% dplyr::inner_join(df2, by = "id")
```

```
## Warning in dplyr::inner_join(., df2, by = "id"): Detected an unexpected many-to-many relationship between
## i Row 3 of ‘x‘ matches multiple rows in ‘y‘.
## i Row 1 of ‘y‘ matches multiple rows in ‘x‘.
## i If a many-to-many relationship is expected, set ‘relationship =
##   "many-to-many"‘ to silence this warning.
```

```
## # A tibble: 4 x 3
##      id value1 value2
```

```
##   <dbl> <chr>  <chr>
## 1     1 a      x
## 2     1 b      x
## 3     2 c      y
## 4     2 c      z
```

Chapter 4

Data Visualisation

Before starting this chapter, make sure that you have the following R packages installed and loaded.

```
#Install packages
utils::install.packages("ggplot2") #also installed via tidyverse
utils::install.packages("patchwork")
utils::install.packages("dplyr") #also installed via tidyverse
utils::install.packages("viridis")
```

Remember you only need to install package once in a computer. And after they're installed you can load them into the current R session:

```
#Load packages
base::library(ggplot2)
base::library(patchwork)
base::library(dplyr)
base::library(viridis)
```

```
## Loading required package: viridisLite
```

```
rm(list=ls()); gc()
```

```
##           used  (Mb) gc trigger  (Mb) max used  (Mb)
## Ncells 2418252 129.2    4157530 222.1  4157530 222.1
## Vcells 4311053  32.9   10146329  77.5  8388608  64.0
```

4.1 Basic plotting

There are two main methods for creating plots in R. We will be using the package ggplot2, which uses the grammar of graphics (a framework for building plots from components such as data, aesthetics, and facets), and we will be using some built-in data sets to start plotting.

The data set we are using for this walk-through is called iris and documents the sepal and petal length and width of three different species of iris. It is one of a number of built-in datasets that come with every installation of R, and it looks like this:

```
utils::data(iris)
utils::head(iris)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1          5.1         3.5          1.4         0.2  setosa
## 2          4.9         3.0          1.4         0.2  setosa
## 3          4.7         3.2          1.3         0.2  setosa
## 4          4.6         3.1          1.5         0.2  setosa
## 5          5.0         3.6          1.4         0.2  setosa
## 6          5.4         3.9          1.7         0.4  setosa
```

Explore the data: 1. What is the range of Sepal.Length? 2. What is the mean of Sepal.Width? 3. What are the names of the three different iris species?

There are many options for how we can plot parts of this data frame, but our first example will be a scatter plot comparing sepal length and sepal width. Each plot should be considered as being made up of a number of layers, so we will build the plot layer by layer.

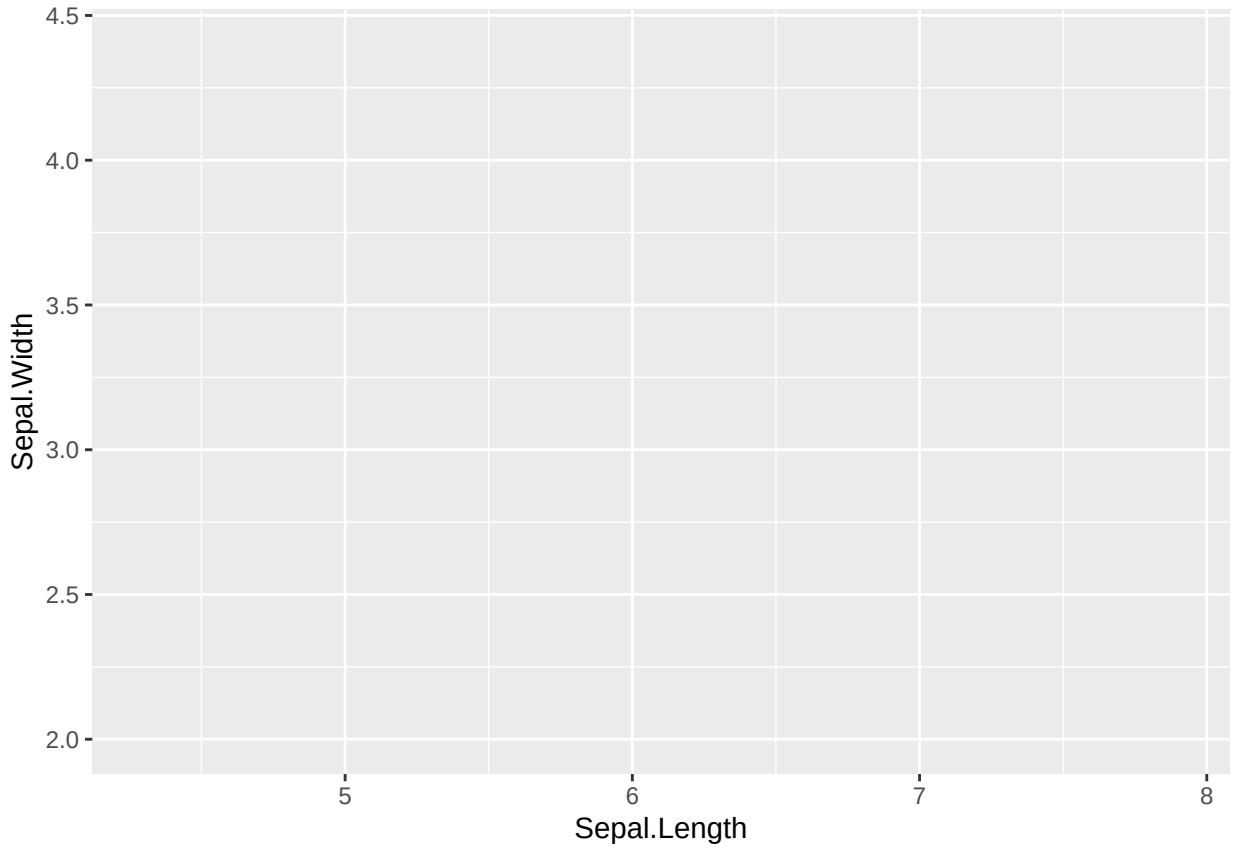
4.1.1 Data layer

The first line of any plot using ggplot2 starts with a call to ggplot. This can either be stand-alone as ggplot or we can define some of the key inputs for the plot. With a few exceptions, it is useful to include the dataset within this call to ggplot (either `ggplot(dataname)` or `ggplot(data = dataname)`). Beyond that, we can also specify some of the aesthetics of the graph - for example, what data is plotted on the x and y axes, and whether there is a grouping that we want to distinguish within the plot by colour, shape, or shading.

We are using the iris dataset and will plot the sepal length against the sepal width in this example, so our code line to set up the base canvas for the plot uses `data = iris` and `aes(x = Sepal.Length, y = Sepal.Width)`. Both versions of the code below produce the same plot:

Code example 1:

```
ggplot2::ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width))
```



Code example 2:

```
ggplot2::ggplot(data = iris, aes(x = Sepal.Length, y = Sepal.Width))
```

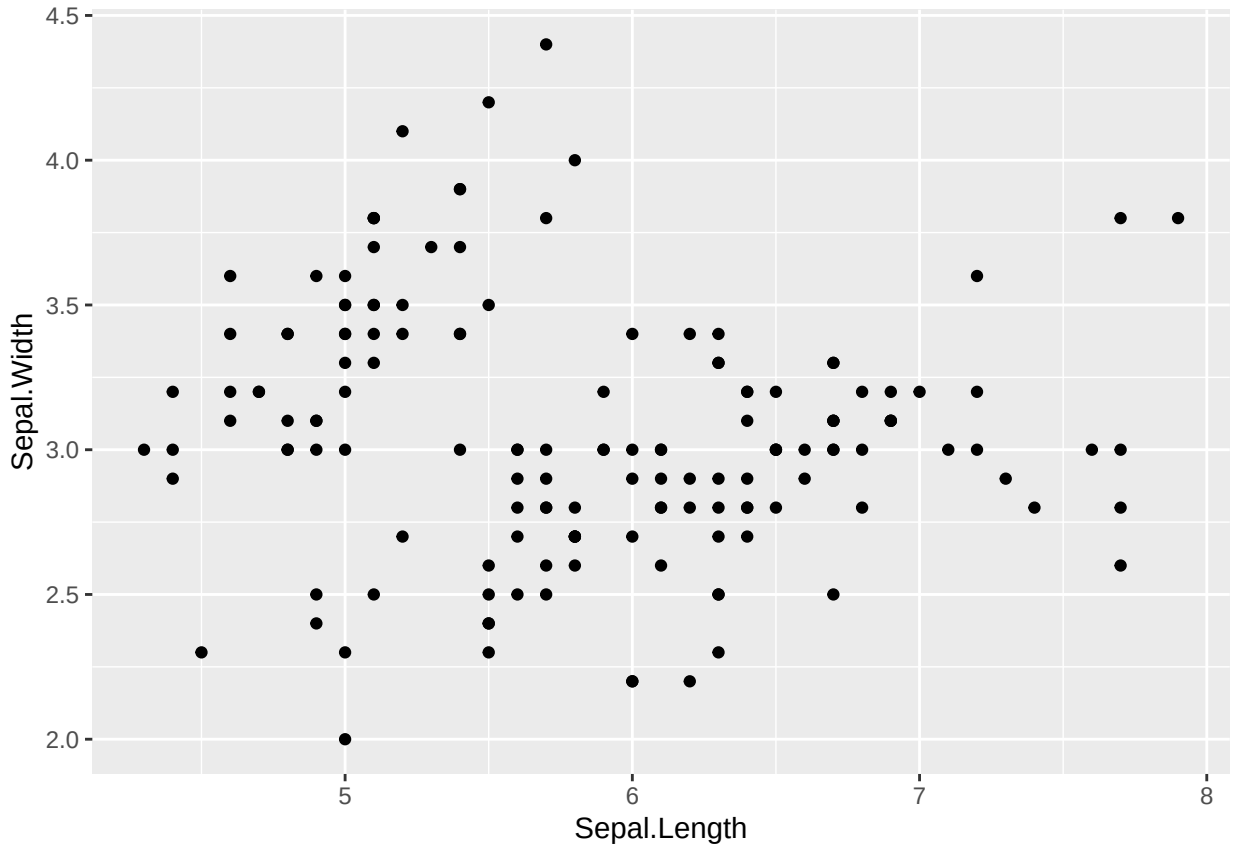
Try it yourself: 1. What happens if you remove one of the x or y values in the `aes` argument? 2. What happens if you remove the `aes` function call entirely? (don't forget to remove the comma after `iris`) 3. What happens when you just call ggplot?

It would be possible to add further aesthetics here, but we will cover those further down.

4.1.2 Plot layer(s)

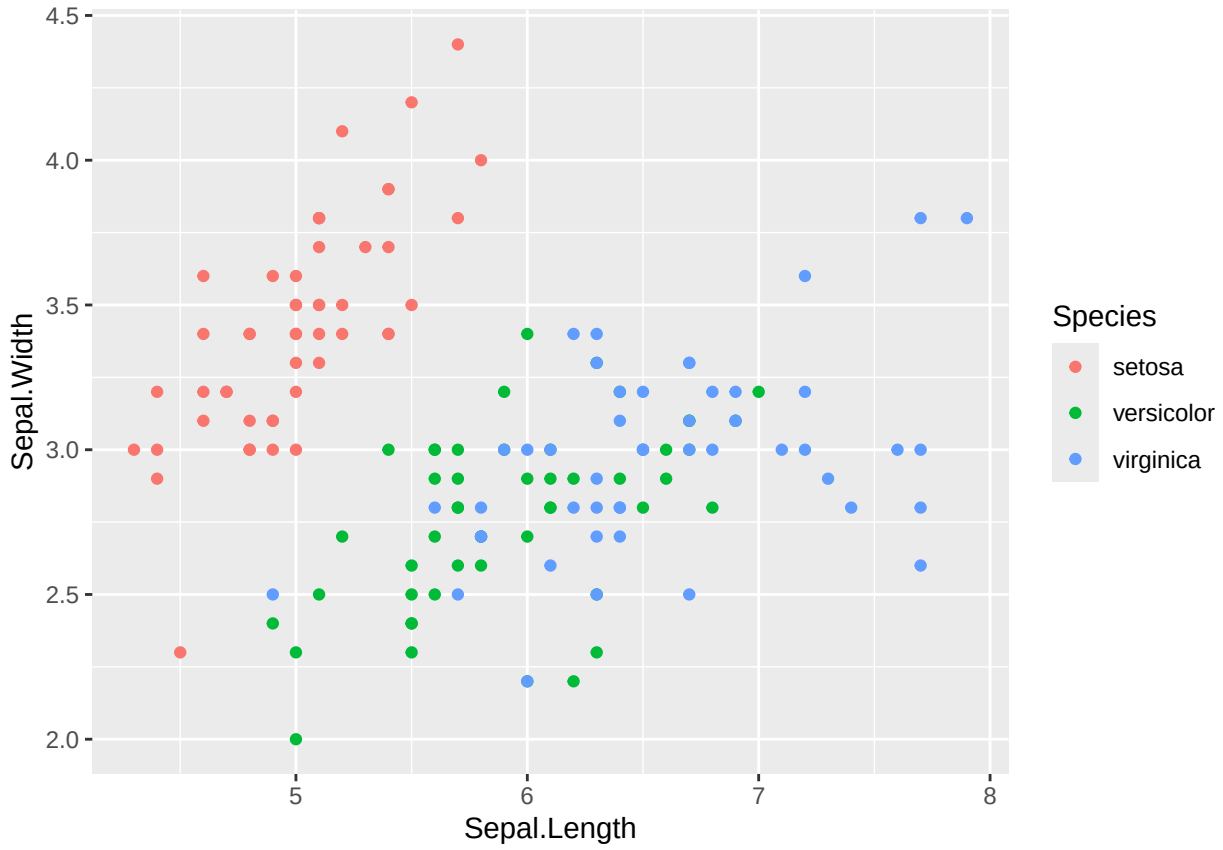
Now we have a canvas for our plot, but there are no data on it. We use **geoms** to represent the data points. We will demonstrate more examples of these later, but the geom function required to make a scatter plot is `geom_point()`. To use this, we need to have declared an x-value and a y-value as an aesthetic, as we did in Step 1. Then we simply add the function to our base canvas:

```
ggplot2::ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width)) +
  geom_point()
```



We can see that there are two groupings in this plot. Since the dataset contains three species of iris, it would be helpful to determine if these groupings are between some of the species or if it's a coincidence. A nice way to do that is by colouring the points according to the species. This would be another aesthetic variable which can either be called in the first call to ggplot or as an argument passed to `aes` within `geom_point()`, we have used the latter below:

```
ggplot2::ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width)) +
  geom_point(aes(colour = Species))
```



NOTE: The difference between aesthetics and other ways of formatting

Above, we have defined a secondary aesthetic, specific to `geom_point()` that says "make the colours of the points be defined by their species". This aesthetic is defined as such because it looks into the dataset for the solution. We could, alternatively, say that we wanted all points to be orange. Orange is not a specific attribute of the dataset, and therefore, we don't specify it as an aesthetic; instead, it is included within the main `geom_point()` function call.

```
ggplot2::ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width)) +
  geom_point(colour = "orange")
```

We can see that all points have turned orange, regardless of the underlying species data in this case.

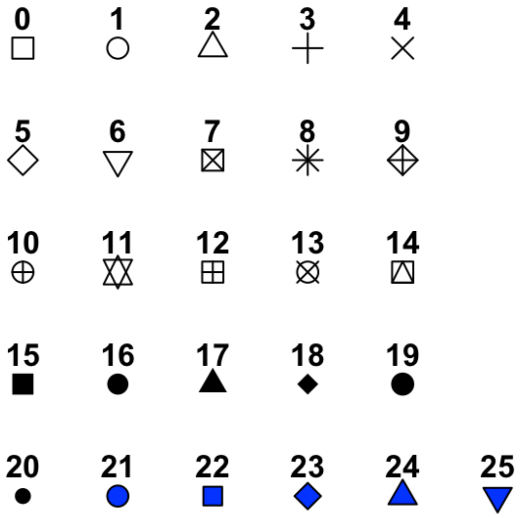
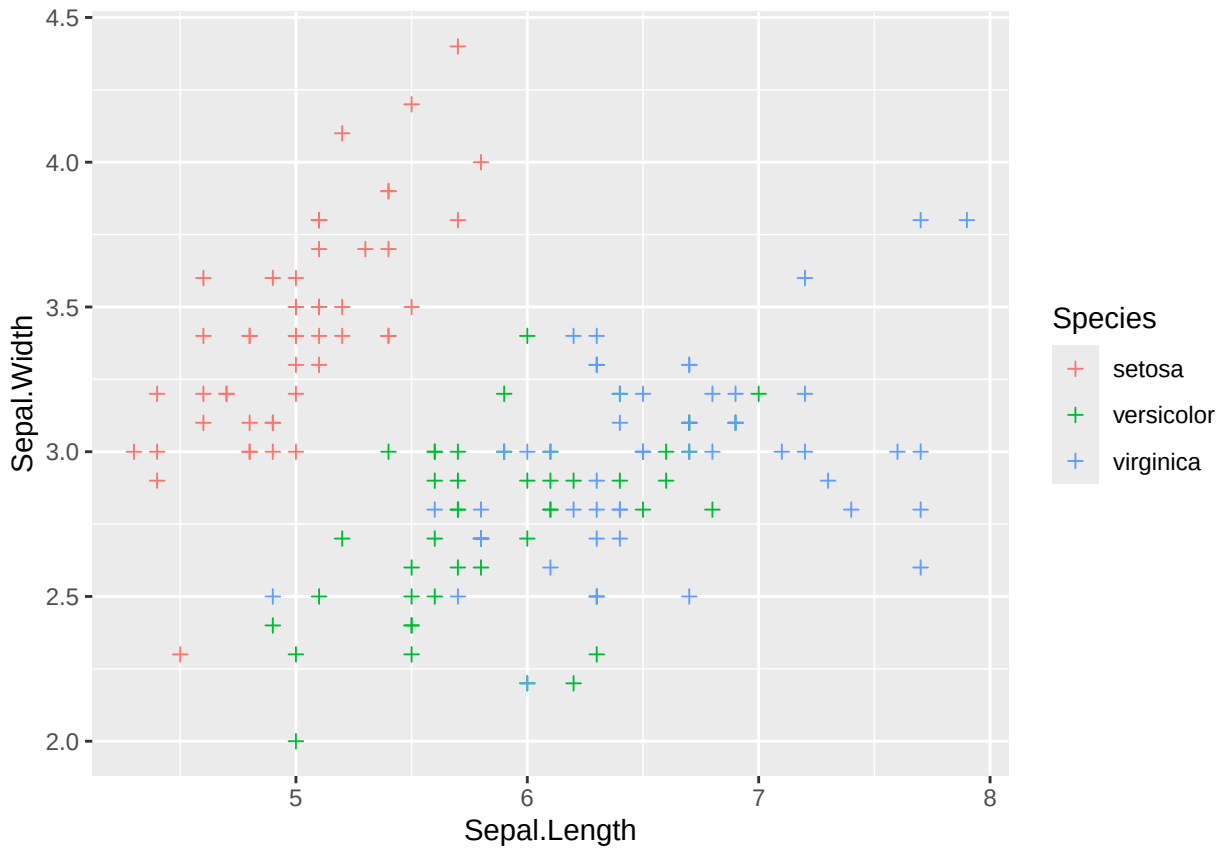


Figure 4.1: An illustration of the different point shapes commonly used in R. The default is number 19, but this can be changed using the 'shape = x' parameter, where x is the shape number.

We could decide that we do not want filled circles as our points. There are 25 different shapes (see Figure 4.1). The default is number 19, which is a single-coloured dot, but we could choose any of the shapes - for example, shape number 3 is a plus sign.

```
ggplot2::ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width)) +
  geom_point(aes(colour = Species), shape = 3)
```



It is also possible to make the shape change depending on data within the dataset as an aesthetic or to use any single letter or character from your keyboard instead of one of the preset shapes (try these as an exercise). Finally, the size of each point is not fixed and can be changed in a similar way. As a default, the points are size 1.5, but it is possible to change that number to any positive number. Below, we show an example where we have increased the point size to 3.

```
ggplot2::ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width)) +
  geom_point(aes(colour = Species), size = 3)
```

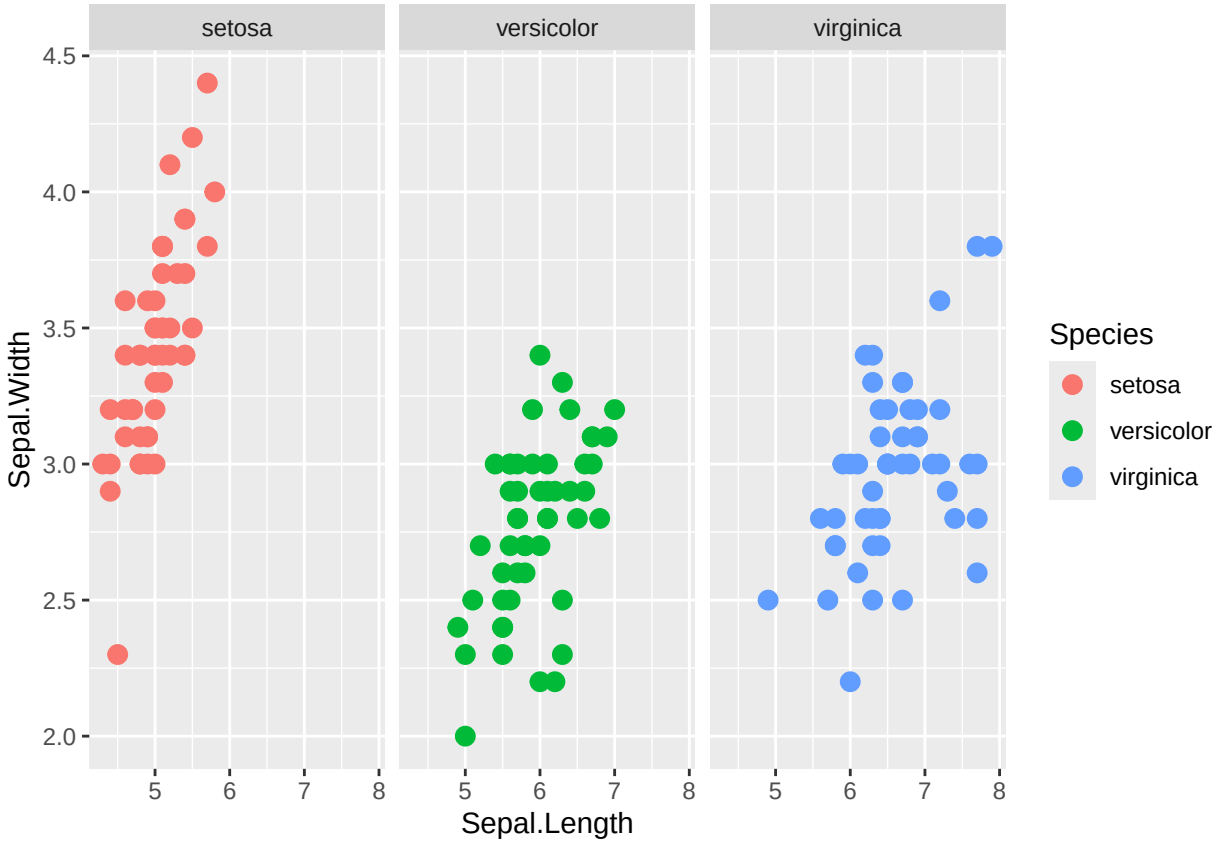
Editing the plot layer: 1. What happens if you make size an aesthetic based on petal width? 2. Shapes 21-25 can take two colours, a "colour" for the edges and a "fill" for the centre. Can you work out how to add your choice of fill? 3. Can you change the shape of the points based on the species of iris? 4. Try to make all the shapes a letter of your choice - remember to include the character within quotation marks.

4.1.3 Facets

Facets split the single graph into separate but linked graphs by a given grouping. For larger datasets, this can be split into a grid by two different groupings, but for our example, we will just split by the different iris species, like we have done by colour. We can either define this split by defining which grouping sets the rows and/or columns, using `cols = vars(group)` / `rows = vars(group)`, or it can be defined with an equation `.~ group` / `group~.` / `group1~ group2`. See Table 3.2 for tilde definition and will be covered further in Section 5.5.1. Examples of both options for the iris dataset are below:

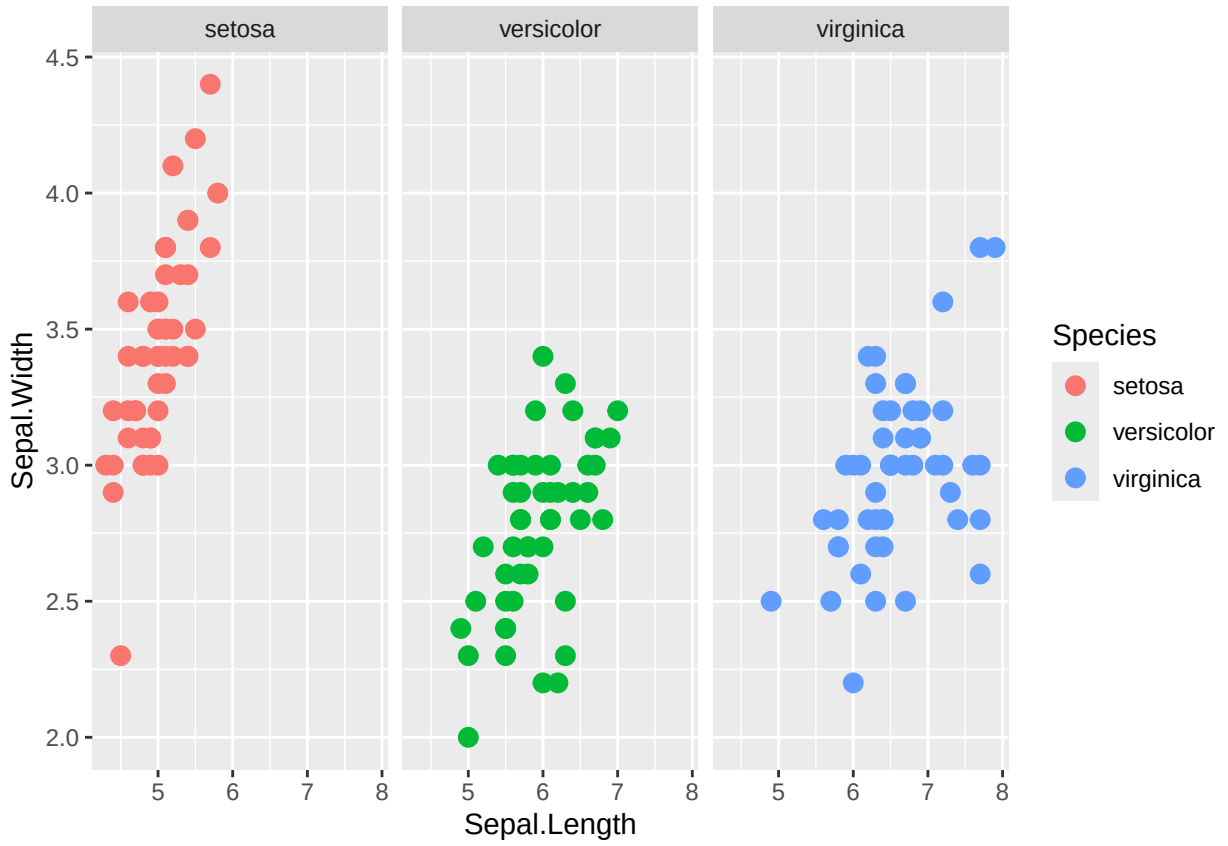
Example 1

```
ggplot2::ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width)) +
  geom_point(aes(colour = Species), size = 3) +
  facet_grid(cols = vars(Species))
```



Example 2

```
ggplot2::ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width)) +
  geom_point(aes(colour = Species), size = 3) +
  facet_grid(.~Species)
```



Facet options: 1. What happens if you set `scales = "free_x"` or `scales = "free_y"` or `scales = "free"`? 2. What happens if you try the same arguments with `space`? 3. Is `cols = vars(Species)` equivalent to `.~Species` or `Species~.`?

4.1.4 Making it all look pretty

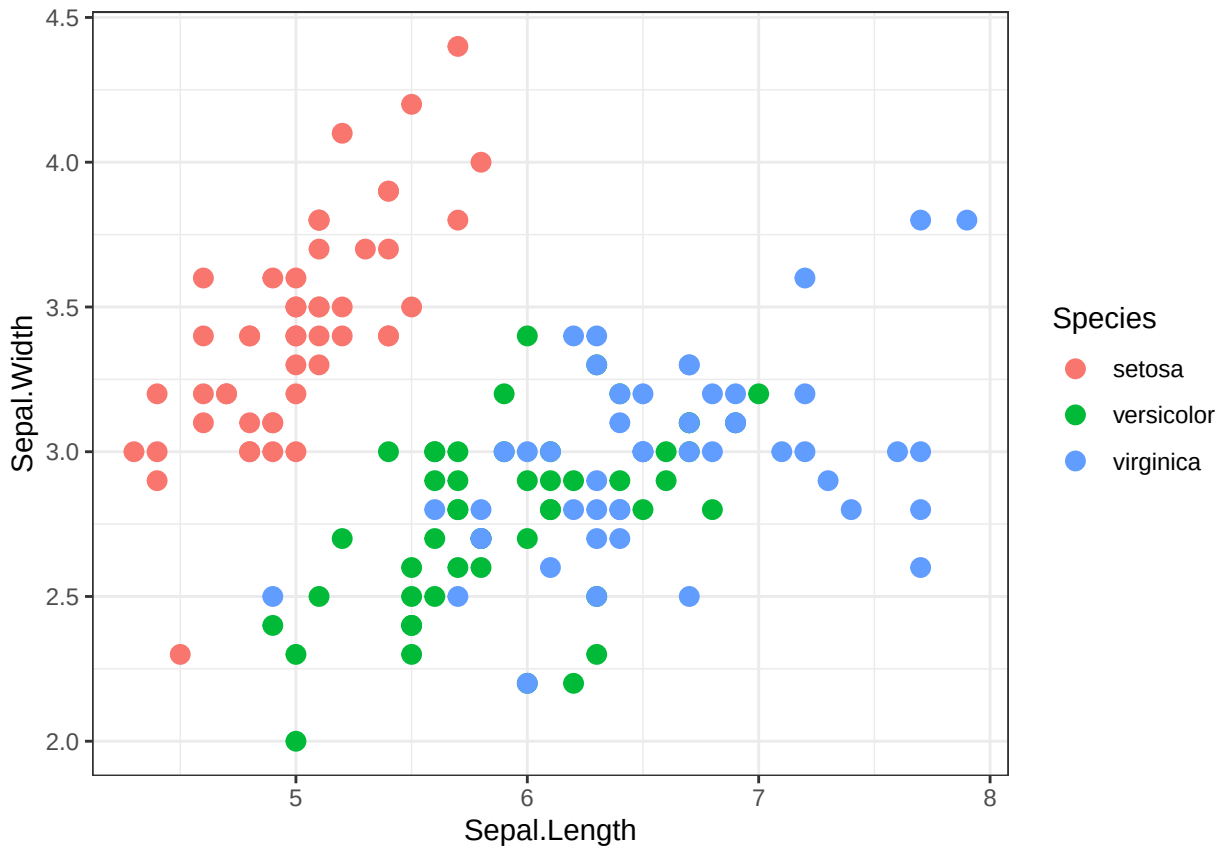
4.1.4.1 Background and theme

All plots up to this point were created using ggplot's default theme. You can change this to alter the plot background. Below, I will give you a few examples of this. You can also click here to explore more of the themes: https://ggplot2.tidyverse.org/reference/ggtheme.html. Below is the first plot, which shows the default theme for ggplot2 graphs.

```
ggplot2::ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width)) +
  geom_point(aes(colour = Species), size = 3)
```

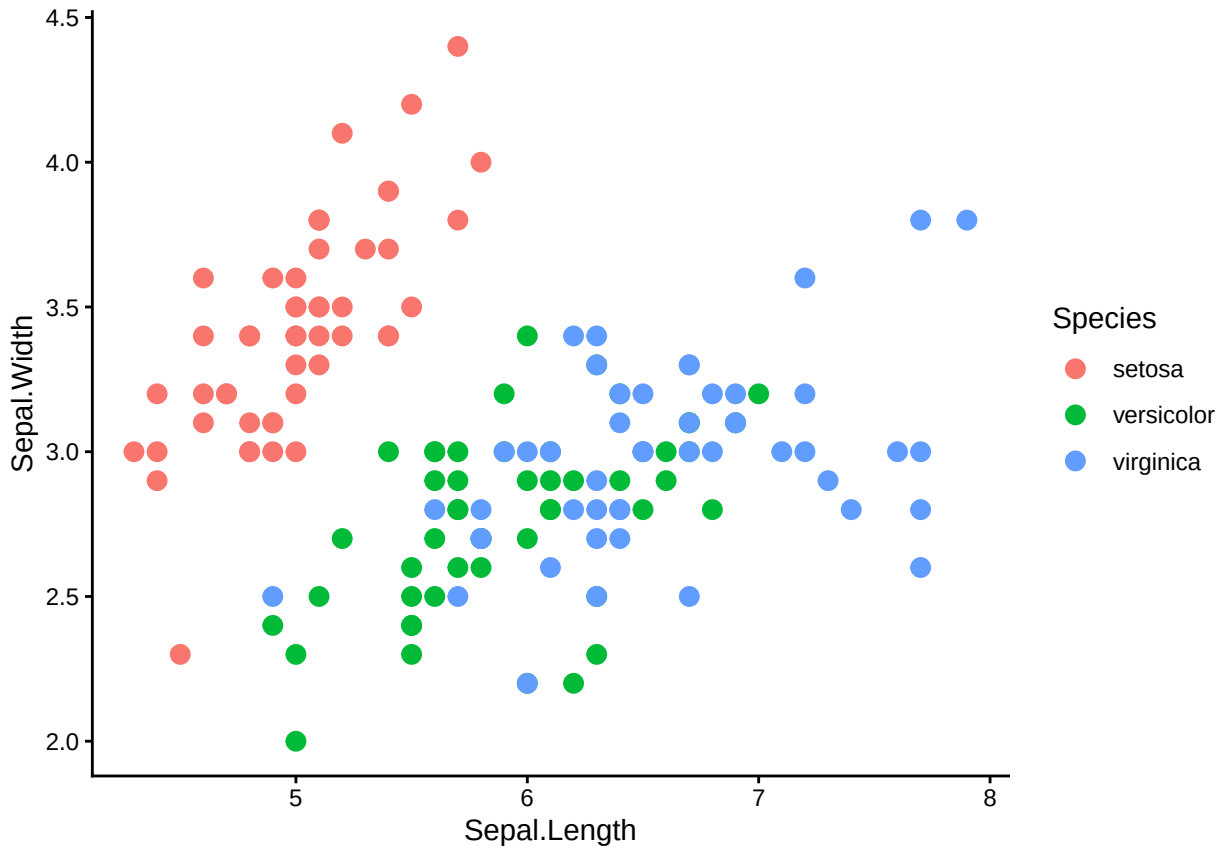
Below is the second plot, which shows the black and white theme for ggplot2 graphs. The plot now has a white background with grid markers.

```
ggplot2::ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width)) +
  geom_point(aes(colour = Species), size = 3)+
  theme_bw()
```



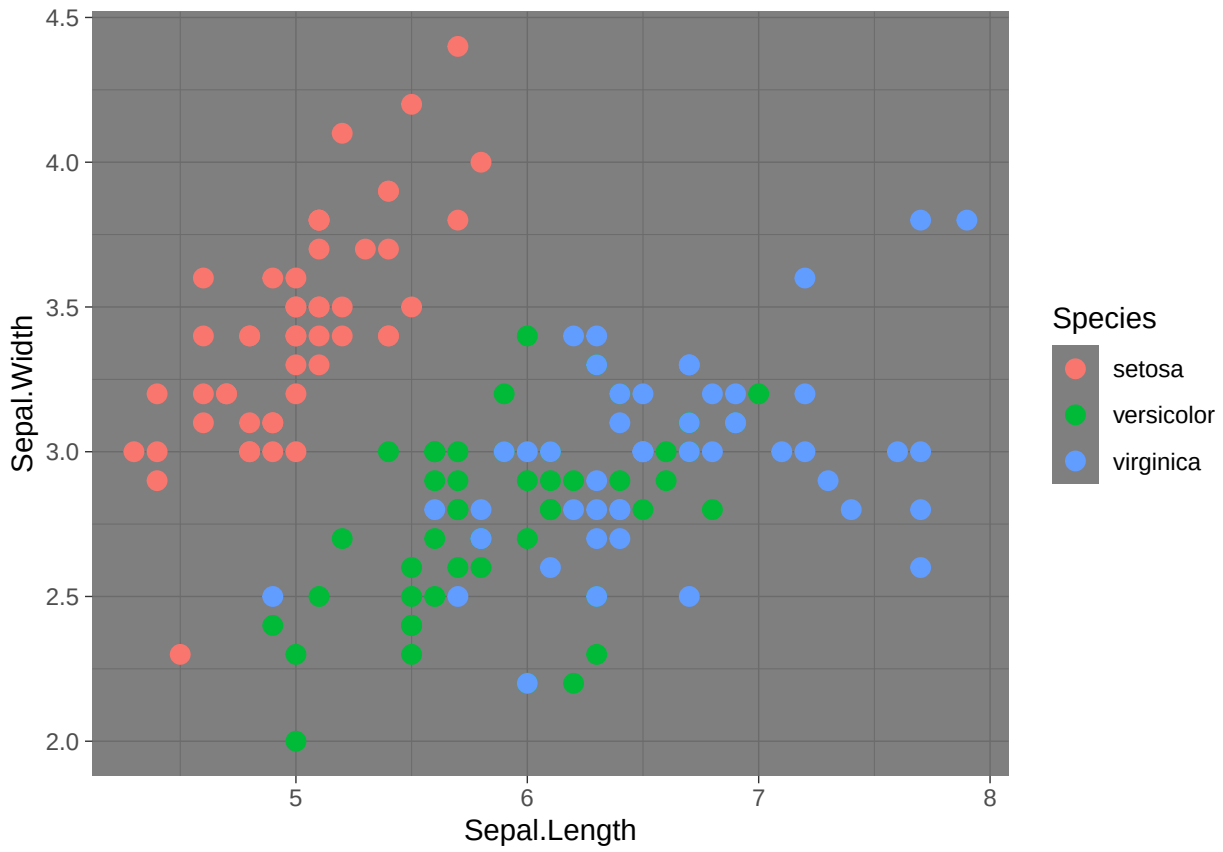
Below is the third plot, which illustrates the classic theme for ggplot2 graphs. The plot now has a white background and no grid markers.

```
ggplot2::ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width)) +
  geom_point(aes(colour = Species), size = 3)+
  theme_classic()
```



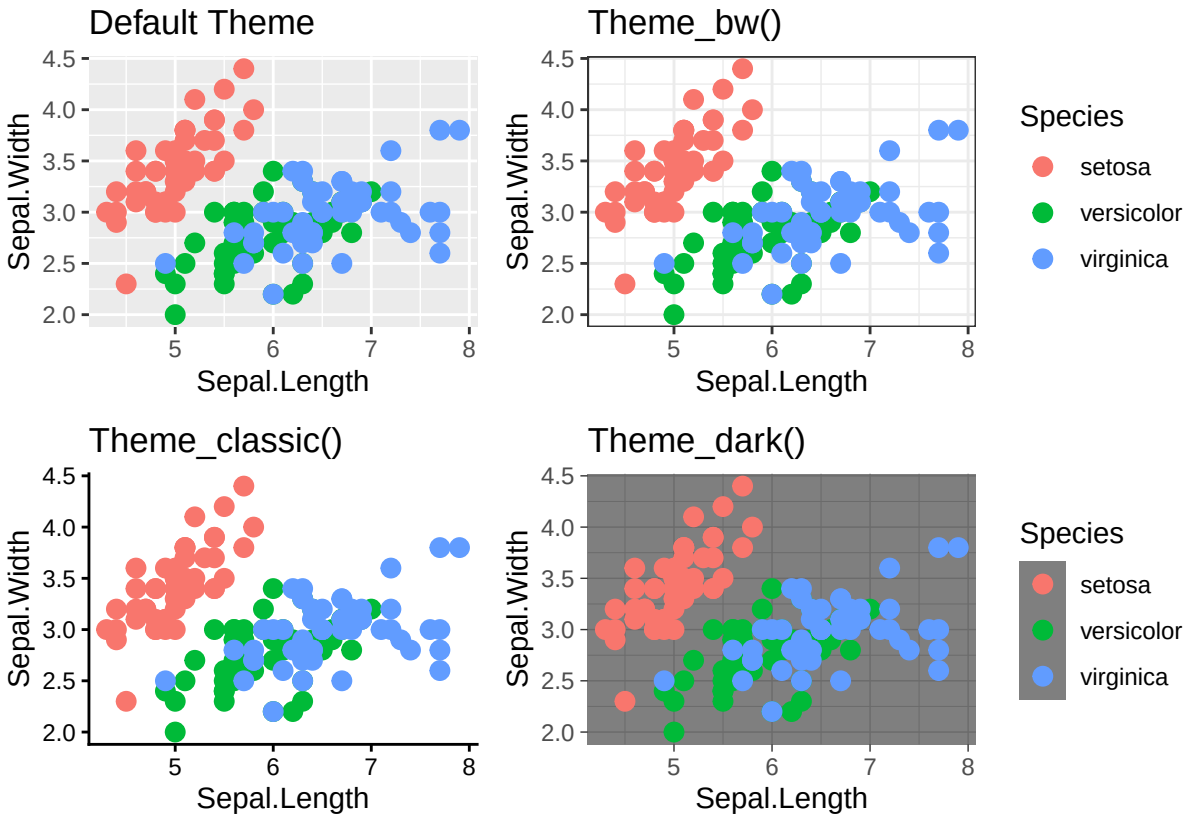
Below is the fourth plot, which shows the dark theme for ggplot2 graphs.

```
ggplot2::ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width)) +
  geom_point(aes(colour = Species), size = 3)+
  theme_dark()
```



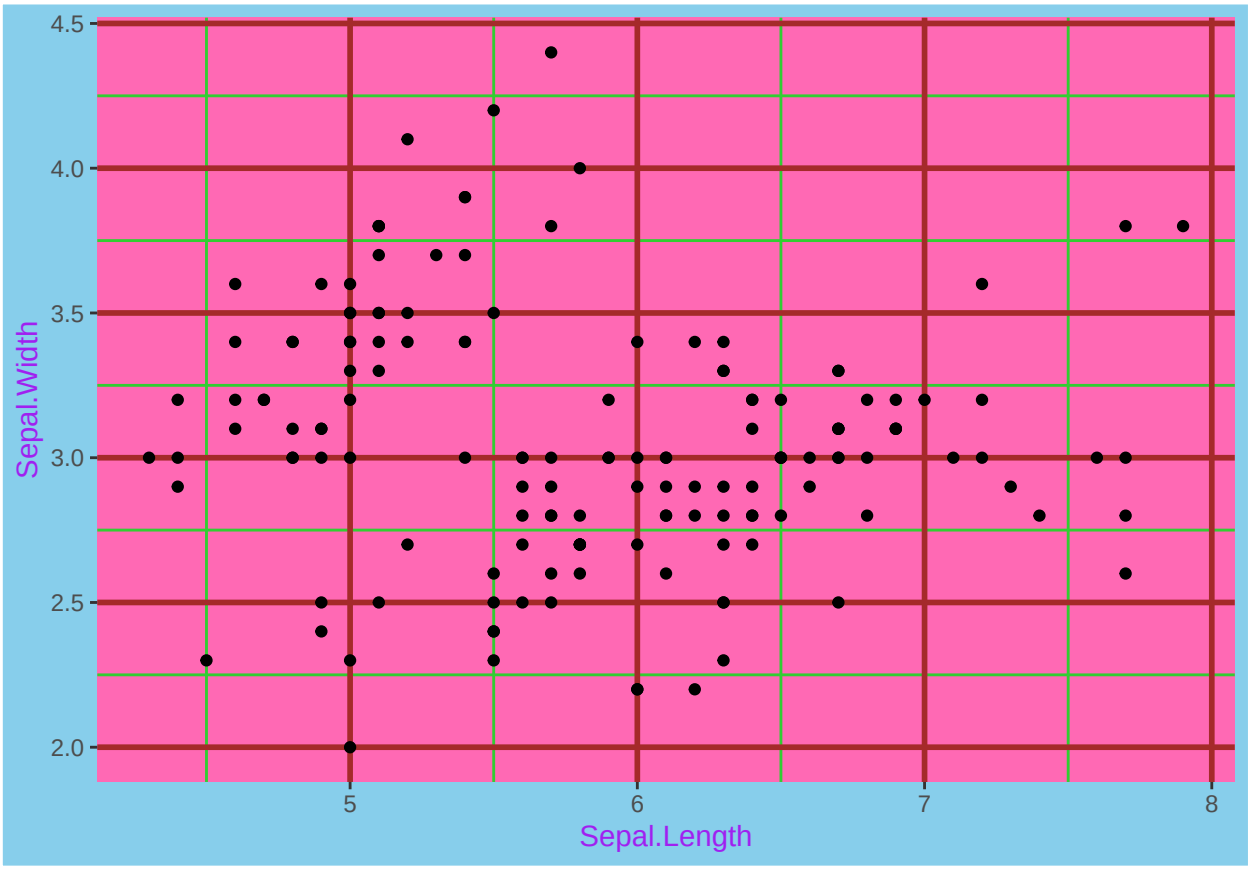
Below is the fifth plot, which shows all of the themes we have covered so far - this includes the default, bw, classic and dark. These were plotted together using another R package that we will discuss further on.

```
library(patchwork)
p_base<- ggplot2::ggplot(iris, aes(x = Sepal.Length,y = Sepal.Width))+
  geom_point(aes(color = Species), size = 3)

# shows all of the plots together using Patchwork Package
p1 <- p_base + ggtitle("Default Theme") +
  theme(legend.position = "none")
p2 <- p_base + theme_bw() + ggtitle("Theme_bw()") +
  theme(legend.position = "right")
p3 <- p_base + theme_classic() + ggtitle("Theme_classic()")+
  theme(legend.position = "none")
p4 <- p_base + theme_dark() + ggtitle("Theme_dark()")+
  theme(legend.position = "right")

# patchwork
(p1|p2)/(p3|p4)
```



Below is a plot that highlights how you can manually change the panel background, plot background and other elements, including the line colour (major and minor), and the axes text colour.

```
ggplot2::ggplot(iris, aes(Sepal.Length, Sepal.Width)) +
  geom_point() +
  theme(
    panel.background = element_rect(fill = "hotpink"),
    plot.background = element_rect(fill = "skyblue"),
    panel.grid.major = element_line(color = "brown",
                                    linewidth = 1),
    panel.grid.minor = element_line(color = "limegreen",
                                    linewidth = 0.5),
    text = element_text(color = "purple")
  )
```



Background and theme options: 1. Look up other themes. What is your favourite theme which you might use by default? 2. Can you use the different theme options in the last example to make a theme based on your favourite film (ours may draw inspiration from Barbie)?

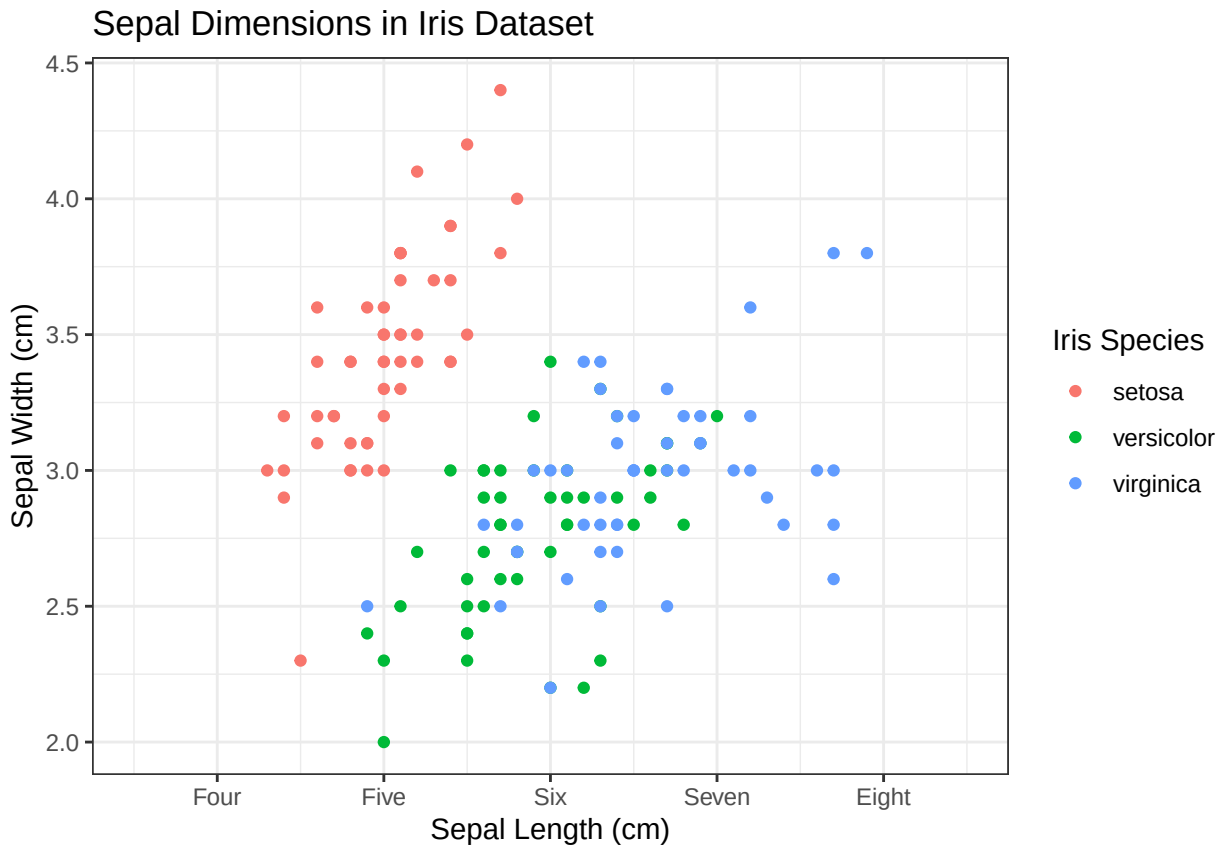
4.1.4.2 Renaming axes

By default, ggplot2 uses the column names used in the aesthetic calls to name the axes; however, that is not always the most useful. In our examples so far, the x-axis has been labelled **`"Sepal.Length"`** and the y-axis has been labelled **`"Sepal.Width"`**. Whilst this is sufficient to describe the data, it could be more descriptive. We can easily rename the axes in ggplot2 using the **`labs`** layer. Within **`labs`** we can change either of the axes, and we can also add a title to the plot and adjust the title of the legend. Note that to change the title of the legend, you have to refer to it as the aesthetic (e.g. **`colour`** or **`fill`**).

You can also use additional functions to make further edits. Here we will demonstrate the function `scale_x_continuous()` which can change aspects of the x-axis. It has an analogous function (`scale_y_continuous()`) that does the same for the y-axis. We are defining three aspects within `scale_x_continuous()`: the breaks, the labels, and the limits. The breaks tell ggplot2 where to place the tick labels. Without any further adjustment, the numbers in breaks will define both the position and the label for each tick. Labels adjust what is written at each tick mark. We have changed the labels from integer numbers to **`"four, five, six, seven, and eight"`**. The limits enable us to define how long we want the axes to be. This can be wider than the data, meaning that there is empty space outside of the data, or can be thinner than the data, meaning some data are not visualised. In the following, we have slightly extended the axes in both directions using **`limits = c(3.5,8.5)`**.

```
ggplot2::ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width))+
  geom_point(aes(color = Species))+
  labs(title = "Sepal Dimensions in Iris Dataset",
       x = "Sepal Length (cm)",
```

```
      y = "Sepal Width (cm)",
      color = "Iris Species")+
scale_x_continuous(breaks = c(4, 5, 6, 7, 8),
                   labels = c("Four", "Five", "Six", "Seven", "Eight"),
                   limits = c(3.5,8.5)) +
theme_bw()
```



Axes options: 1. Can you shrink the x-axis so that only data between 5 and 7cm are shown? 2. Can you increase the y-axis to include negative numbers? 3. Can you change the title of the plot and the legend?

4.1.5 Different Types of Plots

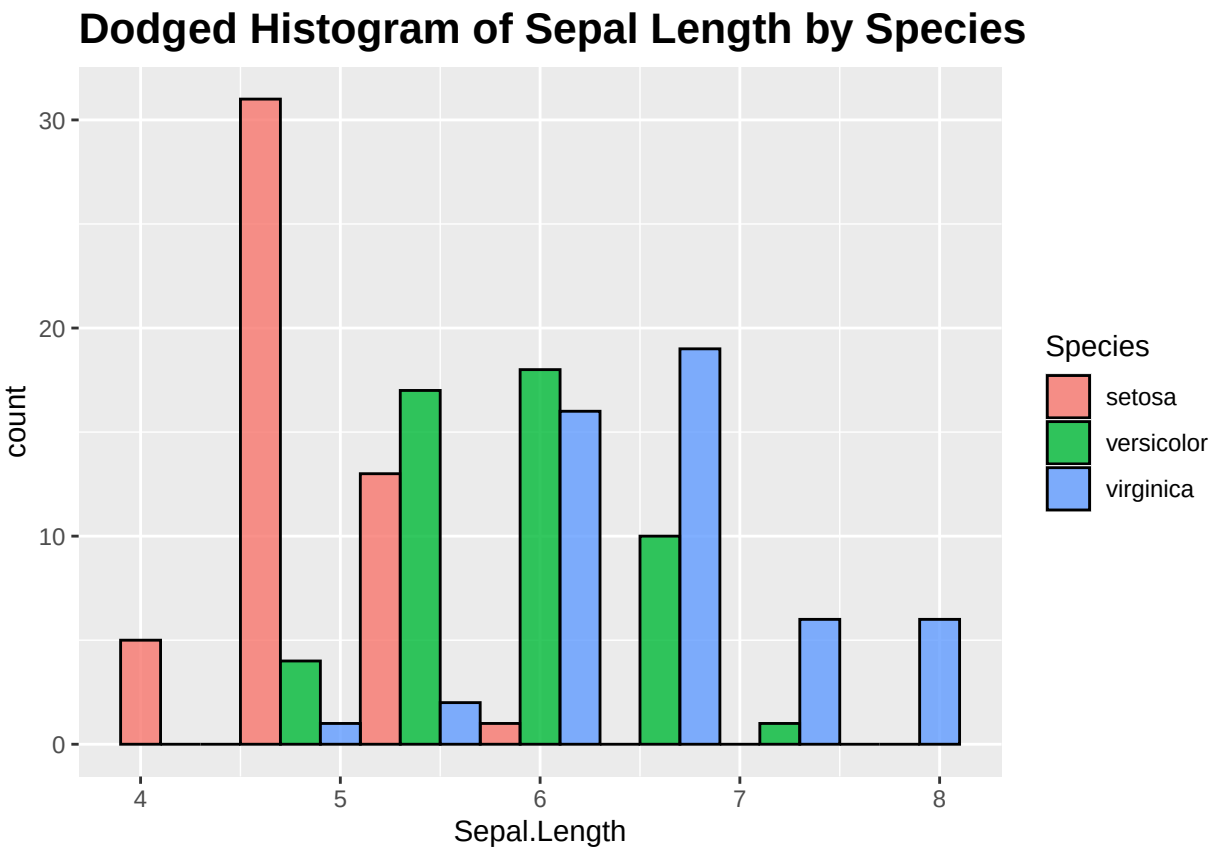
Here we have several types of plots that are useful for visualising different types of data.

4.1.5.1 Histogram

Histograms are useful for showing the distribution of a single numeric variable. This version shows the bars side by side by species within each bin, making it easier to directly compare counts for each species.

Layout in histograms: Look at the plot below 1. What happens if you change the `position` argument from 'dodge' to 'stack'? 2. Change the `binwidth` to values between 0.2 and 1.5 - what is the most useful binwidth? 3. If you change the `position` from 'dodge' to 'identity' the bars should all overlap - can you change the alpha value or the fill and colour values to make it clearer how they overlap?

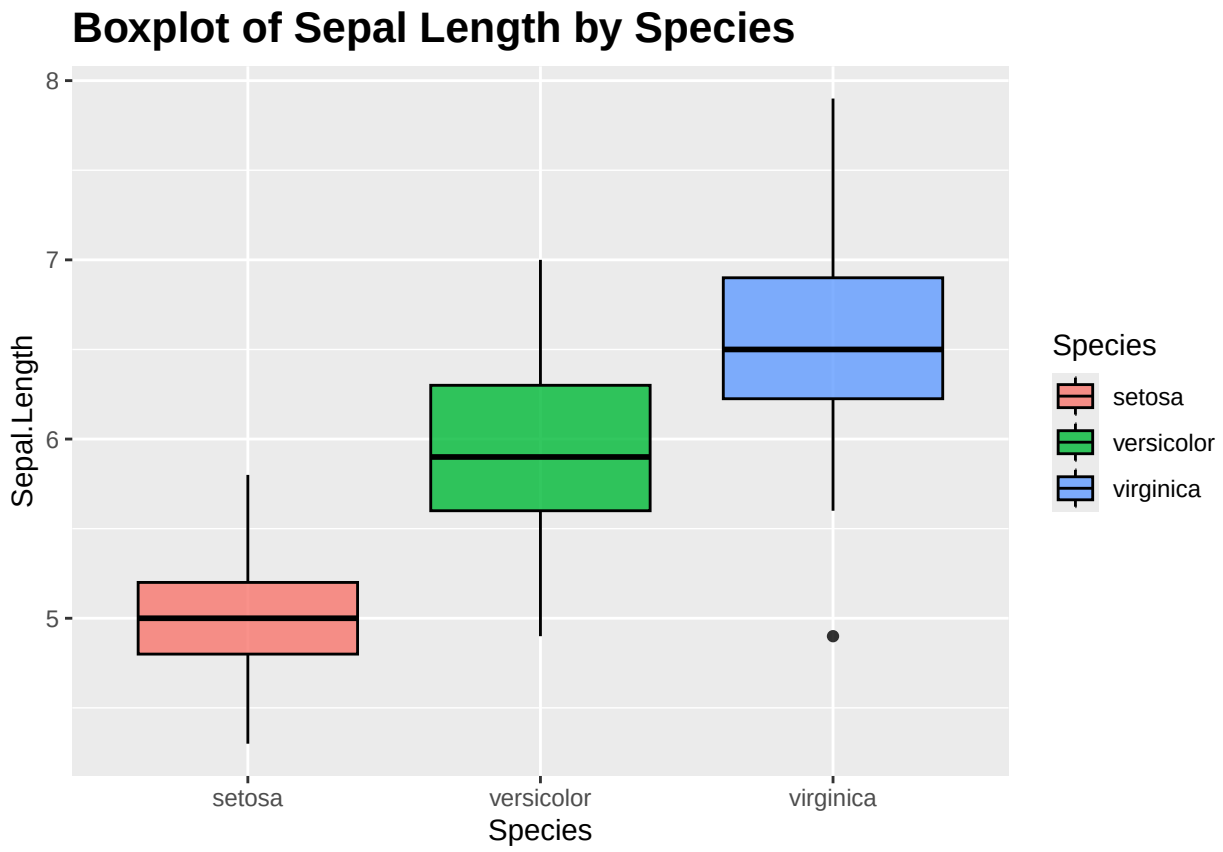
```
ggplot2::ggplot(iris, aes(x = Sepal.Length, fill = Species)) +
  geom_histogram(binwidth = 0.6,
                 color = 'black', alpha = 0.8, position = 'dodge')+
  labs(title = "Dodged Histogram of Sepal Length by Species")+
  theme(plot.title = element_text(size = 16, face = "bold"))
```



4.1.5.2 Box plots

Box plots are great for visualising the spread of data and identifying outliers in continuous data across categories.

```
ggplot2::ggplot(iris, aes(x = Species,
                          y = Sepal.Length,
                          fill = Species))+
  geom_boxplot(color = 'black', alpha = 0.8)+
  labs(title = "Boxplot of Sepal Length by Species")+
  theme(plot.title = element_text(size = 16, face = "bold"))
```

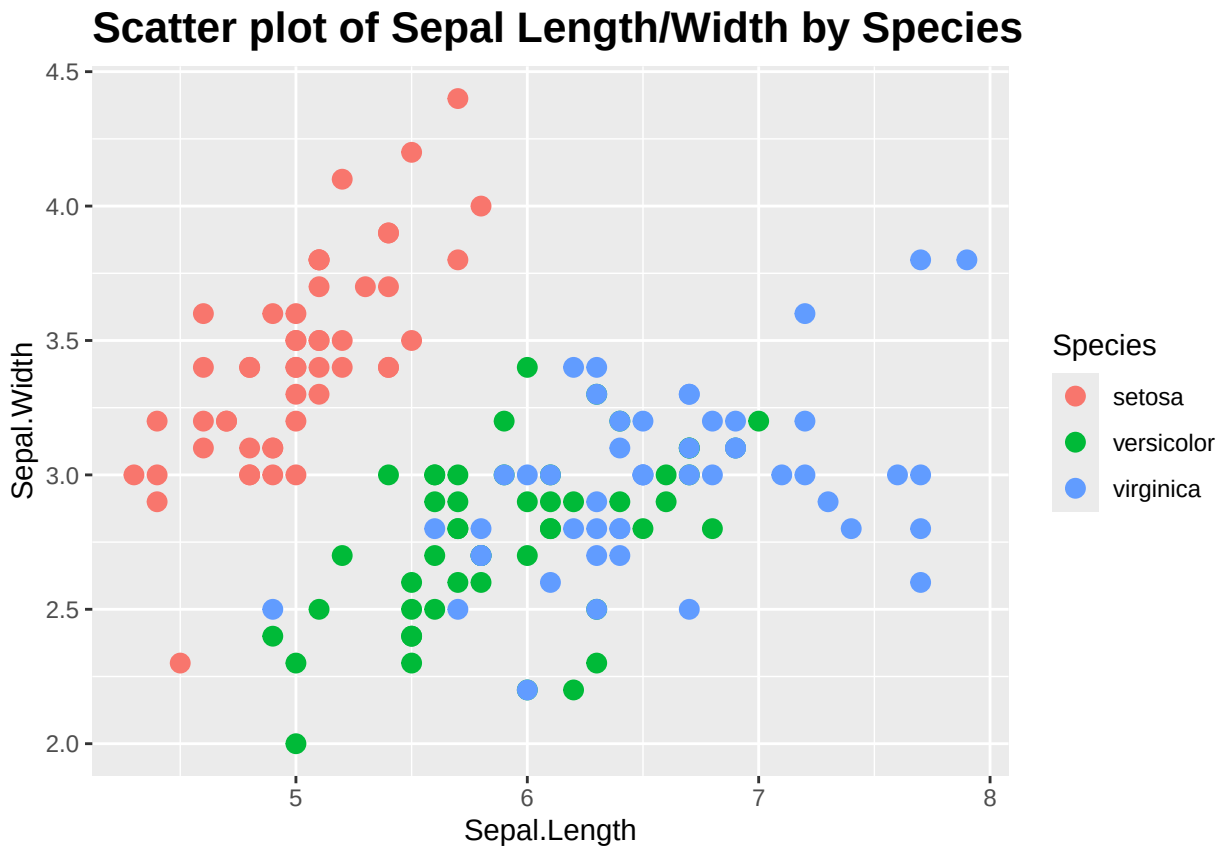


Layout in box plots: 1. Can you make the lines in the box plot turn the same colour as the box fill? Why is this not the best of ideas? 2. If you decide the lines are too thin, you can add in an argument (not an aesthetic) to `geom_boxplot` called `linewidth`.

4.1.5.3 Scatter plots

These are ideal for showing relationships between two continuous variables.

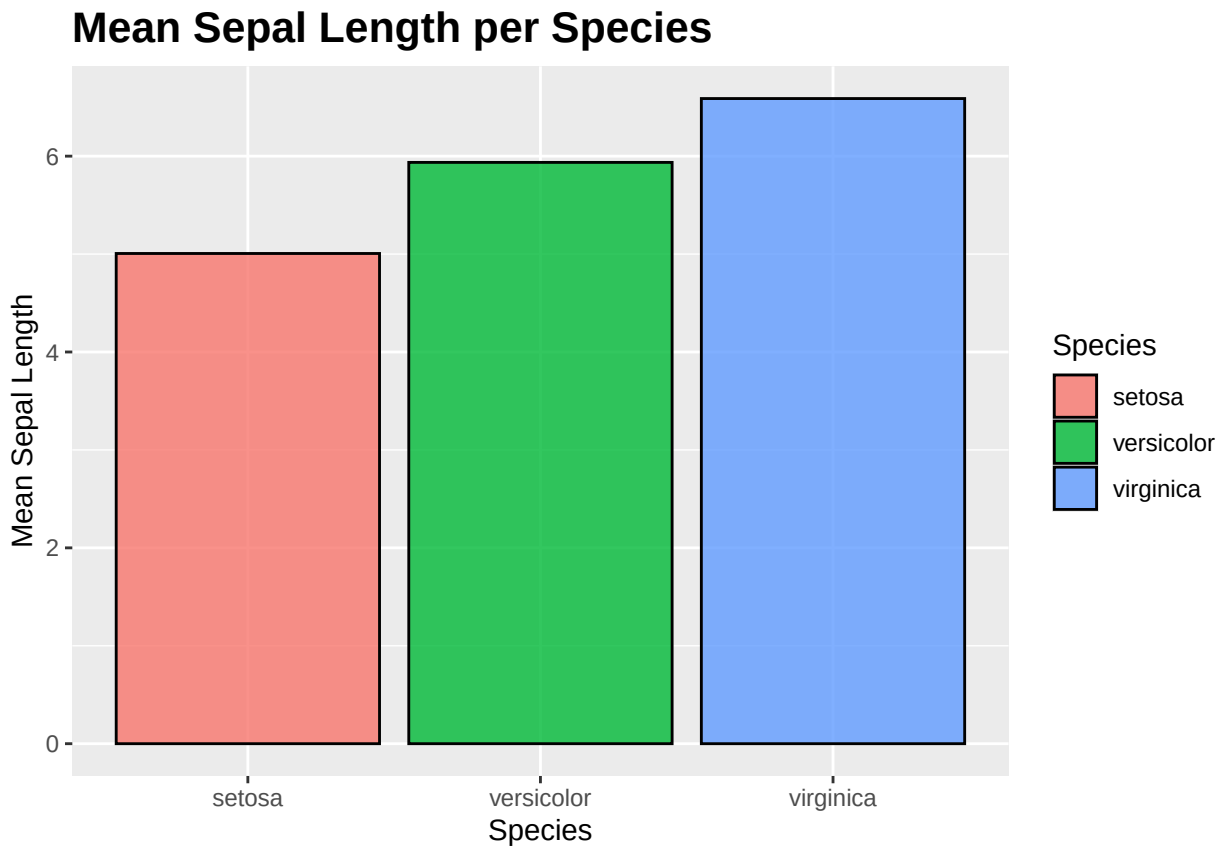
```
ggplot2::ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width)) +
  geom_point(aes(colour = Species), size = 3) +
  labs(title = "Scatter plot of Sepal Length/Width by Species")+
  theme(plot.title = element_text(size = 16, face = "bold"))
```



4.1.5.4 Bar Charts

Bar charts are used for comparing quantities across discrete categories.

```
iris_means <- stats::aggregate(Sepal.Length ~ Species,
                               data = iris, FUN = mean)

ggplot2::ggplot(iris_means, aes(x = Species,
                                y = Sepal.Length,
                                fill = Species)) +
  geom_col(color = 'black', alpha = 0.8) +
  labs(title = "Mean Sepal Length per Species",
       y = "Mean Sepal Length") +
  theme(plot.title = element_text(size = 16, face = "bold"))
```



4.1.5.5 Bar Charts with Error Bars

Here we include error bars. Error bars are graphical representations of the variability or uncertainty in the data. They provide insights into the precision of measurements or the confidence in estimates like means. In bar charts, adding error bars can help people to understand the range within which the true value likely falls.

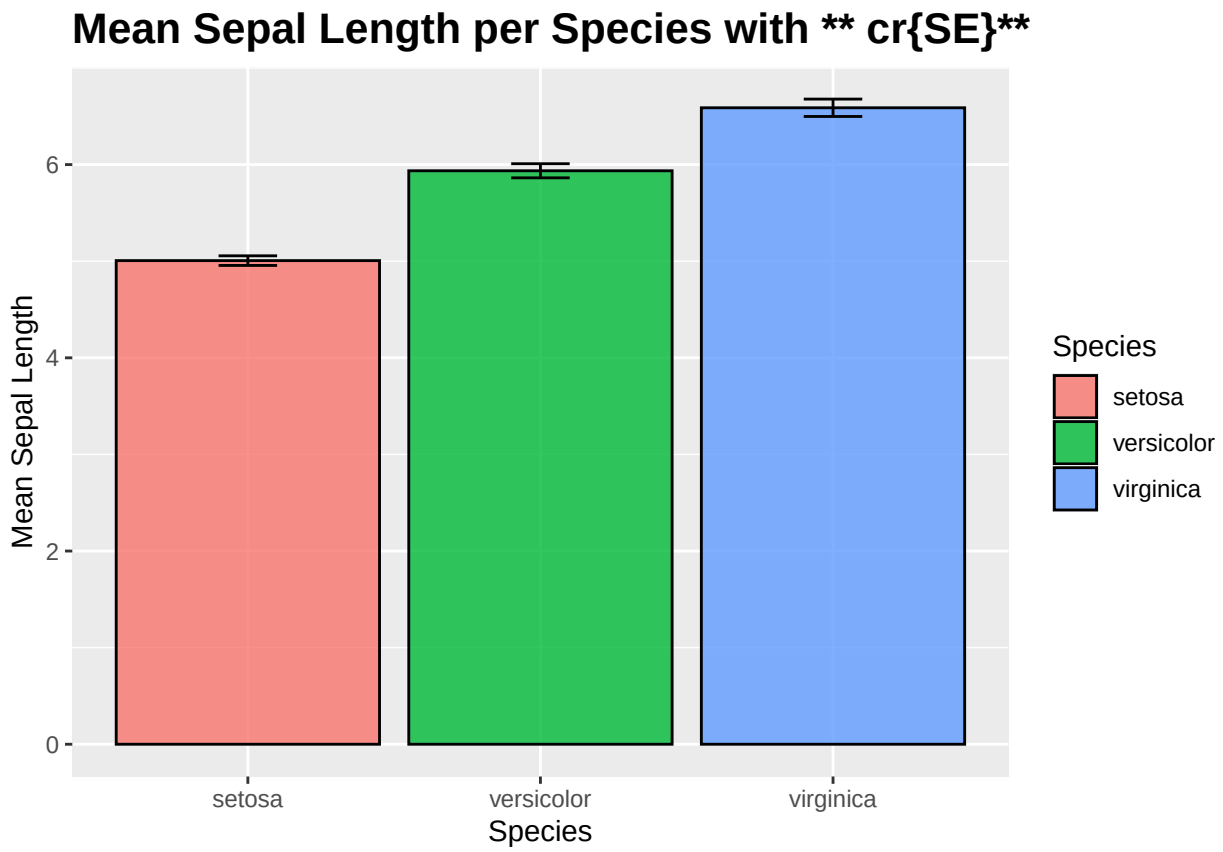
For example, when plotting the mean Sepal Length by species in the iris dataset, we can add error bars to show the **SD** or **SE** within each species group.

```
base::library(dplyr)
iris_summary <- iris %>% dplyr::group_by(Species) %>%
  dplyr::summarise(mean_sepal = base::mean(Sepal.Length),
                   se_sepal = stats::sd(Sepal.Length) /base::sqrt(n()))
```

```
ggplot2::ggplot(iris_summary, aes(x = Species, y = mean_sepal, fill = Species)) +
  geom_col(color = 'black', alpha = 0.8) +
  geom_errorbar(aes(ymin = mean_sepal - se_sepal,
                    ymax = mean_sepal + se_sepal),
                width = 0.2, color = "black") +
  labs(title = "Mean Sepal Length per Species with **\acr{SE}**",
       y = "Mean Sepal Length") +
  theme(plot.title = element_text(size = 16, face = "bold"))
```

```
## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, : font
## width unknown for character 0x07 in encoding cp1252
```

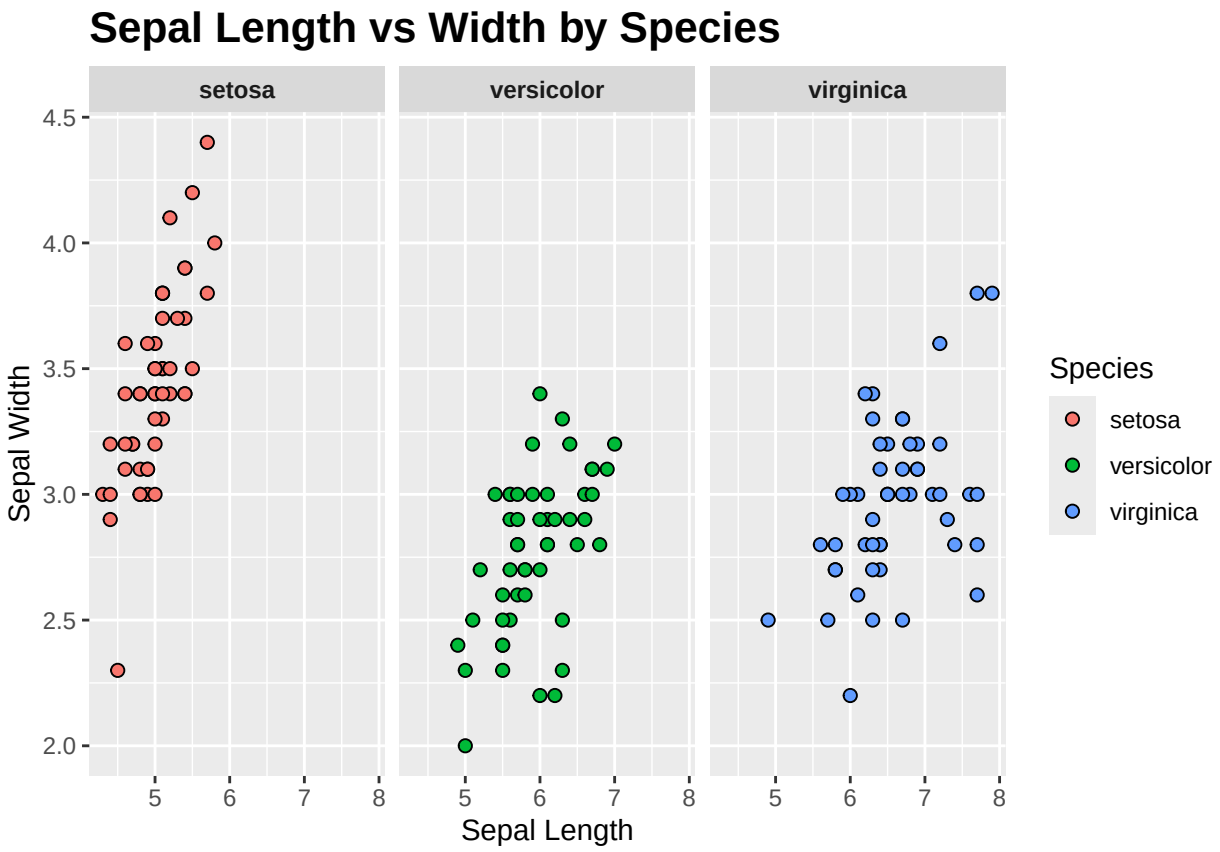
```
## Warning in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x, x$y, :
## font width unknown for character 0x07 in encoding cp1252
```



4.1.5.6 Faceted Plots

Here, we can break the data into subsets and plot each subset in its own panel for comparison.

```
ggplot2::ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width, fill = Species))+
  geom_point(shape = 21, color = 'black', size = 2)+
  facet_wrap(~Species)+
  labs(title = "Sepal Length vs Width by Species",
       x = "Sepal Length", y = "Sepal Width")+
  theme(plot.title = element_text(size = 16, face = "bold"),
        strip.text = element_text(face = "bold"))
```



4.1.5.7 Viridis Colour Palettes

This is an example of how you can use a colour palette in R. Further examples using the `viridis` package can be found here.

```
#base::library(ggplot2)
#base::library(viridis)
ggplot2::ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width, fill = Species)) +
  geom_point(shape = 21, color = "black", size = 2) +
  facet_wrap(~ Species) +
  scale_fill_viridis(option = "plasma", discrete = T) +
  labs(title = "Sepal Length vs Width by Species with Viridis Palette",
       x = "Sepal Length", y = "Sepal Width")  +
  theme(plot.title = element_text(size = 18, face = "bold"),
        strip.text = element_text(face = "bold", size = 12))
```

4.2 Individualising your plots

4.2.1 Colours

We have already shown an example of the viridis colour palette. This set of palettes is designed to be printer and colour-blind friendly. In the example above, we used the "plasma" palette for a discrete selection of data, but all palettes are continuous, and the default for the functions `scale_fill_viridis()` and `scale_colour_viridis()` are for continuous data. More details on the different palettes available through viridis are available here: https://cran.r-project.org/web/packages/viridis/vignettes/intro-to-viridis.html#the-color-scales.

Another similar package that provides different palettes is RColorBrewer. Within ggplot, this can be called, similar to viridis, with the functions `scale_colour_brewer()` and `scale_fill_brewer()`. More information about the colour palettes from RColorBrewer are available here: https://colorbrewer2.org

Sometimes, you just want to define your own palette for plots, and there are many different packages and resources to help select colours. Some of our favourites include:

1. MetBrewer (https://www.blakerobertmills.com/my-work/met-brewer) - palettes based on objects in the Metropolitan Museum of Art in New York
2. coolors (https://coolors.co/generate) - generates new palettes with the click of a space bar
3. PNWColors (https://github.com/jakelawlor/PNWColors) - palettes based on Washington state and the Pacific Northwest of the USA
4. WesAnderson (https://github.com/karthik/wesanderson) - palettes based on Wes Anderson films

It is also worth checking that any palettes you choose are colour-blind friendly. RColorBrewer, viridis, and coolors all are, or have options to easily check, but if you are selecting from other packages or choosing colours by hand, you can use the package colorblindcheck.

To colour sections with your own colours, not from a palette, you need to first define your palette and then use `scale_fill_manual()` or `scale_colour_manual()`, as shown below:

```
SpecCols <- c(setosa = "#A3D9FF", versicolor = "#7E6B8F", virginica = "#96E6B3")
ggplot2::ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width, colour = Species)) +
  geom_point(size  = 3) +
  scale_colour_manual(values = SpecCols) +
  labs(x = "Sepal Length", y = "Sepal Width")  +
  theme_light() +
  theme(plot.title = element_text(size = 18, face = "bold"),
        strip.text = element_text(face = "bold", size = 12))
```

If you use a named list (as above), the colours will match the names provided, regardless of the order in which they are listed. However, if you provide only a list of hex values, R will match up to the order of the groupings (automatically in alphabetical order).

Playing around with colours: 1. Can you create your own palette and apply it to one of the other plot types shown above? 2. What constitutes a good palette, what constitutes a bad palette? 3. Try to use colour as a continuous variable for one of the width or length measurements with a viridis palette. Do you have a favourite?

4.2.2 Changing the position of the legend

The legend automatically appears on the right side of the plot, but is movable. There are four built-in arguments: **`"top"`**, **`"bottom"`**, **`"left"`**, and **`"right"`**. The resulting layout of each of those is shown below.

```
a = ggplot2::ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width)) +
  geom_point(aes(color = Species))

(a + theme(legend.position = "left") + ggtitle("Left")) +
  (a + theme(legend.position = "right") + ggtitle("Right")) +
  (a + theme(legend.position = "top") + ggtitle("Top")) +
  (a + theme(legend.position = "bottom") + ggtitle("Bottom"))
```

We have used `theme(legend.position(...))` for these position changes, but it is also possible to use `guides(colour = guide_legend(position = "bottom"))`. Using the function guides, it is also possible to define a position of `"inside"`. This moves the legend from outside the plot to inside the plot, defaulting to the centre. To adjust the position, assume the plot is a 1x1 box with corners at (0,0), (1,0), (1,1), and (0,1). The position of the inside legend can be anywhere within that box, regardless of the data the plot is showing. Defining that position uses the `theme(legend.position.inside=c(...))` function.

```
a = ggplot2::ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width)) +
  geom_point(aes(color = Species)) +
  guides(colour = guide_legend(position = "inside"))

(a + theme(legend.position.inside = c(0,0)) + ggtitle("bottom left")) +
  (a + theme(legend.position.inside = c(1,1)) + ggtitle("top right")) +
  (a + theme(legend.position.inside = c(0.5,0.5)) + ggtitle("middle")) +
  (a + theme(legend.position.inside = c(0.3, 0.8)) + ggtitle("upper left"))
```

4.2.3 Changing the Font

To make your plots easier to read, you can alter your font, font style, and size. I will give you some examples below. These can also be applied to the facets; click below for more information:

https://www.datanovia.com/en/blog/how-to-change-ggplot-facet-labels/

I have changed the following: 1. The title to **bold, italic and the font serif**. 2. The x-axis title to just **bold and the font sans**. 3. The y-axis title to just **italic and the font mono**. 4. The legend title to just **bold and the font Times**.

```
ggplot2::ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width)) +
  geom_point(aes(color = Species)) +
  labs(
    title = "Sepal Dimensions in Iris Dataset",
    x = "Sepal Length (cm)",
    y = "Sepal Width (cm)",
    color = "Iris Species"
  ) +
  theme_bw() +
  theme(
    plot.title = element_text(size = 20, face = "bold.italic", family = 'serif'),
    axis.title.x = element_text(size = 14, face = "bold", family = 'sans'),
    axis.title.y = element_text(size = 14, face = "italic", family = 'mono'),
    legend.title = element_text(size = 12, face = "bold", family = 'Times')
  )
```

4.2.4 Changing the Font but in Facets

We can also change the font type or style in a facet as well as the background colour.

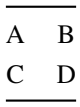
I have changed the following: 1. The background colour of the facet label = `light-blue`. 2. The colour of the facet title text = `darkgreen`. 3. The text box outline for the facet label = `red`. 4. The facet label is now `bold.italic`. 5. The font is now `sans` and size `14`.

```
ggplot2::ggplot(iris, aes(Sepal.Length, Sepal.Width, color = Species)) +
  geom_point(size = 3) +
  facet_wrap(~ Species) +
  theme(strip.text = element_text(
    family = "sans", size = 14,
    face = "bold.italic", color = "darkgreen"
  ),
  strip.background = element_rect(
    fill = "light-blue", color = "red", linewidth = 1)
  )
```

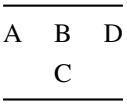
4.2.5 Combining plots

The patchwork package is the simplest package to use if you want to combine multiple graphs into one plot. Save each graph as an object (`graph1 <- ggplot(...) + ...`) and then add each of the graphs together at the end (`graph1 + graph2 + graph3 + graph4`). You can gain more control over the plot by using `|` for plots that you want next to each other and `/` to separate plots that sit above and below each other. As some examples:

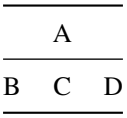
`A + B + C + D` and `(A + B)/(C + D)` and `(A/C) | (B/D)` will all produce the same plot layout:



However, `A + B/C + D` would give:



And `A/(B + C + D)` would give:



For more information on patchwork and further ways of combining plots, visit the website https://patchwork.data-imaginist.com/.

There are other packages that can help combine different plots, such as cowplot. Instead of adding your plots together, with cowplot you use the function `plot_grid(A,B,C,D)` which will give you the same output as the first patchwork example.

4.2.6 Saving your plot

Once you have created your perfect plot, you want to save it to use outside of R, such as in presentations and reports. The important function is `ggsave`. As a minimum, you need to have a file path to save the plot. This should point directly to the folder you want to save to and include the filename you want to save it as, and the file extension. The file extension can be any of the following: .eps, .ps, .tex, .pdf, .jpeg, .tiff, .png, .bmp, .svg.

`ggsave` automatically assumes that the last plot you ran code for is the one that you want to save. If this is not the case, and you have saved a specific graph to the environment, you can add an argument of `ggsave("filepath/filename.png", plot = plotA)`. Other arguments you can add include: width and height, units (can be any of in, cm, mm, or px), **dots per inch (DPI)**, and `bg` (if you want to change the background colour of the plot). There are also other arguments you can add, but these are used less often.

Saving a plot: 1. Can you save one of your plots using `ggsave`? 2. Can you save it in another format? 3. What happens if you change the height, width, or **DPI**?

4.3 Perfect Palmer Penguins Plot

Challenge We have created a plot using aspects of ggplot2 covered above. 1. Can you recreate this plot? 2. Can you improve on this plot?

Before you begin: 1. install the palmer penguins package (`install.packages("palmerpenguins")`) 2. load the palmer penguins package (`library(palmerpenguins)`) 3. the colours used in this plot are **"skyblue"**, **"darkturquoise"**, and **"purple"** 4. it is possible to remove data with **NA** values from the legend using `na.translate = FALSE`

There isn't just one answer to correctly recreate this graph, but it is possible to do it with just the ggplot2 skills acquired above.

```
## Warning: Removed 11 rows containing missing values or values outside the scale range
## (‘geom_point()‘).
```

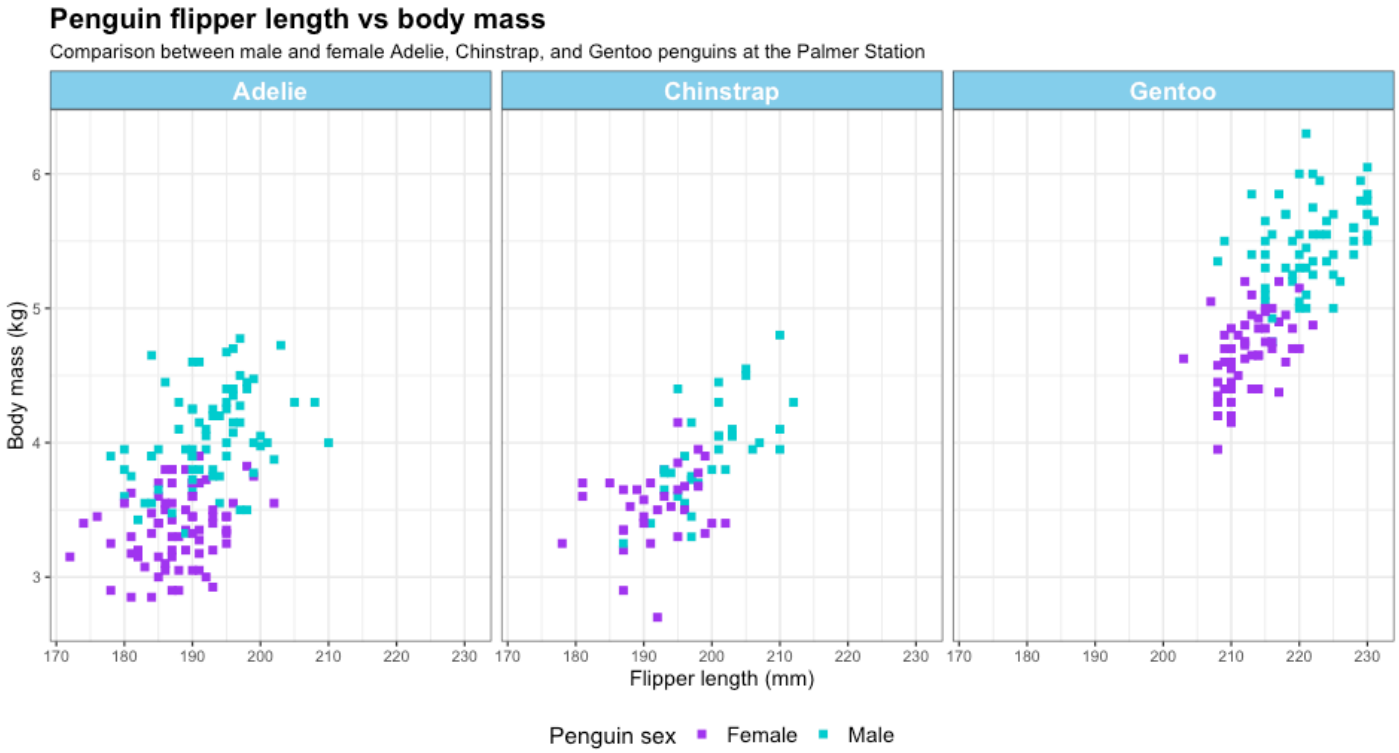
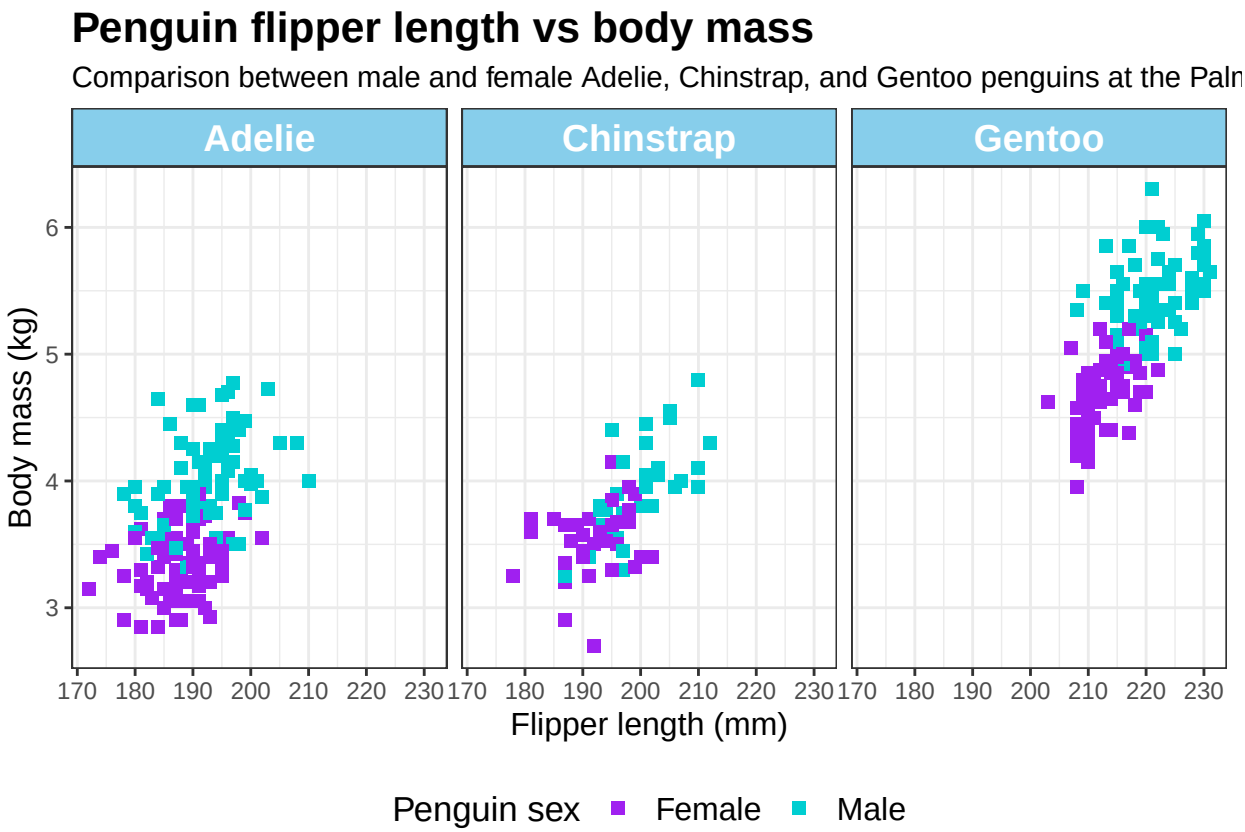


Figure 4.2: Example of plots created using ‘palmerspenguins‘. Can you recreate this or improve on it?



Chapter 5

Hypothesis testing and regression modelling

Before starting this chapter, make sure that you have the following R packages installed and loaded.

```
#Install packages
utils::install.packages("tidyverse")
utils::install.packages("gt")
utils::install.packages("gtsummary")
```

```
library(tidyverse)
library(gt)
library(gtsummary)

rm(list=ls()); gc()
```

```
##            used  (Mb) gc trigger  (Mb) max used  (Mb)
## Ncells 2783975 148.7    4157530 222.1  4157530 222.1
## Vcells 4925432  37.6   10146329  77.5  8388608  64.0
```

One primary aim of statistics is to make inferences, so that we can determine whether the association between the independent variable and the dependent variable (i.e., outcome) is true or could be due to chance. The independent variable may be an "exposure" (thought of as a potential cause), a "predictor" (which may or may not be causal), or a non-causal marker. Hypothesis testing is a common statistical method used to make such inferences.

In this chapter, we will use the data from Chapter 3. To simplify things, we will only use the mean blood tests values, joined with the demographic table:

```
dat1 <- utils::read.csv("data/03-diabetic_demog.csv")
dat2 <- utils::read.csv("data/03-diabetic_labs_final_with_missing.csv")

dat2 <- dat2 %>%
  dplyr::summarise(
    across(Chol:BUN, ~mean(.x, na.rm=T)),
    .by=id
  )
```

```
dat <- dat1 %>%
  dplyr::inner_join(dat2, by='id')

head(dat)
```

```
##   id age Gender BMI SIMD     Chol       TG      HDL      LDL       Cr      BUN
## 1  5  28      M  26    4 4.450000 2.943333 2.286918 3.286918 71.90000 5.113333
## 2 14  12      M  23    4 5.645000 1.133333 2.650251 3.866918 58.03333 4.283333
## 3 16   9      F  30    2 4.336667 1.380000 2.576918 3.296918 68.50000 5.426667
## 4 25   9      M  28    3 4.800000 2.800000 1.350000 2.300000 43.50000 3.850000
## 5 29  13      M  36    4 4.835000 1.765000 1.285000 2.800000 52.45000 4.665000
## 6 46  12      F  20    2 5.670000 0.870000 3.180376 4.405377 65.85000 5.720000
```

Note that the `across()` function has not been introduced in this course, but it can be a handy way to perform the same transformation across multiple variables. Please consult its help page if you're interested.

5.1 Overview of test choices

In statistics courses, you may have learned many different tests for various purposes. The following is a simplified summary of the tests that will be covered in this R course. Note that survival outcomes are taught in the Advanced Statistics course; how their analysis in R will be addressed in a later chapter.

Predictors	Nil	Binary	Categorical	Numeric	Multiple
Numeric outcome	One-sample t-test	Two-sample t-test	Analysis of variance	Correlation coefficients	General linear model
Binary outcome	One-sample proportion test	Two-sample proportion test	Chi-square test	Binary logistic model	Binary logistic model
Count outcome	Count regression	Count regression	Count regression	Count regression	Count regression
Survival outcome	Kaplan-Meier estimator	Log-rank test	Log-rank test	CoxPH model	CoxPH model

5.2 Tests for one outcome variable

Tests for one outcome variable but no predictor variable usually compare the outcome against a fixed benchmark. An example of this is to see if the average **BMI** of the sample is significantly different from 30, the threshold for obesity.

5.2.1 One-sample t-test

In R, all t-tests can be done with the `t.test()` function. In a one-sample t-test, we only need to specify the outcome variable, as well as the benchmark to test against (`mu`). Here, the benchmark is 30.

```
stats::t.test(dat$BMI, mu=30)
```

```
##
##  One Sample t-test
##
## data:  dat$BMI
## t = 2.8949, df = 188, p-value = 0.004242
## alternative hypothesis: true mean is not equal to 30
## 95 percent confidence interval:
##  30.45003 32.37536
## sample estimates:
## mean of x
##   31.4127
```

Here, the output in R shows that the mean **BMI** of the sample was 31.4 (95% **confidence interval (CI)**: 30.5 - 32.4). The p-value of the test was 0.004, indicating the mean **BMI** was significantly different from 30.

5.2.2 Wilcoxon signed rank test

T-tests rely on the central limit theorem to work correctly. If the sample size is too small or the data are highly skewed, the results of those tests might not be reliable. The following shows how we can perform a Wilcoxon signed-rank test against a benchmark. The syntax is similar to that of a t-test: `wilcox.test()`.

```
stats::wilcox.test(dat$BMI, mu=30)
```

```
##
##  Wilcoxon signed rank test with continuity correction
##
## data:  dat$BMI
## V = 9004, p-value = 0.01081
## alternative hypothesis: true location is not equal to 30
```

The R output does not provide any estimates, but just a p-value, which is also very close to zero, indicating the same conclusion as the one-sample t-test.

5.2.3 One-sample proportion test

Similarly, one might also be interested in determining whether the proportion in the sample (e.g., the proportion of individuals with obesity) differs from a specific benchmark (e.g. 10%), which can be tested using the one-sample proportion test. The syntax for this is `prop.test()`. Both the number of diagnosed and the total number of observations need to be specified using **`x`** and **`n`**, respectively.

```
dat <- dat %>%
  dplyr::mutate(obesity=ifelse(BMI >= 30, 1, 0))
stats::prop.test(x=sum(dat$obesity), n=length(dat$obesity), p=0.10)
```

```
##
##  1-sample proportions test with continuity correction
##
## data:  sum(dat$obesity) out of length(dat$obesity), null probability 0.1
## X-squared = 515.05, df = 1, p-value < 2.2e-16
## alternative hypothesis: true p is not equal to 0.1
```

```
## 95 percent confidence interval:
##  0.5240373 0.6676841
## sample estimates:
##         p
## 0.5978836
```

Here, the output in R showed that the proportion (95% **CI**) of obesity was 59.8% (52.4-66.8). The p-value of the test was <0.001, indicating the proportion of **DM** was significantly different from 10%.

5.3 Tests for one outcome variable and one exposure/predictor

5.3.1 Independent two-sample t-test

Note that in the code sample below, instead of specifying `dat$` before **BMI** and Gender, the whole line of code was enclosed by a **`base::with(dat, ...)`** function. The `with()` function tells R to evaluate the `t.test()` function by finding the variables inside `dat`.

```
base::with(dat, stats::t.test(BMI ~ Gender))
```

```
##
##  Welch Two Sample t-test
##
## data:  BMI by Gender
## t = -0.85871, df = 180.49, p-value = 0.3916
## alternative hypothesis: true difference in means between group F and group M is not equal to 0
## 95 percent confidence interval:
##  -2.782425  1.095009
## sample estimates:
## mean in group F mean in group M
##        30.96629        31.81000
```

5.3.2 Wilcoxon rank sum test

When the sample size is too small (a common rule of thumb is n < 30) and/or the variable is highly skewed, the central limit theorem might not work well. In this scenario, using non-parametric tests, such as the Wilcoxon rank-sum test (`wilcox.test()`) could be an option.

```
base::with(dat, stats::wilcox.test(BMI ~ Gender))
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  BMI by Gender
## W = 4106, p-value = 0.3594
## alternative hypothesis: true location shift is not equal to 0
```

5.3.3 Paired t-test

The paired t-test is useful when a study has a matched design or when the data have repeated measures. To illustrate a paired t-test, we will use the data from Chapter 3. The code below demonstrates how to convert long-format data into wide format, including total cholesterol.

```
dat2 <- utils::read.csv("data/03-diabetic_labs_final_with_missing.csv")
dat2 <- dat2 %>%
  dplyr::group_by(id) %>%
  dplyr::mutate(t=1:n()) %>%
  dplyr::ungroup() %>%
  dplyr::select(id, t, Chol) %>%
  tidyr::pivot_wider(id_cols = id, names_from = t, names_prefix = "Chol_",
    values_from = Chol)
head(dat2)
```

```
## # A tibble: 6 x 4
##      id Chol_1 Chol_2 Chol_3
##   <int>  <dbl>  <dbl>  <dbl>
## 1     5   4.9    3.6    4.85
## 2    14   6.49  NA      4.8
## 3    16   3.3    4.31   5.4
## 4    25   5      4.6   NA
## 5    29   4.17   5.5   NA
## 6    46   5.83   5.51  NA
```

Can you deconstruct the code above and explain what it does in each step?

Once the wide-format data are ready, it is very straightforward to conduct a paired t-test. Put the two vectors in and specify **`paired=T`**:

```
base::with(dat2, stats::t.test(Chol_1, Chol_2, paired=T))
```

```
##
##  Paired t-test
##
## data:  Chol_1 and Chol_2
## t = 0.68612, df = 187, p-value = 0.4935
## alternative hypothesis: true mean difference is not equal to 0
## 95 percent confidence interval:
##  -0.1328674  0.2745775
## sample estimates:
## mean difference
##      0.07085507
```

5.3.4 Two-sample proportion test

If the outcome variable is binary, we can perform the two-sample proportion test to examine the differences in proportion by a binary variable (i.e., gender). It is easiest to provide a 2x2 contingency table using the `table()` function.

```
base::with(dat, stats::prop.test(base::table(Gender, obesity)))
```

```
##
##  2-sample test for equality of proportions with continuity correction
##
## data:  base::table(Gender, obesity)
## X-squared = 0.64948, df = 1, p-value = 0.4203
## alternative hypothesis: two.sided
## 95 percent confidence interval:
##  -0.08234507  0.21874956
## sample estimates:
##    prop 1    prop 2
## 0.4382022 0.3700000
```

5.3.5 Chi-square test of independence

When examining the association between two categorical variables, the Chi-square test of independence can be used. For illustrative purposes, we will create a new variable for **BMI** categories.

```
dat <- dat %>%
 dplyr::mutate(BMI_cat = dplyr::case_when(BMI >= 30 ~ 2,
                           BMI >= 25 ~ 1,
                           BMI >= 18.5 ~ 0,
                           BMI >= 0 ~ -1))
head(dat)
```

```
##   id age Gender BMI SIMD     Chol       TG      HDL      LDL       Cr      BUN
## 1  5  28      M  26    4 4.450000 2.943333 2.286918 3.286918 71.90000 5.113333
## 2 14  12      M  23    4 5.645000 1.133333 2.650251 3.866918 58.03333 4.283333
## 3 16   9      F  30    2 4.336667 1.380000 2.576918 3.296918 68.50000 5.426667
## 4 25   9      M  28    3 4.800000 2.800000 1.350000 2.300000 43.50000 3.850000
## 5 29  13      M  36    4 4.835000 1.765000 1.285000 2.800000 52.45000 4.665000
## 6 46  12      F  20    2 5.670000 0.870000 3.180376 4.405377 65.85000 5.720000
##   obesity BMI_cat
## 1       0       1
## 2       0       0
## 3       1       2
## 4       0       1
## 5       1       2
## 6       0       0
```

Can you use R to check the validity of the **BMI** category variable? Hint: check descriptive statistics and/or distribution of **BMI** by **BMI** categories.

```
base::with(dat, stats::chisq.test(Gender, BMI_cat))
```

```
## Warning in stats::chisq.test(Gender, BMI_cat): Chi-squared approximation may be
## incorrect
```

```
##
##  Pearson's Chi-squared test
##
## data:  Gender and BMI_cat
## X-squared = 1.6114, df = 3, p-value = 0.6568
```

Note that R threw a warning that the Chi-squared approximation may be incorrect due to lower cell counts.

5.3.6 Fisher exact test

Since the chi-square test was potentially violated, we can use the Fisher test, `fisher.test()`.

```
base::with(dat, stats::fisher.test(Gender, BMI_cat))
```

```
##
##  Fisher's Exact Test for Count Data
##
## data:  Gender and BMI_cat
## p-value = 0.6525
## alternative hypothesis: two.sided
```

Fisher's test can be computationally intensive. To overcome this, you can specify a larger workspace:

```
base::with(dat, stats::fisher.test(Gender, BMI_cat, workspace = 1e8))
```

```
##
##  Fisher's Exact Test for Count Data
##
## data:  Gender and BMI_cat
## p-value = 0.6525
## alternative hypothesis: two.sided
```

Or use simulation:

```
base::with(dat, stats::fisher.test(Gender, BMI_cat, simulate.p.value = T))
```

```
##
##  Fisher's Exact Test for Count Data with simulated p-value (based on
##  2000 replicates)
##
## data:  Gender and BMI_cat
## p-value = 0.6507
## alternative hypothesis: two.sided
```

5.3.7 Correlation coefficient

Pearson's correlation coefficients can be calculated using `cor.test()`:

```
base::with(dat, stats::cor.test(BMI, Chol))
```

Characteristic	F N = 89[1]	M N = 100[1]
age	16 (10, 35)	15 (9, 28)
BMI	30 (26, 36)	31 (28, 36)
SIMD		
ăăăă1	12 (13%)	19 (19%)
ăăăă2	21 (24%)	19 (19%)
ăăăă3	19 (21%)	17 (17%)
ăăăă4	23 (26%)	26 (26%)
ăăăă5	14 (16%)	19 (19%)
Chol	4.92 (4.57, 5.35)	4.89 (4.57, 5.34)
TG	2.09 (1.65, 2.65)	2.10 (1.52, 2.65)
HDL	1.51 (1.16, 2.59)	2.04 (1.26, 2.64)
LDL	3.19 (2.76, 3.68)	3.13 (2.62, 3.66)
Cr	68 (61, 77)	70 (59, 78)

[1]Median (Q1, Q3); n (%)

```
##
##  Pearson's product-moment correlation
##
## data:  BMI and Chol
## t = -1.8635, df = 187, p-value = 0.06396
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
##  -0.272503911  0.007856303
## sample estimates:
##        cor
## -0.1350254
```

For ordinal variables, use Spearman's by including `method = "spearman"`.

Attempt the following: 1. Conduct a formal statistical test to check whether total cholesterol is statistically different by diagnosis of diabetes. 2. Create a binary variable to indicate high cholesterol (1 where **cholesterol (Chol)** >= 5 or 0 where **Chol** < 5). Conduct a test to examine if there is an association between high cholesterol and **BMI** categories. 3. Is there a correlation between total cholesterol and **low-density lipoprotein (LDL)**? Can you use a plot to illustrate and conduct a formal statistical test to confirm?

5.4 Using `gtsummary` for "Table 1"

In R, we can use the `gtsummary` package to help us with publication-quality tables. Suppose we want to show the characteristics of our study sample by gender:

```
dat %>%
  gtsummary::tbl_summary(by=Gender, include=2:10)
```

If we want to change it to mean and **SD**:

Characteristic	**F** N = 89[1]	**M** N = 100[1]
age	22 (16)	20 (14)
BMI	31 (7)	32 (6)
SIMD		
ăăăă1	12 (13%)	19 (19%)
ăăăă2	21 (24%)	19 (19%)
ăăăă3	19 (21%)	17 (17%)
ăăăă4	23 (26%)	26 (26%)
ăăăă5	14 (16%)	19 (19%)
Chol	4.97 (0.67)	4.94 (0.56)
TG	2.21 (0.84)	2.13 (0.76)
HDL	1.95 (0.93)	2.02 (0.91)
LDL	3.21 (0.76)	3.12 (0.68)
Cr	72 (41)	71 (21)

[1]Mean (SD); n (%)

```
dat %>%
  gtsummary::tbl_summary(by=Gender, include=2:10,
              statistic = list(all_continuous() ~ "{mean} ({sd})"))
```

To add p-values, use `add_p()`:

```
dat %>%
  gtsummary::tbl_summary(by=Gender, include=2:10,
              statistic = list(all_continuous() ~ "{mean} ({sd})")) %>%
  gtsummary::add_p()
```

Can you create a characteristic table to display the demographic and biomarker values of the study sample overall, as well as by diabetes diagnosis? Please also add p-values for the differences by diabetes diagnosis.

5.5 General linear models

General linear models describe a family of regression models for numeric outcomes and can be modelled using the `lm()` function in R.

5.5.1 Simple linear regression

In R, regression models are most often specified using a "formula" notation containing a `~`. The left side is the outcome; the right side is the predictor.

```
mod <- stats::lm(Chol ~ BMI, data = dat)
base::summary(mod)
```

```
##
## Call:
## stats::lm(formula = Chol ~ BMI, data = dat)
##
```

Characteristic	F N = 89[1]	M N = 100[1]	p-value[2]
age	22 (16)	20 (14)	0.3
BMI	31 (7)	32 (6)	0.4
SIMD			0.7
ăăăă1	12 (13%)	19 (19%)	
ăăăă2	21 (24%)	19 (19%)	
ăăăă3	19 (21%)	17 (17%)	
ăăăă4	23 (26%)	26 (26%)	
ăăăă5	14 (16%)	19 (19%)	
Chol	4.97 (0.67)	4.94 (0.56)	0.8
TG	2.21 (0.84)	2.13 (0.76)	0.6
HDL	1.95 (0.93)	2.02 (0.91)	0.6
LDL	3.21 (0.76)	3.12 (0.68)	0.5
Cr	72 (41)	71 (21)	0.5

[1]Mean (SD); n (%)
[2]Wilcoxon rank sum test; Pearson's Chi-squared test

```
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.62666 -0.35944 -0.04105  0.40239  1.74120
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  5.342058   0.212017  25.196   <2e-16 ***
## BMI         -0.012302   0.006601  -1.864    0.064 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6072 on 187 degrees of freedom
## Multiple R-squared:  0.01823,    Adjusted R-squared:  0.01298
## F-statistic: 3.473 on 1 and 187 DF,  p-value: 0.06396
```

5.5.2 Multiple regression

Multiple regression can be done by adding `+ variables` on the right side of the formula.

```
mod <- stats::lm(Chol ~ BMI + age + Gender, data = dat)
base::summary(mod)
```

```
##
## Call:
## stats::lm(formula = Chol ~ BMI + age + Gender, data = dat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.70777 -0.35841 -0.03938  0.39706  1.84994
```

```
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  5.460372   0.226468  24.111   <2e-16 ***
## BMI         -0.012549   0.006611  -1.898   0.0592 .
## age         -0.004555   0.002978  -1.530   0.1278
## GenderM     -0.030336   0.088909  -0.341   0.7333
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6066 on 185 degrees of freedom
## Multiple R-squared:  0.03072,    Adjusted R-squared:  0.01501
## F-statistic: 1.955 on 3 and 185 DF,  p-value: 0.1224
```

5.5.3 Analysis of variance

The **analysis of variance (ANOVA)** model is a special case of the general linear model, used when the predictor is ordinal or categorical.

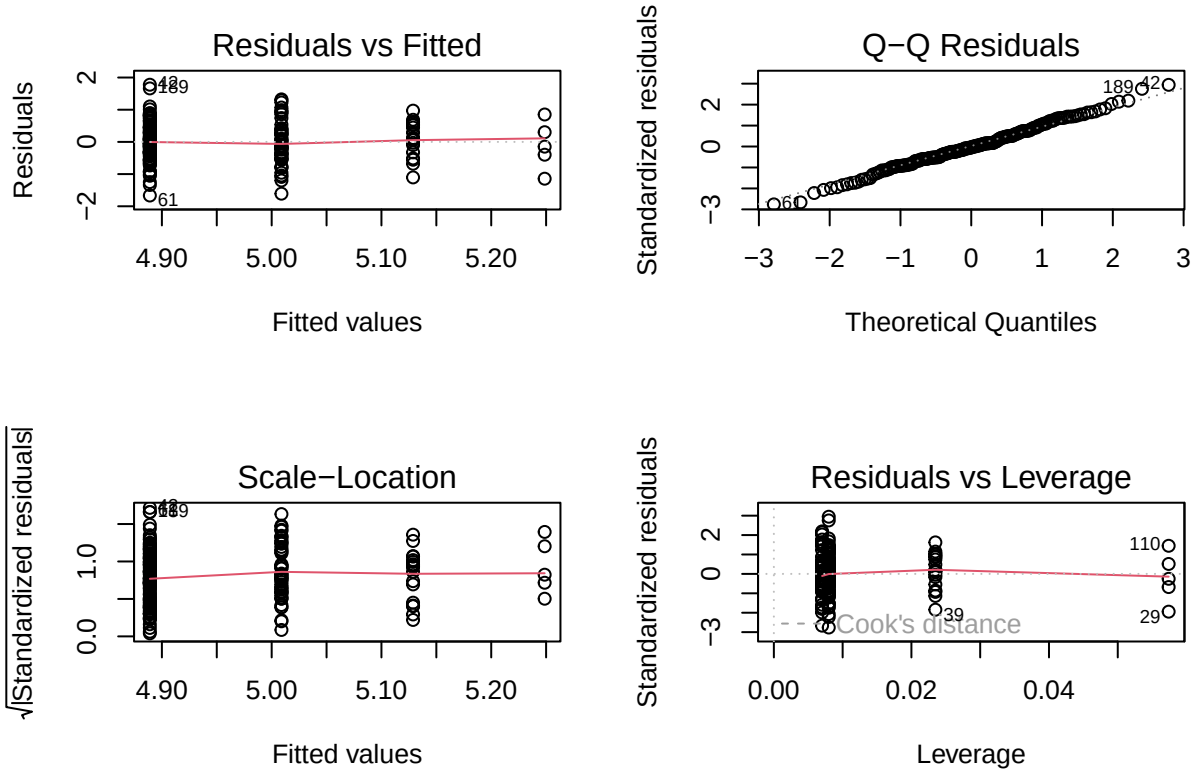
```
mod <- stats::aov(Chol ~ BMI_cat, data = dat)
base::summary(mod)
```

```
##              Df Sum Sq Mean Sq F value Pr(>F)
## BMI_cat       1   1.65  1.6479   4.493 0.0354 *
## Residuals   187  68.59  0.3668
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

5.5.4 Diagnostics

In general linear models, diagnostics can be done easily by plotting the model object. We use `graphics::par(mfrow=c(2, 2))` to arrange plots in a 2x2 panel.

```
graphics::par(mfrow=c(2, 2))
graphics::plot(mod)
```



5.6 Generalised linear models

The generalised linear model (GLM) is a even more general version of the general linear model. But the main difference is that general linear model only handles numeric outcomes, while the GLM handles other types of outcome data. Note that because both model classes have the same acronyms and it could be confusing which 'GLM' people refer to. But in this ebook 'GLM' refers to generalised linear model.

5.6.1 Binary outcomes

Below is an example of a binary logistic regression using `family = "binomial"`.

```
dat <- dat %>%
  dplyr::mutate(High_chol = base::ifelse(Chol >= 5, 1, 0))
mod <- stats::glm(High_chol ~ BMI, data = dat, family = "binomial")
base::summary(mod)
```

```
##
## Call:
## stats::glm(formula = High_chol ~ BMI, family = "binomial", data = dat)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  1.63209    0.74039   2.204   0.0275 *
## BMI         -0.06080    0.02338  -2.601   0.0093 **
```

Characteristic	OR	95% CI	p-value
BMI	0.94	0.90, 0.98	0.009

Abbreviations: CI = Confidence Interval, OR = Odds Ratio

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 258.69  on 188  degrees of freedom
## Residual deviance: 251.50  on 187  degrees of freedom
## AIC: 255.5
##
## Number of Fisher Scoring iterations: 4
```

To calculate Odds Ratios (**odds ratio (OR)**) and CIs manually:

```
base::data.frame(OR = base::exp(base::summary(mod)$coef[2, 1]),
           LCI=base::exp(stats::confint(mod)[2, 1]),
           UCI=base::exp(stats::confint(mod)[2, 2]),
           p=base::summary(mod)$coef[2, 4])
```

```
## Waiting for profiling to be done...
## Waiting for profiling to be done...
```

```
##          OR       LCI       UCI          p
## 1 0.9410078 0.8974717 0.9840495 0.00929689
```

Can you deconstruct the code above and explain what each part does? Specifically why the indices `[2, 1]`, `[2, 2]`, `[2, 4]` were used?

Alternatively, use `gtsummary`:

```
mod %>% gtsummary::tbl_regression(exp = T)
```

5.6.2 Count outcomes

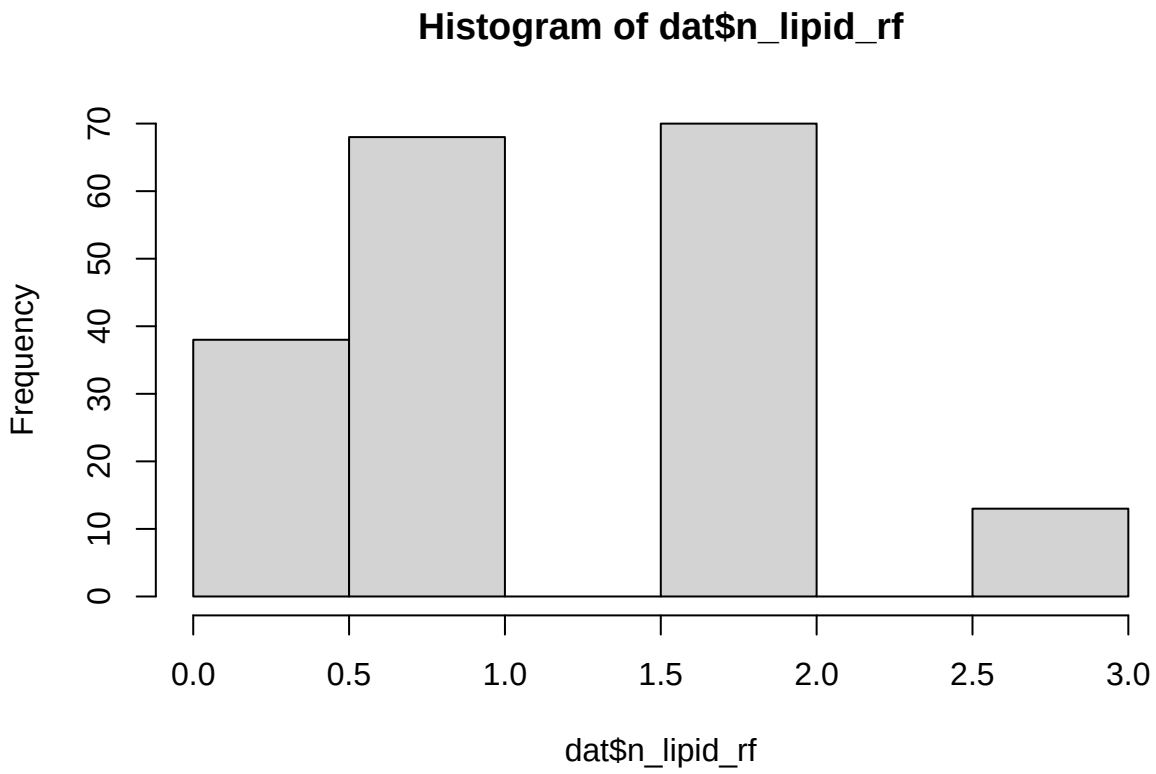
Poisson regression can be used for count outcomes using `family = "poisson"`.

```
dat <- dat %>%
  dplyr::mutate(
  n_lipid_rf = (Chol >= 5) +
              case_when(
                Gender=='M' & HDL < 1 ~ 1,
                Gender=='F' & HDL < 1.2 ~ 1,
                .default = 0,
              ) +
              (TG >= 1.7)
```

Characteristic	IRR	95% CI	p-value
age	1.00	0.99, 1.01	0.8
Gender			
ăăăăF			
ăăăăM	0.85	0.66, 1.09	0.2
BMI	0.98	0.96, 1.00	0.075

Abbreviations: CI = Confidence Interval, IRR = Incidence Rate Ratio

```
  )
graphics::hist(dat$n_lipid_rf)
```




```
mod <- stats::glm(n_lipid_rf ~ age + Gender + BMI,
    data = dat, family = "poisson")
mod %>% gtsummary::tbl_regression(exp = T)
```

Check for overdispersion using the `performance` package:

```
performance::check_overdispersion(mod)
```

```
## # Overdispersion test
##
##        dispersion ratio =   0.567
##   Pearson's Chi-Squared = 104.905
```

Characteristic	IRR	95% CI	p-value
age	1.00	0.99, 1.01	0.8
Gender			
ăăăăF			
ăăăăM	0.85	0.66, 1.09	0.2
BMI	0.98	0.96, 1.00	0.075

Abbreviations: CI = Confidence Interval, IRR = Incidence Rate Ratio

```
##                 p-value =       1

## No overdispersion detected.
```

If overdispersed, use Negative Binomial regression via the `MASS` package:

```
mod <- MASS::glm.nb(n_lipid_rf ~ age + Gender + BMI, data = dat)
```

```
## Warning in theta.ml(Y, mu, sum(w), w, limit = control$maxit, trace =
## control$trace > : iteration limit reached
## Warning in theta.ml(Y, mu, sum(w), w, limit = control$maxit, trace =
## control$trace > : iteration limit reached
```

```
mod %>% gtsummary::tbl_regression(exp=T)
```

5.6.3 Sandwich estimator

The sandwich estimator provides robust standard errors.

```
mod <- stats::glm(n_lipid_rf ~ age + Gender + BMI, data = dat, family = "poisson")
lmtest::coeftest(mod, vcov = sandwich::sandwich)
```

```
##
## z test of coefficients:
##
##               Estimate Std. Error z value  Pr(>|z|)
## (Intercept)  0.9098045  0.2337570  3.8921 9.938e-05 ***
## age         -0.0013351  0.0031579 -0.4228   0.67247
## GenderM     -0.1608495  0.0941015 -1.7093   0.08739 .
## BMI         -0.0171825  0.0071394 -2.4067   0.01610 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

1. Can you test the association between total cholesterol (exposure) and diabetes diagnosis (outcome), adjusting for age and gender?
2. Create a new count variable on the number of metabolic risk factors: **BMI** >= 30; total cholesterol >= 5; diabetes=1. Is there an association between age and gender with the number of metabolic risk factors? Please use an appropriate count regression to examine this.

5.7 Summary

In this chapter, you have learnt to perform common statistical tests and regression models in R. The coding of these is generally straightforward, but the proper use and interpretation of them will require subject, epidemiological, and statistical knowledge.

Chapter 6

Advanced Topics in Regression

Chapter 5 introduced how to fit an **GLM** for outcomes of different data types. However, there is still one common type of outcome that cannot be handled using **GLM**: **survival outcome**. Survival outcome can be thought of an extension of a binary outcome. But instead of just a yes/no for a disease, it has two pieces of information:

1. The length of follow-up (thus the data have to be generated from a study design with some temporality);
2. Whether the individual had the outcome by the end of follow-up: Yes (the individual had the event [e.g. disease]); No (the individual had not had the event, a.k.a **censored**).

Because of this survival outcome is also sometimes called **time-to-event**.

There are usually three ways where the follow-up end:

1. End of observation: There is no more information about the presence of the event after that. This is the most common way of ending a follow-up.
2. Observation of an event: In majority of scenarios, only the incident (i.e. first) event was of interest, and thus follow-up would end at the incident event, even though the actual observation could be longer.
3. Occurrence of a **competing event**: Briefly, a competing event is one that would inhibit the observation of the main event of interest. A common competing event is death from other causes. For example, if one is interested in the incidence of heart diseases, dying from cancer is a competing event. Handling competing event is actually a very complex topic. In simplest term, if one is only interested in the **relative risk**, they can look at cause-specific hazard ratio (HR), where a competing event will end the follow-up. Please note that this approach, however, will provide a biased estimate for **absolute risk** (e.g. cumulative hazard) (Lau et al. 2009). There is another method to handle competing event ('Fine and Grey subdistribution model'), but is outside the scope of this course.

```
library(tidyverse)
library(gtsummary)
library(interactionR)
library(car)
library(mfp2)
library(mgcv)
library(survival)
library(haven)
```

```r
# Import datasets
alertrct <- haven::read_dta('data/06-alert_rct.dta')
usoc <- readr::read_csv('data/06-understanding_soc.csv') %>%
  mutate(
    obesity=if_else(bmi >= 30, 1, 0),
    bmi_factor=case_when(
      bmi <= 18.5 ~ 'Underweight',
      bmi <= 25 ~ 'Normal weight',
      bmi <= 30 ~ 'Overweight',
      bmi <= 999 ~ 'Obesity',
    ) %>%
      factor() %>%
      relevel('Normal weight')
  )
```

```
## Rows: 11447 Columns: 10
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (2): sex, income
## dbl (8): age, bmi, pw1, mw1, pw5, mw5, pw9, mw9
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

6.1 Survival Analysis

Survival analysis specifically deals with time-to-event data. This is unique because it tracks both the duration of follow-up and the presence of the event at the end (disease/death).

6.1.1 Kaplan-Meier Estimator

The Kaplan-Meier estimator (KM) is a non-parametric method to estimate the survival function, i.e. how survival (or, disease-free, for disease outcomes) probability varies over time.

The code below shows the simplest form of KM - estimating the major adverse cardiovascular event (MACE)-free probability for the all participants in the *Alert RCT*. Note that We use `Surv(time, event)` to create a survival object in R that combines follow-up time and event status. The `~ 1` indicates we are interested in the overall survival function for the entire sample.

Note that when calling the km1 object, it showed the data has 1000 participants with 109 MACE. However, the median and corresponding CI were both NA. That is because to calculate median survival from this non-parametric method, at least 50% of the participants have to had developed the event. There are parametric models that could be used to extraplote beyond the observation, even though those usually would be regarded very reliable.

Calling `summary(km1, times=5)` asked R to produce the 5-year survival probability, which was 0.9.

```r
library(survival)
# Produce survival function
```

```
km1 <- survfit(Surv(mace_t, mace_i) ~ 1, data=alertrct)
km1
```

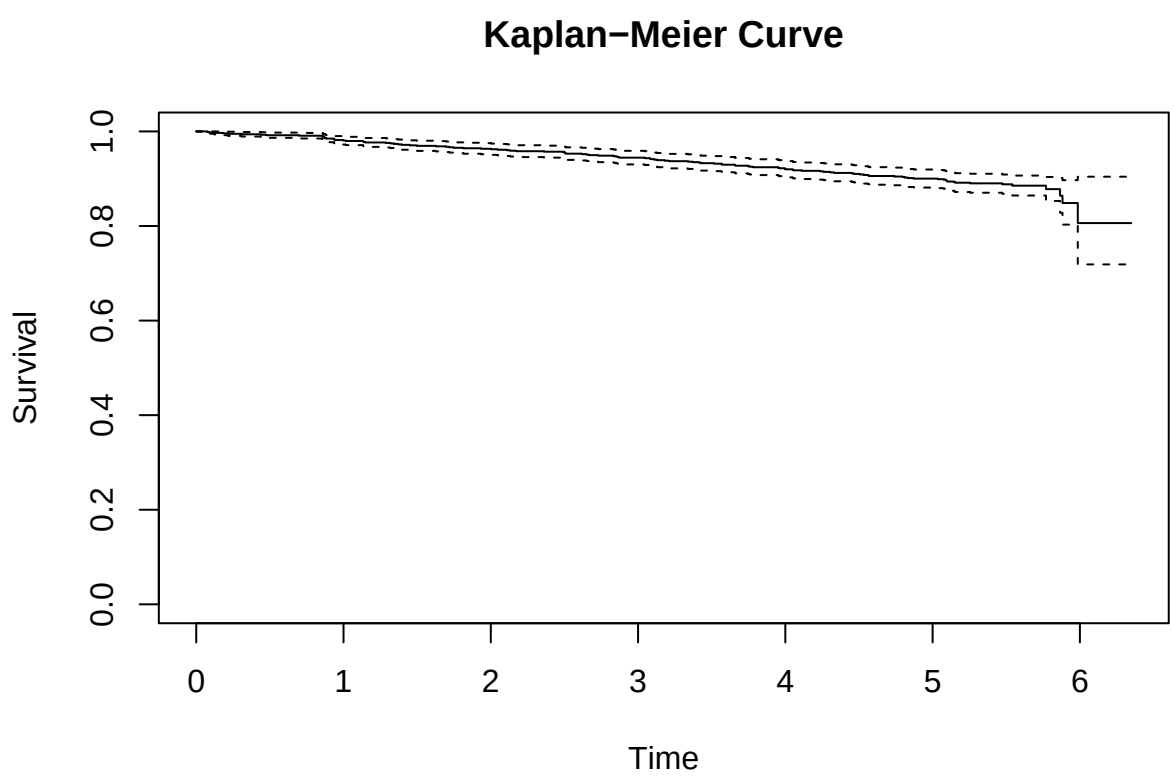
```
## Call: survfit(formula = Surv(mace_t, mace_i) ~ 1, data = alertrct)
##
##         n events median 0.95LCL 0.95UCL
## [1,] 1000    109     NA      NA      NA
```

```
summary(km1, times=5)
```

```
## Call: survfit(formula = Surv(mace_t, mace_i) ~ 1, data = alertrct)
##
##  time n.risk n.event survival std.err lower 95% CI upper 95% CI
##     5    775      95      0.9 0.00973        0.881        0.919
```

Instead of the table, one can also visualise the KM, a Kaplan-Meier curve using the following command.

```
# Plot survival function
plot(km1, main="Kaplan-Meier Curve", xlab="Time", ylab="Survival")
```



The plot showed that the longest follow-up was just over 6 years (as shown in the x-axis), and that by the end of the study there were still over 80% of the participants to be disease-free.

What do you think `summary(km1, times=10)` will show?

6.1.2 Log-rank Test

KM is just an estimator but it could also be a basis to conduct simple comparison, e.g. by subgroup. This is specifically done via the **log-rank test**. The example below examined if the disease-free survival were different by randomised group (fluvastatin vs. placebo).

```
# Standard log-rank test (rho=0)
km2 <- survdiff(Surv(mace_t, mace_i) ~ randgrp, data=alertrct, rho=0)
km2
```

```
## Call:
## survdiff(formula = Surv(mace_t, mace_i) ~ randgrp, data = alertrct,
##     rho = 0)
##
##             N Observed Expected (O-E)^2/E (O-E)^2/V
## randgrp=0 506       60     55.1     0.437     0.884
## randgrp=1 494       49     53.9     0.447     0.884
##
##  Chisq= 0.9  on 1 degrees of freedom, p= 0.3
```

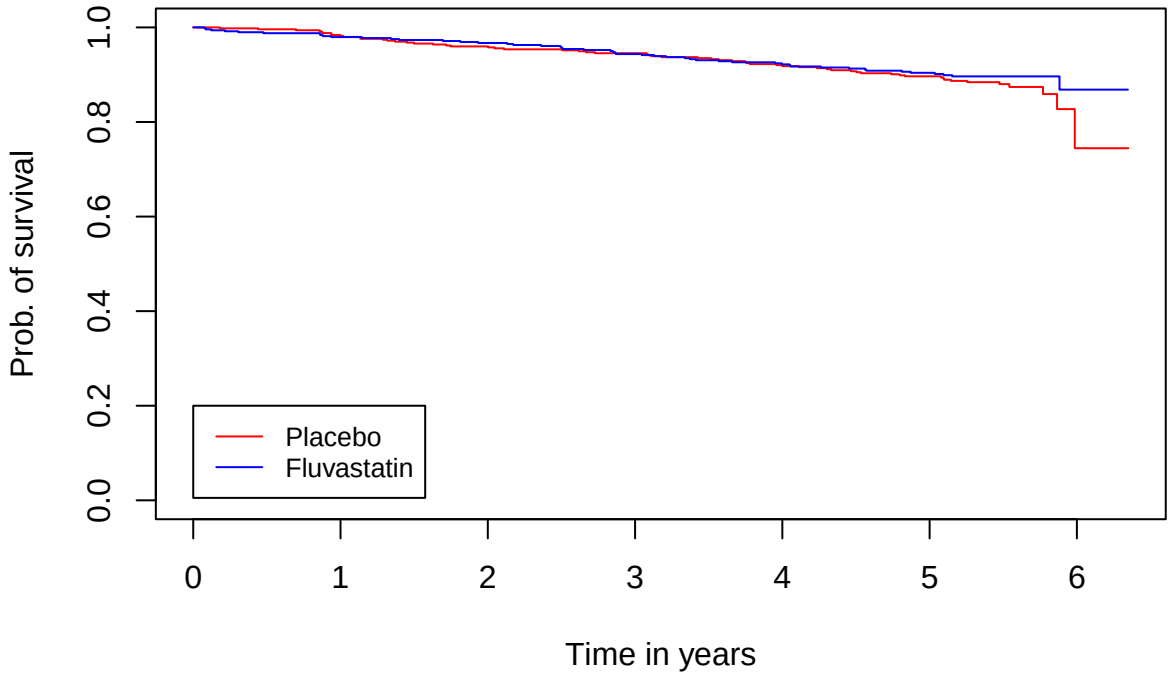
The test p-value is 0.30, above the usual cut-off of statistical significance. But this could very well be due to lower statistical power than actually no difference. Note that in survival analysis, the number of events is the main driver of power.

Do you know why `rho=0` is specified in the code above?

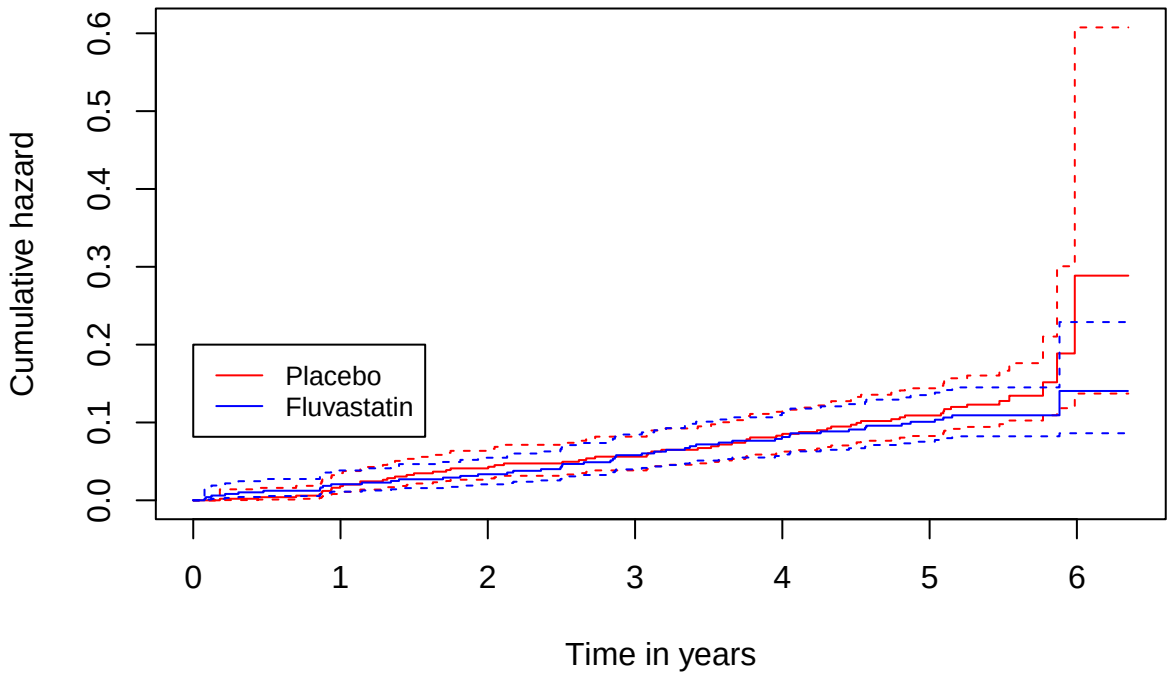
Can you investigate by running `?survfit`?

You can also plot the survival curves or, alternatively, Nelson-Aalan estimator for cumulative hazard by subgroups. Note the differences in arguments in the two plotting functions below. The `conf.int` indicates whether the 95% CI are to be plotted. In the Nelson-Aalan plot because CI were plotted, the syntax also included `lty=c(1, 2, 2, 1, 2, 2)` to ask R to use dashed lines for the CI. The `fun='cumhaz'` asks R to specifically to plot the cumulative hazard. Without that, the default would be the plot the KM estimator for survival.

```
km2 <- survfit(Surv(mace_t, mace_i) ~ randgrp, data=alertrct)
plot(km2, conf.int = FALSE, ylab="Prob. of survival", xlab="Time in years",
     col=c('red', 'blue'))
legend(0, 0.2, legend=c("Placebo", "Fluvastatin"), col=c("red", "blue"),
       lty=1, cex=0.8)
```




```
plot(km2, conf.int = TRUE, ylab="Cumulative hazard", xlab="Time in years",
     col=c('red', 'blue'), lty=c(1, 2, 2, 1, 2, 2), fun="cumhaz")
legend(0, 0.2, legend=c("Placebo", "Fluvastatin"), col=c("red", "blue"),
       lty=1, cex=0.8)
```



Note that hazard is very similar to incidence rate, so cumulative hazard can be thought of as the cumulative incidence rate. It is not equal to the probability of having the disease. It is actually similar to the **cumulative probability** of having the disease, and can be greater than 1 (probabilities are bounded by 0 and 1).

6.1.3 Cox proportional hazard model

The log-rank test can be thought of as the t-test for survival variables in that they shared the same limitations: not being able to quantify the difference, no inherent way to accommodate numeric predictor/exposure, and the lack of ability to adjust for covariates. For numeric, binary, and count outcomes, the GLM provides a solution to these two limitations. Similarly, the Cox proportional hazards (Cox PH) model provides a solution for survival data.

Cox PH model (a.k.a. Cox model or Cox regression) is semi-parametric in that it doesn't assumes the underlying survival / hazard function of the survival outcome (non-parametric part). However, it does assumes how the relative hazard varies by the exposure / predictors (the parametric part).

Below are examples to examine the assocation of Fluvastatin (compared with placebo) with MACE in two adjustment models.

```
cm1 <- coxph(Surv(mace_t, mace_i) ~ randgrp, data=alertrct)
summary(cm1)
```

```
## Call:
## coxph(formula = Surv(mace_t, mace_i) ~ randgrp, data = alertrct)
##
##   n= 1000, number of events= 109
##
```

```
##            coef exp(coef) se(coef)      z Pr(>|z|)
## randgrp -0.1809    0.8345   0.1926 -0.939    0.348
##
##         exp(coef) exp(-coef) lower .95 upper .95
## randgrp    0.8345      1.198    0.5721     1.217
##
## Concordance= 0.514  (se = 0.024 )
## Likelihood ratio test= 0.89  on 1 df,   p=0.3
## Wald test            = 0.88  on 1 df,   p=0.3
## Score (logrank) test = 0.88  on 1 df,   p=0.3
```

```
cm2 <- coxph(Surv(mace_t, mace_i) ~ randgrp + age + female, data=alertrct)
summary(cm2)
```

```
## Call:
## coxph(formula = Surv(mace_t, mace_i) ~ randgrp + age + female,
##     data = alertrct)
##
##   n= 1000, number of events= 109
##
##              coef exp(coef)  se(coef)      z Pr(>|z|)
## randgrp -0.172099  0.841896  0.192871 -0.892    0.372
## age      0.042483  1.043398  0.009388  4.525 6.03e-06 ***
## female  -0.171887  0.842075  0.207171 -0.830    0.407
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##         exp(coef) exp(-coef) lower .95 upper .95
## randgrp    0.8419     1.1878    0.5769     1.229
## age        1.0434     0.9584    1.0244     1.063
## female     0.8421     1.1875    0.5611     1.264
##
## Concordance= 0.62  (se = 0.029 )
## Likelihood ratio test= 22.29  on 3 df,   p=6e-05
## Wald test            = 21.82  on 3 df,   p=7e-05
## Score (logrank) test = 22.34  on 3 df,   p=6e-05
```

The HR shown in the column of `exp(coef)` indicated Fluvastatin was associated with 16% lower hazard of MACE, even though the difference is not statistically significant. Also note that age, after adjusted for sex and Fluvastatin, was associated with MACE, with an HR (95% CI) of 1.04 (1.02-1.06). This suggests that each year of age was associated with 4% high hazard in MACE.

Alternatively, you can also use the `gtsummary::tbl_regression()` function to display the results from a Cox model:

```
tbl_merge(list(tbl_regression(cm1, exponentiate = T),
               tbl_regression(cm2, exponentiate = T)),
          tab_spanner = c('Unadjusted', 'Age and sex adjusted'))
```

```
## The number rows in the tables to be merged do not match, which may result in
```

	Unadjusted			Age and sex adjusted		
Characteristic	**HR**	**95% CI**	**p-value**	**HR**	**95% CI**	**p-value**
randomised group	0.83	0.57, 1.22	0.3	0.84	0.58, 1.23	0.4
age of patient				1.04	1.02, 1.06	<0.001
sex				0.84	0.56, 1.26	0.4

Abbreviations: CI = Confidence Interval, HR = Hazard Ratio

```
## rows appearing out of order.
## i See `tbl_merge()` (`?gtsummary::tbl_merge()`) help file for details. Use
##   `quiet=TRUE` to silence message.
```

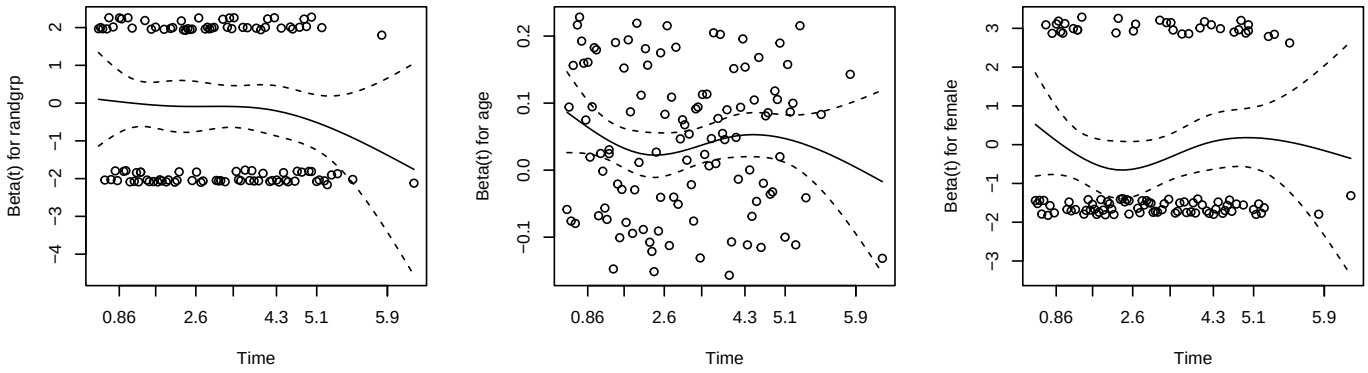
6.1.3.1 Proportional hazard assumption

One of the main assumption behind Cox model is the proportional hazard assumption. The proportional hazards assumption requires that the HR remains constant over the entire follow-up.

For instance, if the placebo group is twice as likely to experience MACE **only** at year 6 (something supported by the KM plot, albeit with CI overlaps), the assumption is violated.

To formally examine this assumption, one can look at the Schoenfeld residuals with the syntax below.

```
par(mfrow=c(1, 3))
plot(cox.zph(cm2))
```




```
cox.zph(cm2)
```

```
##         chisq df    p
## randgrp 1.297  1 0.25
## age     0.201  1 0.65
## female  0.164  1 0.69
## GLOBAL  1.639  3 0.65
```

The first two lines plot the Schoenfeld residual for each included exposure/predictor in a 1x3 (1 row; 3 columns) layout. The third line calculate the p-values of a time-varying HR (i.e. HR varies over time). However even though the p-value for `randgrp` is 0.25, the plot did show a tendency for HR to be lower towards the end of follow-up.

When the HR is indeed time-varying, the HR estimated from Cox model becomes a weighted average of the time-varying HR, with the weight being the number of disease-free participants and number of events at a particular time. Some people argue that test for proportional hazard is unnecessary because it is almost always violated to some extent (Stensrud & Hernán 2020). However, there are actually more advanced methods to model time-varying HR, which is outside the scope of this course.

Please use the hip fracture data and examine if hip fracture hazard was associated with `protect` after adjusting for age and calcium.

6.2 Interaction and Effect Modification

Interaction and **effect modification** were often used interchangeably in public health and medical literature, but, in fact, they refer to two distinct (but close related) concepts (VanderWeele 2009). The confusion is partly stemmed from terminology. 'Interaction' can refer to both 'causal interaction' and 'statistical interaction' and it is often not made which was referred to explicitly.

Effect modification involves a single exposure/intervention. It asks whether the causal effect of one primary exposure naturally varies across some population subgroups (e.g. biological sex or baseline employment). Because we are only assessing the effect of the primary exposure, we only need to adjust for the confounders of that specific exposure-outcome relationship.

On the other hand, a (causal) interaction is one that assumes two exposures/interventions to have causal effect on the outcome, and whether the two have synergistic ($1 + 1 > 2$) or antagonistic ($1 + 1 < 2$) effects. For example, prescribing both a weight-loss drug and an exercise programme to a patient with severe obesity could have better weight loss effect (e.g. BMI reduction = 20%) than the combination of individual intervention (e.g. BMI reduction = 5% in each intervention). Because we are making a causal claim about two simultaneous interventions, claiming causal interaction requires a much higher methodological bar: we must rigorously adjust for the confounders of both exposures. While subgroup analysis can be used to examine (causal) interaction, a more straightforward way is to examine the joint exposure variable. In the example, it would be a 4-level categorical variable: no interventions; weight-loss drug only; exercise programme only; both interventions, as this allows the examination of causal effects of both interventions as well as any synergistic/antagonistic effects. The way to quantify such synergistic/antagonistic effects is by examining **statistical interaction** (e.g. a product term in a regression model, or calculating the relative excess risk due to additive interaction (RERI)).

When researchers observe different effect estimates across subgroups (e.g., a drug appearing more effective in women than in men) - an example of effect modification - they must first determine if this difference is mathematically real or simply random sampling noise. To do this, they also use tests of **statistical interaction**. However, while the mathematical test is the same, we should understand the conceptual difference between **effect modification** and **causal interaction** based on the causal assumption.

Below is a summary table to capture the distinction and statistical approach between the two:

The table below summarizes the key differences between the two causal concepts and how they dictate our analytical approach in R, even though both ultimately rely on the exact same statistical tests to confirm their findings.

Concept	Primary Goal	Analytical Approach	Statistical Confirmation
Effect Modification	Assess if the effect of *one* intervention varies across existing groups.	**Subgroup analysis:** Calculating estimates within specific subgroup.	Statistical interaction test (e.g., product term in model, RERI)
Causal Interaction	Assess the joint, synergistic effect of *two* distinct interventions.	**Joint group analysis:** Comparing all combinations of exposures against a single reference group.	Statistical interaction test (e.g., product term in model, RERI)

Characteristic	Beta	95% CI	p-value
(Intercept)	23	22, 24	<0.001
mw1	0.41	0.39, 0.43	<0.001
age	0.12	0.11, 0.13	<0.001
sex			
ăăăăFemale			
ăăăăMale	0.49	0.14, 0.83	0.005
income			
ăăăăHigher income			
ăăăăLower income	-1.3	-1.6, -0.93	<0.001
obesity	-1.3	-1.7, -0.89	<0.001

Abbreviation: CI = Confidence Interval

Characteristic	Beta	95% CI	p-value
(Intercept)	23	22, 24	<0.001
mw1	0.41	0.39, 0.43	<0.001
age	0.12	0.11, 0.13	<0.001
sex			
ăăăăFemale			
ăăăăMale	0.48	0.14, 0.83	0.006
income			
ăăăăHigher income			
ăăăăLower income	-1.1	-1.5, -0.70	<0.001
obesity	-0.81	-1.4, -0.22	0.007
income * obesity			
ăăăăLower income * obesity	-0.93	-1.7, -0.12	0.024

Abbreviation: CI = Confidence Interval

Below is an example to examine effect modification using the longitudinal data from the Understanding Society study (Buck & McFall 2012). Assume we are interested in estimating the potentially causal association between obesity (binary) with mental well-being (MW) at Wave 9 (continuous), adjusted for MW at Wave 1, age, sex, and income. The syntax for linear regression model below should be quite familiar now:

```
mod1 <- lm(mw9 ~ mw1 + age + sex + income + obesity, data=usoc)
mod1 |> tbl_regression(intercept = T)
```

To examine if the association of obesity differs based on a person's income level, we include an (statistical) interaction term between income and obesity. In R, the * operator computes both the main effects and the interaction term simultaneously.

```
mod2 <- lm(mw9 ~ mw1 + age + sex + income * obesity, data=usoc)
mod2 |> tbl_regression(intercept = T)
```

Exercise: Can you use the same data to examine if the association of age with MW differ by sex. Can you interpret the interaction term?

Characteristic	OR	95% CI	p-value
(Intercept)	0.55	0.46, 0.66	<0.001
age	0.97	0.97, 0.98	<0.001
sex			
ăăăăFemale			
ăăăăMale	0.73	0.66, 0.82	<0.001
income			
ăăăăHigher income			
ăăăăLower income	1.47	1.31, 1.66	<0.001
obesity	1.59	1.33, 1.90	<0.001
income * obesity			
ăăăăLower income * obesity	1.09	0.86, 1.38	0.5

Abbreviations: CI = Confidence Interval, OR = Odds Ratio

6.2.1 Additive Interaction

In GLM, an statistical interaction term can have different meaning depends entirely on the **link function** used in the model.

For a linear regression (which uses an **identity** link), the effects of predictors are naturally summed together. Therefore, an interaction term in a linear model tests for a departure from strict additivity (e.g. 1 + 1 > 2). However, when dealing with binary, count, or time-to-event outcomes, we typically use models like logistic regression (which uses a **logit** link), Poisson / negative binomial (NB), or Cox proportional hazards (which use a **log** link). The primary reason these models are fitted with logit or log link functions is to ensure the model's predictions respect the natural mathematical boundaries of the outcome data. For example, a logit link forces all predicted probabilities to fall strictly between 0 and 1, while a log link ensures that predicted rates or hazards are always positive. If we were to use a standard linear (identity) link for a binary outcome, the model could predict mathematically impossible values, such as a negative probability of disease. However, as a side effect, the exposures' effect are then assumed to be multiplied (rather than added) with a logik and log link function. As a result, the statistical interaction product term in these GLMs tests for a departure from a **multiplicative** relationship (e.g. 2 * 2 > 4).

Additive interaction is sometimes thought to be more relevant and preferred for public health decision-making. People intuitively assume the absolute risk or health burden of two independent exposures should simply add together. If the combined absolute risk is substantially larger than the sum of their individual excess risks, there is a synergistic additive interaction. This absolute scale is crucial for policymakers because it directly translates to the number of cases that could be prevented by intervening on specific target groups.

To illustrate how to test for additive interaction when the underlying model's link function is multiplicative, we will artificially create a binary 'poorer mental wellbeing' variable, defined as 1-SD below the population average. Suppose we want to examine whether obesity and lower income have an additive interaction on the risk of having poorer mental wellbeing.

```
usoc <- usoc |>
  mutate(pmw9=if_else(mw9 < 40, 1, 0))
```

To estimate RERI, the most commonly used measure for additive interaction, we will need to have a model with both main effects and the multiplicative interaction term.

```
mod4 <- glm(pmw9 ~ age + sex + income * obesity, data=usoc, family=binomial)
mod4 |> tbl_regression(intercept = T, exponentiate = T)
```

We can then call the `interactionR::interactionR()` function on the model. Note that to estimate RERI we need both 'main effects' to be risk factors, i.e. associated with higher risk of outcome (OR > 1 in this example). This is a technical limitation of RERI. The RERI estimated would be erronuous if one (or both) main effects are in the protective direction (e.g. OR < 1).

```
library(interactionR)
reri <- interactionR(mod4, exposure_names=c('income', 'obesity'))
reri$dframe
```

```
##                                         Measures Estimates      CI.ll    CI.ul
## 1                                             OR00 1.0000000         NA       NA
## 2                                             OR01 1.5932568 1.33345435 1.903678
## 3                                             OR10 1.4732500 1.30601832 1.661895
## 4                                             OR11 2.5568260 2.17868246 3.000602
## 5 OR(obesity on outcome [incomeLower income==0] 1.5932568 1.33345435 1.903678
## 6 OR(obesity on outcome [incomeLower income==1] 1.7355004 1.48990510 2.021579
## 7                           Multiplicative scale 1.0892785 0.86193451 1.376587
## 8                                             RERI 0.4903191 0.05562521 0.925013
##         p
## 1      NA
## 2 0.00000
## 3 0.00000
## 4 0.00000
## 5 0.00000
## 6 0.00000
## 7 0.47400
## 8 0.01353
```

In our example this isn't a problem since the two main effects (obesity and lower income) are already risk factors so we don't need to recode. The results show that having obesity and lower income simultaneously was associated with 50.2% (RERI=0.5019) higher risk, in addition to the sum of the excess risk due to obesity and lower income individually. This can be verified with hand calculation from the first 4 lines in the output:

$$RERI = (2.529\ 1)\ (1.589\ 1)\ (1.438\ 1) = 0.502$$

.

6.2.1.1 Recoding for calculating RERI

As an example, examine the RERI of older age (a protective factor) and male with poorer mental wellbeing. `interactionR()` would complains about that with an error. But we can specify `recode=T` to ask the function to recode older variable to become 'younger'. When we use the `interactionR()`'s recode argument, we will need to make sure both variables (older and male) are coded as 0 and 1.

The warning message indicates the new reference values after recoding. The findings shown are for older==0 compared to older==1 (i.e. younger, compared to older), and for male==0 compared to male==1 (i.e. female compared to older). Therefore, the RERI indicates that being a younger female was associated with disproportionately higher risk of poorer mental wellbeing, with an excess risk of 66.3% (RERI==0.6628).

```
usoc <- usoc |>
  mutate(
```

Characteristic	OR	95% CI	p-value
(Intercept)	0.19	0.17, 0.21	<0.001
older	0.38	0.32, 0.45	<0.001
male	0.71	0.63, 0.80	<0.001
income			
ăăăăHigher income			
ăăăăLower income	1.59	1.43, 1.76	<0.001
obesity	1.60	1.43, 1.80	<0.001
older * male	1.19	0.91, 1.55	0.2

Abbreviations: CI = Confidence Interval, OR = Odds Ratio

```
    older=if_else(age >= 60, 1, 0),
    male=if_else(sex=='Male', 1, 0))

mod5 <- glm(pmw9 ~ older * male + income + obesity, data=usoc, family=binomial)
mod5 |> tbl_regression(intercept = T, exponentiate = T)
```

```
interactionR(mod5, exposure_names=c('older', 'male'), recode=T)$dframe
```

```
## Warning in interactionR(mod5, exposure_names = c("older", "male"), recode = T):
## Recoding exposures; new reference category for older is 1 and for male is 1

##                         Measures Estimates     CI.ll    CI.ul       p
## 1                           OR00  1.000000        NA       NA      NA
## 2                           OR01  1.183831 0.9301038 1.506773 0.17031
## 3                           OR10  2.209131 1.7918866 2.723532 0.00000
## 4                           OR11  3.103290 2.5467919 3.781388 0.00000
## 5 OR(male on outcome [older==0]  1.183831 0.9301038 1.506773 0.17031
## 6 OR(male on outcome [older==1]  1.404756 1.2461765 1.583515 0.00000
## 7          Multiplicative scale  1.186619 0.9087170 1.549508 0.20881
## 8                           RERI  0.710328 0.3557683 1.064888 0.00004
```

6.3 Multicollinearity

Multicollinearity occurs when two or more independent variables in a regression model are highly correlated with each other. This overlap in variance (formally known as **covariance**) makes it extremely difficult for the model to isolate the individual effect of each independent variable, leading to artificially inflated standard errors and unstable coefficients.

6.3.1 Model with collinearity

The model below erroneously included both BMI and BMI categories, making the interpretation difficult.

```
mod_collin <- lm(mw9 ~ age + sex + income + bmi_factor + bmi, data=usoc)
mod_collin |> tbl_regression(intercept = T)
```

Characteristic	Beta	95% CI	p-value
(Intercept)	47	45, 48	<0.001
age	0.16	0.15, 0.17	<0.001
sex			
ăăăăFemale			
ăăăăMale	1.1	0.73, 1.5	<0.001
income			
ăăăăHigher income			
ăăăăLower income	-1.6	-2.0, -1.3	<0.001
bmi_factor			
ăăăăNormal weight			
ăăăăObesity	0.09	-0.91, 1.1	0.9
ăăăăOverweight	0.47	-0.07, 1.0	0.086
ăăăăUnderweight	-2.6	-8.1, 2.9	0.4
bmi	-0.17	-0.25, -0.10	<0.001

Abbreviation: CI = Confidence Interval

6.3.2 Correlation coefficient

Simple correlation test showed very strong correlation between the two. Note that the code below have converted BMI categories from `factor` into `numeric` so that the ordinal nature is preserved.

```
with(usoc, cor.test(bmi, as.numeric(bmi_factor), method='spearman'))
```

```
## Warning in cor.test.default(bmi, as.numeric(bmi_factor), method = "spearman"):
## Cannot compute exact p-value with ties

##
##  Spearman's rank correlation rho
##
## data:  bmi and as.numeric(bmi_factor)
## S = 9.0116e+10, p-value < 2.2e-16
## alternative hypothesis: true rho is not equal to 0
## sample estimates:
##       rho
## 0.6395212
```

6.3.3 Variance Inflation Factor

There is a measure to quantify multicollinearity - variance inflation factor (VIF), which can be calculated using the `car::vif()` function The VIF is not very large, even though the obvious concept problem in including both BMI categories and BMI. Generally, a VIF above 5 or 10 warrants close inspection.

```
vif(mod_collin)
```

```
##                GVIF Df GVIF^(1/(2*Df))
## age        1.044667  1        1.022090
## sex        1.086370  1        1.042291
```

Characteristic	Beta	95% CI	p-value
(Intercept)	43	42, 43	<0.001
age	0.16	0.15, 0.17	<0.001
sex			
ăăăăFemale			
ăăăăMale	1.1	0.71, 1.5	<0.001
income			
ăăăăHigher income			
ăăăăLower income	-1.6	-2.0, -1.3	<0.001
bmi_factor			
ăăăăNormal weight			
ăăăăObesity	-2.0	-2.4, -1.5	<0.001
ăăăăOverweight	-0.35	-0.75, 0.05	0.089
ăăăăUnderweight	-1.9	-7.4, 3.6	0.5

Abbreviation: CI = Confidence Interval

```
## income     1.092281  1        1.045123
## bmi_factor 4.517411  3        1.285726
## bmi        4.435897  1        2.106157
```

6.3.4 Model without collinearity

Removing BMI numeric variable could solve the problem easily.

```
mod_clean <- lm(mw9 ~ age + sex + income + bmi_factor, data=usoc)
mod_clean |> tbl_regression(intercept = T)
```

6.4 Non-linearity

Standard regression models typically assume a straightforward linear relationship between continuous expsures/predictors and the outcome. However, real-world biological and epidemiological data frequently exhibit non-linear trends, such as U-shaped or J-shaped curves, and are different ways to capture these non-linear associations (Ho & Cole 2023).

6.4.1 Model with categorical variable

One conventional method to bypass non-linearity is to categorise the continuous variable, e.g. by using the BMI category variable.

```
mod_cat <- lm(mw9 ~ age + sex + income + bmi_factor, data=usoc)
mod_cat |> tbl_regression(intercept = T)
```

This is the conventional method because it is straightforward to implement, but the problem is that it is also crude. This essentially assumes people within each category to have the same outcome, on average. Another problem is that many times the categorisation is arbitrary and could affect the conclusion.

Characteristic	Beta	95% CI	p-value
(Intercept)	43	42, 43	<0.001
age	0.16	0.15, 0.17	<0.001
sex			
ăăăăFemale			
ăăăăMale	1.1	0.71, 1.5	<0.001
income			
ăăăăHigher income			
ăăăăLower income	-1.6	-2.0, -1.3	<0.001
bmi_factor			
ăăăăNormal weight			
ăăăăObesity	-2.0	-2.4, -1.5	<0.001
ăăăăOverweight	-0.35	-0.75, 0.05	0.089
ăăăăUnderweight	-1.9	-7.4, 3.6	0.5

Abbreviation: CI = Confidence Interval

Characteristic	Beta	95% CI	p-value
(Intercept)	44.25	40.93, 47.56	<0.001
age	0.1598	0.1477, 0.1720	<0.001
sex			
ăăăăFemale			
ăăăăMale	1.122	0.7489, 1.494	<0.001
income			
ăăăăHigher income			
ăăăăLower income	-1.650	-2.020, -1.279	<0.001
bmi	-0.0048	-0.2295, 0.2199	>0.9
I(bmi^2)	-0.0027	-0.0063, 0.0010	0.2

Abbreviation: CI = Confidence Interval

6.4.2 Model with quadratic term

Another way would be to include mathematically function of the exposure/predictor, such as the quadratic form of BMI. Note that in the syntax below, the coefficients are rounded to nearest 4 significant figures, mainly because the coefficient for the quadratic form of BMI was very small.

```
mod_quad <- lm(mw9 ~ age + sex + income + bmi + I(bmi^2), data=usoc)
mod_quad |> tbl_regression(intercept = T, estimate_fun=function(.x) style_sigfig(.x, digits=4))
```

6.4.3 Model with fractional polynomials

A more systematic approach to model nonlinearity using polynomial is by using The fractional polynomials. `mfp2::mfp2()` function provide easy estimation of fractional polynomials. For `age`, `sex`, and `income` I have included `fp(xxx, 1)` to indicate we don't need to transform any of those variables. `1` indicates just use it as is. We didn't specify any numbers in `fp(bmi)` so that the function will estimate what powers to use. The `verbose==FALSE` suppress the technical reports. You can write down the explicit formula based on the fractional polynomials. Here:

$$MW = 49.902 + 1.539 \text{ Œ } age\ 80.371 \text{ Œ } BMI^{2} + 73.188 \text{ Œ } BMI^{1}\ 1.541 \text{ Œ } income + 1.187 \text{ Œ } sex$$

The `fracplot()` function plots the estimated curve. `partial_only = T` suppresses the residuals / actual data points. `type = 'contrasts'` plots the coefficients (differences in MW) rather than the actual MW.

```
mod_fp <- mfp2(mw9 ~ fp(age, 1) + fp(sex, 1) + fp(income, 1) + fp(bmi), data=usoc, verbose=FALSE)
print(mod_fp)
```

```
## Shifting, Scaling and Centering of covariates
##        shift scale center
## age        0    10   TRUE
## bmi        0    10   TRUE
## income     0     1   TRUE
## sex        0     1   TRUE
##
## Final Multivariable Fractional Polynomial for y
##        df_initial select alpha selected df_final power1 power2
## age             1   0.05  0.05     TRUE        1      1     NA
## bmi             4   0.05  0.05     TRUE        4     -2     -2
## income          1   0.05  0.05     TRUE        1      1     NA
## sex             1   0.05  0.05     TRUE        1      1     NA
##
## MFP algorithm convergence: TRUE
##
## Call:  glm(formula = y ~ ., family = family, data = data, weights = weights,
##     offset = offset, x = TRUE, y = TRUE)
##
## Coefficients:
## (Intercept)        age.1        bmi.1        bmi.2     income.1        sex.1
##   5.002e+01    1.583e-01   -2.620e+04    1.007e+04   -1.625e+00    1.092e+00
##
## Degrees of Freedom: 11446 Total (i.e. Null);  11441 Residual
## Null Deviance:       1150000
## Residual Deviance: 1071000   AIC: 84450
```

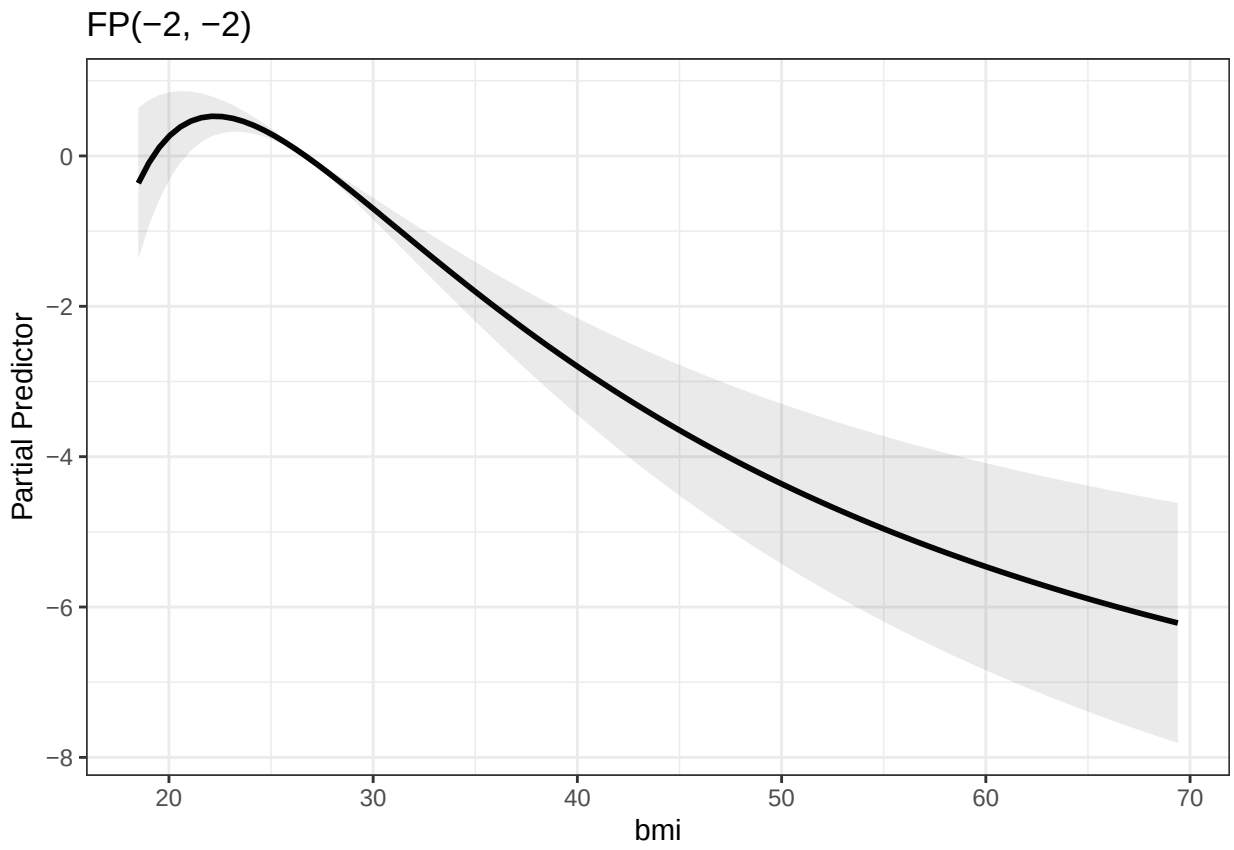
```
summary(mod_fp)
```

```
##
## Call:
## glm(formula = y ~ ., family = family, data = data, weights = weights,
##     offset = offset, x = TRUE, y = TRUE)
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  5.002e+01  1.661e-01 301.136  < 2e-16 ***
## age.1        1.583e-01  6.229e-03  25.411  < 2e-16 ***
```

```
## bmi.1       -2.620e+04  4.772e+03  -5.490 4.11e-08 ***
## bmi.2        1.007e+04  1.739e+03   5.787 7.34e-09 ***
## income.1    -1.625e+00  1.892e-01  -8.592  < 2e-16 ***
## sex.1        1.092e+00  1.904e-01   5.735 9.98e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for gaussian family taken to be 93.56991)
##
##     Null deviance: 1149883  on 11446  degrees of freedom
## Residual deviance: 1070533  on 11441  degrees of freedom
## AIC: 84448
##
## Number of Fisher Scoring iterations: 2
```

```
fracplot(mod_fp, terms = 'bmi', partial_only = T, type = 'contrasts')
```

```
## $bmi
```



The fractional polynomial curves suggest a sharp drop in MW in underweight and obesity range.

6.4.4 Model with spline

It should be noted that fractional polynomials are 'global' in the sense that the polynomial selected model the whole range of the exposure/predictor. One potential issue is that it is more vulnerable to the influence of outliers at the extreme end of the exposure/predictor.

Characteristic	Beta	95% CI	p-value
age	0.16	0.15, 0.17	<0.001
sex			
ăăăăFemale			
ăăăăMale	1.1	0.73, 1.5	<0.001
income			
ăăăăHigher income			
ăăăăLower income	-1.6	-2.0, -1.3	<0.001
s(bmi)			<0.001

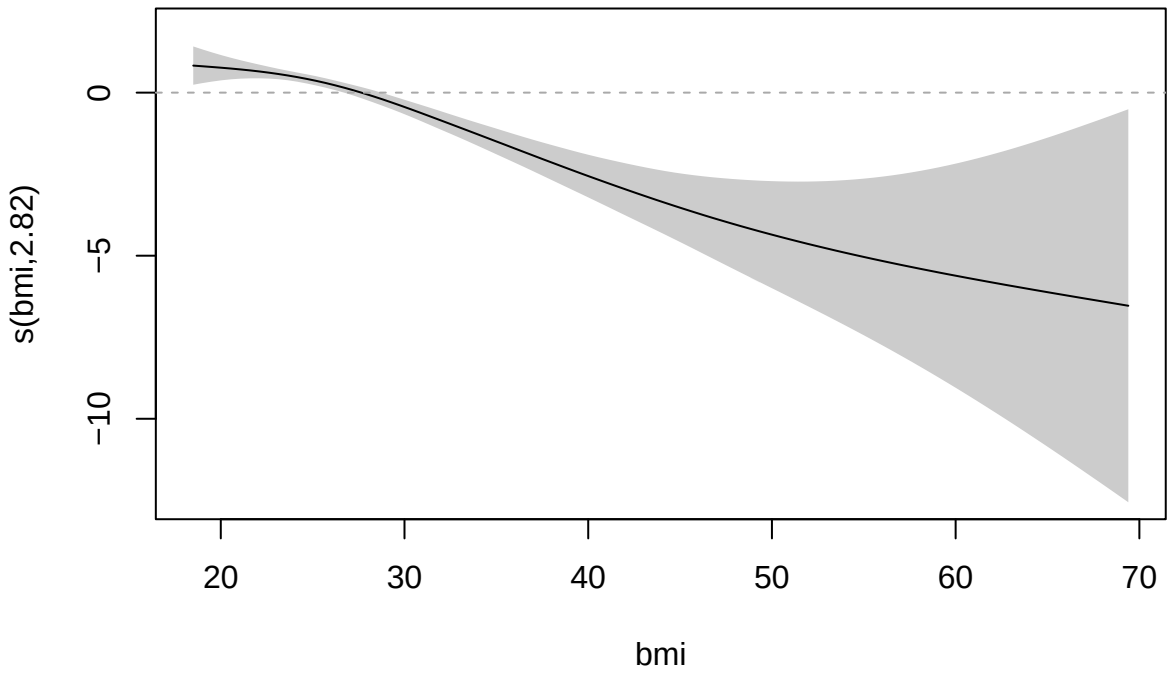
Abbreviation: CI = Confidence Interval

Instead, we can fit regression splines, which, in simplest term, are just smoothed piecewise polynomials. The `mgcv::gam()` function fits generalised additive model (GAM), including both 'normal' predictors, and splines. GAM can be thought of as an extension of the GLM by including splines. The `s(., bs='cs')` term in the regression formula fits a restricted cubic spline. Note that there is another package called `gam` which is NOT what we use here.

Plotting the `gam` object shows the spline. `abline()` was used to add a reference line. The results are consistent at the high BMI range, but different at the underweight range. Usually splines describe the curves better but the drawback is it doesn't provide any explicit formula. Here combining both results from fractional polynomials and splines, it showed we aren't too sure about the results in the underweight range, possibly due to the lack of data points.

```
mod_spline <- gam(mw9 ~ age + sex + income + s(bmi, bs='cs'), data=usoc)
mod_spline |> tbl_regression()
```

```
plot(mod_spline, scheme = 1, ylim=c(-12.5, 2))
abline(h=0, lty=2, col='darkgrey')
```


s(bmi,2.82)
0
−5
−10
20
30
40
50
60
70
bmi

Chapter 7

Causal Inference: Mediation and Instrumental Variables

In previous chapters, we focused on establishing associations between variables. However, in health data science, we often want to understand the *causal* mechanisms driving these associations. This chapter introduces two advanced causal inference techniques: **Mediation Analysis** (to understand *how* an exposure affects an outcome) and **Instrumental Variables** (to estimate causal effects in the presence of unmeasured confounding).

```
devtools::install_github("BS1125/CMAverse")
```

```
# Load required libraries for this chapter
library(tidyverse)
library(gtsummary)
library(bda)
library(AER)
library(CMAverse)

# Note: We will simulate the data structures used in the examples below for rendering purposes.
```

7.1 Data Preparation for Causal Modeling

When investigating causal pathways, the timing of measurements is critical. An exposure must precede a mediator, which in turn must precede the outcome. Therefore, longitudinal data is typically required.

Rationale & Code Explanation: To prepare our data for regression and mediation, we often need to transform it from a "long" format (multiple rows per patient for different time points/waves) to a "wide" format (one row per patient, with variables denoting different time points, e.g., `bmi_1`, `bmi_5`). In the code below, we use `pivot_wider()` from the `tidyr` package to achieve this. We then select our variables of interest (age, sex, income, **BMI**, and mental/physical wellbeing scores at different waves) and drop cases with missing data using `na.omit()`.

```
library(tidyverse)
library(gtsummary)
```

```r
# 1. Raw data import and reshaping
dat <- read_tsv('../UKDA-8715-tab/tab/longitudinal_td.tab') %>%
  select(pidp, indscus_lw_9, strata, psu,
         wave, age_dv, sex_dv, fimngrs_dv,
         bmi_dv, sf12pcs_dv, sf12mcs_dv) %>%
  filter(wave %in% c(1, 5, 9)) %>%
  pivot_wider(id_cols=c(pidp, indscus_lw_9, strata, psu),
              names_from=wave,
              values_from=age_dv:sf12mcs_dv) |>
  select(-c(age_dv_5, age_dv_9, sex_dv_5, sex_dv_9,
            fimngrs_dv_5, fimngrs_dv_9, bmi_dv_5, bmi_dv_9)) |>
  na.omit()

# 2. Data formatting
dat <- dat %>%
  transmute(
    age = age_dv_1,
    sex = factor(sex_dv_1, labels=c('Male', 'Female')),
    income = if_else(fimngrs_dv_1 > median(fimngrs_dv_1),
                     'Higher income', 'Lower income'),
    bmi = bmi_dv_1,
    pw1 = sf12pcs_dv_1, mw1 = sf12mcs_dv_1, # Physical/Mental wellbeing Wave 1
    pw5 = sf12pcs_dv_5, mw5 = sf12mcs_dv_5, # Wave 5
    pw9 = sf12pcs_dv_9, mw9 = sf12mcs_dv_9  # Wave 9
  ) |>
  filter(bmi >= 18.5)

# Optional: Save cleaned data
# write_csv(dat, '../Data/W7_USoc.csv')
```

Rationale: Before running complex models, it is essential to inspect the descriptive statistics. The `tbl_summary()` function generates a clean summary table. We use the `statistic` argument to specify that we want the Mean and Standard Deviation (**SD**) for all continuous variables, and the `digits` argument to round the results neatly.

```r
# Generate descriptive statistics table
tbl_summary(
  dat,
  statistic = list(all_continuous() ~ '{mean} ({sd})'),
  digits = list(all_continuous() ~ c(1, 1),
                all_categorical() ~ c(0, 1))
)
```

7.2 Mediation Analysis

Mediation analysis attempts to unpack the **Total Effect** of an exposure on an outcome into two parts: 1. **Direct Effect:** The effect of the exposure directly on the outcome. 2. **Indirect Effect:** The effect of the exposure on the outcome that passes *through* a third

variable (the mediator).

7.2.1 The Base Model (Total Effect)

Rationale: Before looking for a mediator, we establish the baseline association. Here, we model mental wellbeing at Wave 9 (`mw9`) based on baseline **BMI** (`bmi`), adjusting for demographic confounders and baseline wellbeing.

```
# Base model predicting Wave 9 mental wellbeing
m0 <- lm(mw9 ~ bmi + age + sex + income + mw1 + pw1, data=dat)
tbl_regression(m0)
```

7.2.2 Regression Adjustment for Mediation

Rationale: The simplest way to check for mediation is the "difference method". We add our candidate mediator (Physical Wellbeing at Wave 5, `pw5`) into the regression model. If physical wellbeing mediates the relationship between **BMI** and mental wellbeing, the coefficient for **BMI** should drop noticeably in the new model compared to the base model.

Code Explanation: We fit a second linear model (`m1`) that includes `pw5`. We then use `tbl_merge()` from the `gtsummary` package to put both models side-by-side for easy visual comparison. Minimal differences between the **BMI** coefficients across the two models suggest minimal mediation.

```
# Adjusted model including the mediator (pw5)
m1 <- lm(mw9 ~ bmi + age + sex + income + pw5 + mw1 + pw1, data=dat)

# Merge tables for side-by-side comparison
tbl_merge(
  list(tbl_regression(m0), tbl_regression(m1)),
  tab_spanner = c('Not adjusted for PW', 'Adjusted for PW')
)
```

7.2.3 The Sobel Test

Rationale: The Sobel test is a traditional statistical test to determine if the indirect effect is significantly different from zero.

Code Explanation: We use `mediation.test(mediator, exposure, outcome)` from the `bda` package. *Warning:* This test is rudimentary and does not adjust for any confounders. It is strictly not recommended for observational health data unless you are absolutely certain no confounding exists.

```
library(bda)
# Sobel test (Unadjusted)
with(dat, mediation.test(pw5, bmi, mw9))
```

7.2.4 The Parametric G-Formula

Modern causal inference relies on more robust methods, such as the parametric G-formula, which can handle complex confounding scenarios (including interactions and non-linearities). In R, we use the `cmest()` function from the `CMAverse` package.

7.2.4.1 Without Exposure-Induced Confounders

Rationale: We define our causal pathways explicitly. The total effect (TE) is calculated by comparing an 'active' **BMI** (`a = 26.69`, the sample mean) to a 'reference' **BMI** (`astar = 25`).

Code Explanation: * `basec`: Lists the baseline confounders. * `mreg` and `yreg`: Specify the regression types for the mediator and outcome models (e.g., 'linear' or 'logistic'). * `mval`: Indicates a reference value for the mediator. * `EMint=FALSE`: Specifies that we are not evaluating exposure-mediator interaction in this step.

```
library(CMAverse)
# G-formula mediation analysis
mm1 <- cmest(
  data = dat,
  model = 'gformula',
  exposure = 'bmi',
  mediator = 'pw5',
  outcome = 'mw9',
  basec = c('age', 'sex', 'income', 'mw1', 'pw1'),
  a = 26.69,
  astar = 25,
  mreg = list('linear'),
  mval = list(50),
  yreg = 'linear',
  EMint = FALSE
)

summary(mm1)
```

7.2.4.2 With Exposure-Induced Confounders

Rationale: In longitudinal data, an exposure (**BMI** at Wave 1) might cause a change in a confounding variable (Mental Wellbeing at Wave 5), which then affects both the mediator and the final outcome. Traditional regression adjustment utterly fails in this scenario, but the G-formula can handle it.

Code Explanation: We add the `postc` argument to specify the intermediate (post-exposure) confounder (`mw5`). We also must tell R how to model this intermediate variable using the `postcreg` argument (here, as a 'linear' regression).

```
# G-formula handling exposure-induced confounding (mw5)
mm2 <- cmest(
  data = dat,
  model = 'gformula',
  exposure = 'bmi',
  mediator = 'pw5',
  outcome = 'mw9',
  basec = c('age', 'sex', 'income', 'mw1', 'pw1'),
  postc = c('mw5'),       # The exposure-induced confounder
  a = 26.69,
  astar = 25,
  mreg = list('linear'),
```

```
  mval = list(50),
  postcreg = list('linear'), # Regression model for the confounder
  yreg = 'linear',
  EMint = FALSE
)

summary(mm2)
```

Interpretation Check: If the Indirect Effect (IE) becomes significant and the mediation proportion jumps (e.g., to 34%) only after adjusting for `postc`, it implies that ignoring intermediate confounding severely underestimated the mediating role of physical wellbeing.

7.3 Instrumental Variables (IV)

In observational data, or in clinical trials with non-compliance, unmeasured confounding can severely bias our effect estimates. Instrumental Variable (IV) analysis offers a workaround. An "instrument" is a variable that is strongly associated with the exposure, but has *no direct pathway* to the outcome except through the exposure itself.

In a Randomized Controlled Trial (RCT) with poor adherence, the randomized `group` assignment is the perfect instrument: it dictates whether someone *should* exercise, but only the *actual attendance* impacts their stress levels.

7.3.1 Intention-to-Treat (ITT) vs. Per-Protocol (PP)

Rationale: 1. **ITT Analysis:** We analyze patients based purely on the group they were randomized to, regardless of whether they actually attended the exercise sessions. This is statistically safe from confounding but usually underestimates the true biological effect of the intervention. 2. **Per Protocol (PP) Analysis:** We analyze patients based on their actual attendance. While this looks at the "true" intervention, it is heavily confounded (e.g., highly motivated people might attend more *and* have naturally lower stress).

```
dat_iv <- read_csv('../Data/W7_IV.csv')

# 1. Intention-to-Treat (Conservative, unconfounded)
summary(lm(stress ~ group, data=dat_iv))

# 2. Per Protocol (Confounded by adherence behaviors)
summary(lm(stress ~ attendance, data=dat_iv))
```

7.3.2 IV Analysis using 2-Stage Least Squares

Rationale: IV analysis extracts the variation in `attendance` that is purely driven by the randomization `group` (the instrument), effectively purging the confounding noise from the model.

Code Explanation: We use the `ivreg()` function from the `AER` package. The formula `stress ~ attendance | group` is read as: model the outcome (`stress`) against the exposure (`attendance`), using the variable after the pipe (`group`) as the instrument. We also test the strength of the instrument using `anova()`. An F-statistic > 10 generally indicates a strong instrument. Because `group` is a randomized allocation, it safely meets the exclusion criteria.

```r
library(AER)

# Fit the Instrumental Variable regression
iv_model <- ivreg(stress ~ attendance | group, data=dat_iv)
summary(iv_model)

# Check instrument strength (F-statistic)
anova(glm(attendance ~ group, data=dat_iv), test='F')
```

List of abbreviations

95% CI 95% confidence interval
ACM all-cause mortality
ANOVA analysis of variance
BMI body mass index
BUN blood urea nitrogen
CI confidence interval
CIF cumulative incidence function
CSV comma separated value
Chol cholesterol
Cox PH Cox proportional hazards
Cr serum creatinine
DM diabetes mellitus
DPI dots per inch
GAM generalised additive model
GLM generalised linear model
HDL high-density lipoprotein
HR hazard ratio
HRQoL health-related quality of life
HTML hypertext markup language
IDE integrated development environment
IQR interquartile range
IRR incidence rate ratio
KM Kaplan-Meier estimator
LCI lower confidence interval
LDL low-density lipoprotein
MACE major adverse cardiovascular event
MPH Master's of Public Health
MRE minimal reproducible example
MW mental well-being
MWE minimal working example
NA not available
NB negative binomial
NaN not a number
OR odds ratio
PDF portable document format

RERI relative excess risk due to additive interaction
SD standard deviation
SE standard error
SIMD Scottish Index of Multiple Deprivation
TG triglycerides
TSV tab separated value
TXT text file document
UCI upper confidence interval
UK United Kingdom
VIF variance inflation factor
WHO World Health Organization

References

Bibliography

Anjali, S. (n.d.), 'Health test by blood dataset'. Accessed 2 June 2025.
URL: *https://www.kaggle.com/datasets/aravindpcoder/diabetes-dataset*

Buck, N. & McFall, S. (2012), 'Understanding society: design overview', *Longitudinal and Life Course Studies* **3**(1), 5–17.

Ho, F. K. & Cole, T. J. (2023), 'Non-linear predictor outcome associations', *BMJ medicine* **2**(1), e000396.

Holdaas, H., Fellström, B., Jardine, A. G., Holme, I., Nyberg, G., Fauchald, P., Grönhagen-Riska, C., Madsen, S., Neumayer, H.-H., Cole, E. et al. (2003), 'Effect of fluvastatin on cardiac outcomes in renal transplant recipients: a multicentre, randomised, placebo-controlled trial', *The Lancet* **361**(9374), 2024–2031.

Lau, B., Cole, S. R. & Gange, S. J. (2009), 'Competing risk regression models for epidemiologic data', *American journal of epidemiology* **170**(2), 244–256.

Stensrud, M. J. & Hernán, M. A. (2020), 'Why test for proportional hazards?', *Jama* **323**(14), 1401–1402.

The Scottish Government (2012), 'The Scottish Index of Multiple Deprivation 2012'.
URL: *https://www.gov.scot/binaries/content/documents/govscot/publications/statistics/2012/12/scottish-index-multiple-deprivation-2012-executive-summary/documents/scottish-index-multiple-deprivation-2012-executive-summary/scottish-index-multiple-deprivation-2012-executive-summary/govscot%3Adocument/00410895.pdf*

VanderWeele, T. J. (2009), 'On the distinction between interaction and effect modification', *Epidemiology* **20**(6), 863–871.